



Version 0.10.0

Kim Hammar

July 1, 2026



Swedish
Research
Council

Contents

1	The Cyber Security Learning Environment (CSLE)	4
1.1	What is CSLE?	4
1.2	Why CSLE?	5
1.3	Background	6
1.4	Resources	6
1.4.1	Documentation	6
1.4.2	Source code, Binaries, and Docker Images	6
1.4.3	Publications	8
1.5	Contributing	11
1.5.1	Getting Help	11
1.5.2	List of Contributors	11
1.6	License	12
1.7	Document structure	12
2	Architecture and Design	13
2.1	Overall Architecture	13
2.2	The Management System	14
2.3	The Metastore	16
2.4	The Emulation System	17
2.4.1	Emulating Physical Hosts	20
2.4.2	Emulating Physical Network Links	22
2.4.3	Emulating Physical Switches	23
2.4.4	Emulating Network Conditions	24
2.4.5	Emulating Client Populations	25
2.4.6	Emulating Attackers	26
2.4.7	Emulating Defenders	27
2.4.8	The Management Network	28
2.4.9	Monitoring Agents	29
2.4.10	Management Agents	34
2.4.11	Starting an Emulation Execution	38
2.5	The Simulation System	40
2.5.1	Simulation Environments	40
2.5.2	Numerical Algorithms	41
2.6	The Web Interface	43
2.6.1	The Policy Examination System	49
2.6.2	Implementation	51
2.6.3	The REST API	52
2.7	The Command-Line Interface (CLI)	58
3	Getting Started	62

3.1	Installing CSLE	62
3.1.1	Prerequisites	62
3.1.2	Installation Setup	63
3.2	Manual Installation of CSLE	65
3.2.1	Physical Resources	66
3.2.2	Installation setup	66
3.2.3	Installing the Metastore	70
3.2.4	Installing the Simulation System	73
3.2.5	Installing the Emulation System	74
3.2.6	Installing the Management System	78
3.3	Quick Start	86
3.4	Uninstalling CSLE	87
3.5	Operating CSLE	87
3.5.1	Listing the Available Commands in the CLI	88
3.5.2	Listing the State of CSLE	88
3.5.3	Starting, Stopping, and Resetting the Metastore	88
3.5.4	Starting and Stopping the REST API Server	89
3.5.5	Starting and Stopping Monitoring Systems	89
3.5.6	Starting and Stopping Emulation Executions	90
3.5.7	Access a Terminal in an Emulated Container	91
3.6	Examples	91
3.6.1	Data collection	91
3.6.2	System Identification	93
3.6.3	Strategy Training	94
3.6.4	Strategy Evaluation	95
4	Developer Guide	96
4.1	Development Conventions	96
4.1.1	General Principles	96
4.1.2	Structure of CSLE	97
4.1.3	Code Readability	98
4.1.4	Comments in Code	99
4.1.5	Unit and Integration Testing	100
4.1.6	How We Use Git	101
4.1.7	Continuous Integration	102
4.1.8	Bug Reports	102
4.1.9	Contribution flow	103
4.2	Installing development tools	104
4.3	Writing Documentation	108
4.4	Log files and Debugging	108
4.5	Tests	109
4.5.1	Unit Tests	109

4.5.2	Integration Tests	110
4.6	Static Code Analysis	112
4.7	Dependency Management	113
4.8	Release Management	117
5	How-to Guides and Tutorials	121
5.1	How to: Add Base Containers	121
5.2	How to: Add Derived Containers	122
5.3	How to: Add Emulation Configurations	123
5.4	How to: Add Simulation Configurations	124
5.5	How to: Update Monitoring Agents	125
5.6	How to: Add Numerical Algorithms	126
5.7	How to: Add Configuration Parameters	128
6	Frequently Asked Questions	129
A	Setup PyCharm for Remote Development	131
B	Useful Git Commands	136

1 The Cyber Security Learning Environment (CSLE)

This section contains an overview of the Cyber Security Learning Environment (CSLE), its purpose, origin, and organization.

1.1 What is CSLE?

CSLE is a framework for evaluating and developing reinforcement learning agents for control problems in cyber security. Everything from network emulation to simulation and learning in CSLE has been co-designed to provide an environment where it is possible to train and evaluate reinforcement learning agents for practical cyber security tasks.

CSLE is an opinionated framework based on a specific methodology for learning and evaluating security strategies for a given IT infrastructure (see Fig. 1). This methodology includes two systems: an emulation system and a simulation system.

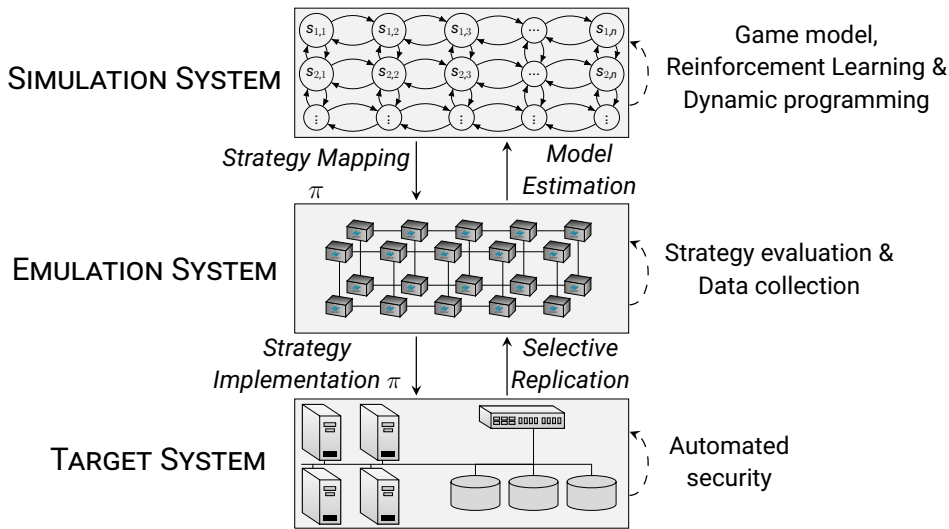


Figure 1: The methodology used to automatically find effective security strategies in CSLE.

The emulation system closely approximates the functionality of the target infrastructure and is used to run attack scenarios and defender responses. Such runs produce system measurements and logs, from which infrastructure statistics are estimated, which then are used to instantiate Markov decision processes (MDPs).

The simulation system is used to simulate the instantiated MDPs and to learn security strategies through reinforcement learning. Learned strategies

are extracted from the simulation system and evaluated in the emulation system.

Three benefits of this methodology are: (i) the emulation system provides a realistic environment to evaluate strategies; (ii) the emulation system allows evaluating strategies without affecting operational workflows on the target infrastructure; and (iii) the simulation system enables efficient and rapid learning of strategies.

1.2 Why CSLE?

As the ubiquity and evolving nature of cyber attacks is of growing concern to society, *automation* of security processes and functions has been recognized as an important part of the response to this threat. A promising approach to achieve this automation is *reinforcement learning*, which has proven effective in finding near-optimal solutions to control problems in several domains (e.g., robotics [39] and industry automation [42]), and is actively investigated as an approach to automated security (see survey [13]). While encouraging results have been obtained in this line of research, key challenges remain. Chief among them is narrowing the gap between the environment where the reinforcement learning agents are evaluated and a scenario playing out in a real system. Most of the results obtained so far are limited to simulation environments, and it is not clear how they generalize to practical IT infrastructures. Another limitation of prior research is the lack of common benchmarks and toolsets.

CSLE was developed to address precisely the above limitations. By using high-fidelity emulations, it narrows the gap between the evaluation environment and a real system, and by being open-source, it provides a foundation for further research to build on.

Recently, efforts to build similar frameworks as CSLE have started (see [13] for a review). Most notably, there is CYBERBATTLESIM by Microsoft [3], CYBORG by the Australian Department of Defence [59], YAWNING TITAN by the UK Defence Science and Technology Laboratory (DSTL) [1], CYGIL by Defence Research and Development Canada (DRDC) [43], NASIMEMU by the Czech Technical University in Prague [38], FARLAND, which is developed at USA's National Security Agency (NSA) [47], and CYBERWHEEL by the Oak Ridge national laboratory [49]. Some of these frameworks only include simulation systems and some of them include both simulation systems and emulation systems. In contrast to these frameworks, CSLE is fully open-source, includes both a simulation system and an emulation system, and has been thoroughly tested on an intrusion response use case (see the list of publications in §1.4.3).

1.3 Background

The development of CSLE started as a research project in 2020 at KTH Royal Institute of Technology, led by Prof. Rolf Stadler and Prof. Pontus Johnson. This research has been supported in part by the Swedish armed forces, the Defense Advanced Research Project Agency (DARPA) through the CASTLE program under Contract No. W912CG23C0029, and the Swedish research council through Contract No. 2024-06436. The research is conducted at the Division of Network and Systems Engineering (NSE) at KTH, the KTH Center for Cyber Defense and Information Security (CDIS), Arizona State University, the University of Melbourne, and Imperial College London. The project is currently maintained by Kim Hammar (kimham@kth.se).

1.4 Resources

Resources for getting started with CSLE are described below. §1.4.1 describes documentation resources, §1.4.2 describes where to find the source code and binaries of CSLE, and §1.4.3 describes scientific publications related to CSLE.

1.4.1 Documentation

Apart from the official release documentation (i.e., the document you are reading now), which is available in PDF format for each release, the latest documentation of CSLE can always be found at: <https://kim-hammar.github.io/csle/>. Video demonstrations of earlier versions of CSLE are available on YouTube:

- demonstration of v0.0.1: <https://www.youtube.com/watch?v=18P7MjPKNDg>.
- demonstration of v0.2.0: <https://www.youtube.com/watch?v=iE2KPmtIs2A&>.
- demonstration of the installation of v0.5.0: https://www.youtube.com/watch?v=l_g3sRJwwhc.
- demonstration of the LLM integration in v0.9.0: <https://www.youtube.com/watch?v=XXo4Y6LCWk4>.

1.4.2 Source code, Binaries, and Docker Images

The source code of CSLE is available on GITHUB: <https://github.com/Kim-Hammar/csle> and the DOCKER images are available on DOCKERHUB: <https://hub.docker.com>.

`com/u/kimham`. Further, binaries of the PYTHON libraries in CSLE are available on PYPI:

- <https://pypi.org/project/csle-base>
- <https://pypi.org/project/csle-collector>
- <https://pypi.org/project/csle-common>
- <https://pypi.org/project/csle-attacker>
- <https://pypi.org/project/csle-defender>
- <https://pypi.org/project/gym-csle-stopping-game>
- <https://pypi.org/project/gym-csle-apt-game>
- <https://pypi.org/project/gym-csle-cyborg>
- <https://pypi.org/project/csle-ryu>
- <https://pypi.org/project/csle-rest-api>
- <https://pypi.org/project/csle-agents>
- <https://pypi.org/project/csle-system-identification>
- <https://pypi.org/project/csle-cli>
- <https://pypi.org/project/csle-cluster>
- <https://pypi.org/project/gym-csle-intrusion-response-game>
- <https://pypi.org/project/csle-tolerance>
- <https://pypi.org/project/csle-cyborg>
- <https://pypi.org/project/csle-attack-profiler>

Lines of code CSLE consists of around 275,000 lines of PYTHON [63], 40,000 lines of JAVASCRIPT [7], 3000 lines of DOCKERFILES [46], 5400 lines of MAKE-FILE, 14000 lines of BASH, and 7600 lines of YAML. The lines of code can be counted by executing the following commands from the project root:


```

1 find . -name '*.py' | xargs wc -l
2 cd management-system/src; find . -name '*.js' | xargs wc -l
3 find . -name 'Dockerfile' | xargs wc -l
4 find . -name 'Makefile' | xargs wc -l
5 find . -name '*.sh' | xargs wc -l
6 find . -name '*.yaml' | xargs wc -l

```

Listing 1: Commands for counting the lines of code in CSLE.

1.4.3 Publications

CSLE is an open-source project mainly developed for research purposes. Below is a list of publications related to CSLE. Feel free to contact us to have your publication added to this list.

- *Online Identification of IT Systems through Active Causal Learning [28]*
 - **Published in:** Under review.
 - **Author:** Kim Hammar & Rolf Stadler.
- *Online Incident Response Planning under Model Misspecification through Bayesian Learning and Belief Quantization [17]*
 - **Published in:** ACM CCS AISeC 2025: 18th ACM Workshop on Artificial Intelligence and Security, Taipei, Taiwan.
 - **Author:** Kim Hammar & Tao Li.
- *Incident Response Planning Using a Lightweight Large Language Model with Reduced Hallucination [15]*
 - **Published in:** Network and Distributed System Security (NDSS) Symposium 2026.
 - **Author:** Kim Hammar, Tansu Alpcan, & Emil C. Lupu.
- *Adaptive Network Security Policies via Belief Aggregation and Rollout [30]*
 - **Published in:** Under review.
 - **Author:** Kim Hammar, Yuchao Li, Tansu Alpcan, Emil C. Lupu, & Dimitri Bertsekas.
- *Optimal Security Response to Network Intrusions in IT Systems [14]*

- **Published in:** Doctoral thesis in electrical engineering, KTH, School of Electrical Engineering and Computer Science (EECS).
 - **Author:** Kim Hammar.
- *Optimal Defender Strategies for CAGE-2 using Causal Modeling and Tree Search [16]*
 - **Published in:** Under review.
 - **Authors:** Kim Hammar, Neil Dhir, & Rolf Stadler.
- *Intrusion Tolerance as a Two-Level Game [23]*
 - **Published in:** International Conference on Decision and Game Theory for Security (GAMESEC 24).
 - **Authors:** Kim Hammar & Rolf Stadler.
- *Conjectural Online Learning with First-order Beliefs in Asymmetric Information Stochastic Games [44]*
 - **Published in:** IEEE Conference on Decision and Control (CDC'24).
 - **Authors:** Tao Li, Kim Hammar, Rolf Stadler, & Quanyan Zhu.
- *Intrusion Tolerance for Networked Systems through Two-Level Feedback Control [24]*
 - **Published in:** IEEE/IFIP Dependable Systems and Networks Conference (DSN'24).
 - **Authors:** Kim Hammar, Rolf Stadler.
- *Automated Security Response through Online Learning with Adaptive Conjectures [31]*
 - **Published in:** IEEE Transactions on Information Forensics and Security.
 - **Authors:** Kim Hammar, Tao Li, Rolf Stadler, & Quanyan Zhu.
- *IT Intrusion Detection Using Statistical Learning and Testbed Measurements [64]*
 - **Published in:** IEEE/IFIP Network Operations and Management Symposium 2024.
 - **Authors:** Xiaoxuan Wang & Rolf Stadler.

- *Scalable Learning of Intrusion Response through Recursive Decomposition [26]*
 - **Published in:** International Conference on Decision and Game Theory for Security 2023 (GAMESEC 23).
 - **Authors:** Kim Hammar & Rolf Stadler.
- *Learning Near-Optimal Intrusion Responses Against Dynamic Attackers [26]*
 - **Published in:** IEEE Transactions on Network and Service Management 2023.
 - **Authors:** Kim Hammar & Rolf Stadler.
- *Digital Twins for Security Automation [20]*
 - **Published in:** NOMS 2023 IEEE/IFIP Network Operations and Management Symposium.
 - **Authors:** Kim Hammar & Rolf Stadler.
- *An Online Framework for Adapting Security Policies in Dynamic IT Environments [19]*
 - **Published in:** International Conference on Network and Service Management (CNSM 2022).
 - **Authors:** Kim Hammar & Rolf Stadler.
- *Learning Security Strategies through Game Play and Optimal Stopping [27]*
 - **Published in:** Proceedings of the ML4CYBER workshop ICML 2022.
 - **Authors:** Kim Hammar & Rolf Stadler.
- *Intrusion Prevention Through Optimal Stopping [22]*
 - **Published in:** IEEE Transactions on Network and Service Management, special issue on recent advances in security, 2022.
 - **Authors:** Kim Hammar & Rolf Stadler.
- *A System for Interactive Examination of Learned Security Policies [18]*
 - **Published in:** NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium.
 - **Authors:** Kim Hammar & Rolf Stadler.

- *Learning Intrusion Prevention Policies through Optimal Stopping [25]*
 - **Published in:** International Conference on Network and Service Management (CNSM 2021).
 - **Authors:** Kim Hammar & Rolf Stadler.
- *Finding Effective Security Strategies through Reinforcement Learning and Self-Play [21]*
 - **Published in:** International Conference on Network and Service Management (CNSM 2020).
 - **Authors:** Kim Hammar & Rolf Stadler.

1.5 Contributing

To anyone interested in making CSLE better, many improvements need to be made. A list of tasks is available at <https://github.com/Kim-Hammar/csle/issues>. Contributors are also welcome to report bugs and request new features. To report a bug or make a feature request, use GITHUB issues¹. To submit a patch or a new feature, use GITHUB pull requests². Before you open a pull request, make sure that your code changes make sense. In particular, read the developer guide in §4 and make sure that your code changes follow the guidelines.

1.5.1 Getting Help

To contact the maintainers of CSLE, use GITHUB issues or email. The GITHUB issues page is reviewed regularly by the maintainers.

1.5.2 List of Contributors

The following people have contributed to the development of CSLE (contact kimham@kth.se if your name should be on the list but is not):

- Kim Hammar, creator, and main developer.
- Rolf Stadler, technical advisor.
- Pontus Johnson, technical advisor.

¹<https://docs.github.com/en/issues/tracking-your-work-with-issues/about-issues>

²<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>

- Tansu Alpcan, technical advisor.
- Emil Lupu, technical advisor.
- Forough Shahab Samani, software development, testing, and documentation.
- Antonio Frederico Nesti Lopes, software development.
- Jakob Stymne, software development.
- Arvid Lagerqvist, software development.
- Nils Forsgren, software development.
- Bengt Roland Pappila, software development.
- Yu Hu, software development.
- Yan Wang, software development.
- Aws Jaber, software development.
- Xiaoxuan Wang, software development and testing.
- @arunderor, software development.

1.6 License

CSLE is released under the Creative Commons Attribution-ShareAlike (CC BY-SA) 4.0 international license³.

1.7 Document structure

The remainder of this document is structured as follows. The following section, §2, describes the architecture and design of CSLE. The subsequent section contains installation instructions (§3). §4 and §5 contain the developer guide and usage tutorials, respectively. Lastly, §6 contains answers to frequently asked questions.

³<https://creativecommons.org/licenses/by-sa/4.0/>

2 Architecture and Design

In this section, we describe the architecture and design of CSLE. We first describe the overall architecture (§2.1), and then we describe each of the main components: the management system, the metastore, the emulation system, the simulation system, the web interface, and the command-line interface (§2.2-2.7).

2.1 Overall Architecture

The overall architecture of CSLE is shown in Fig. 2.

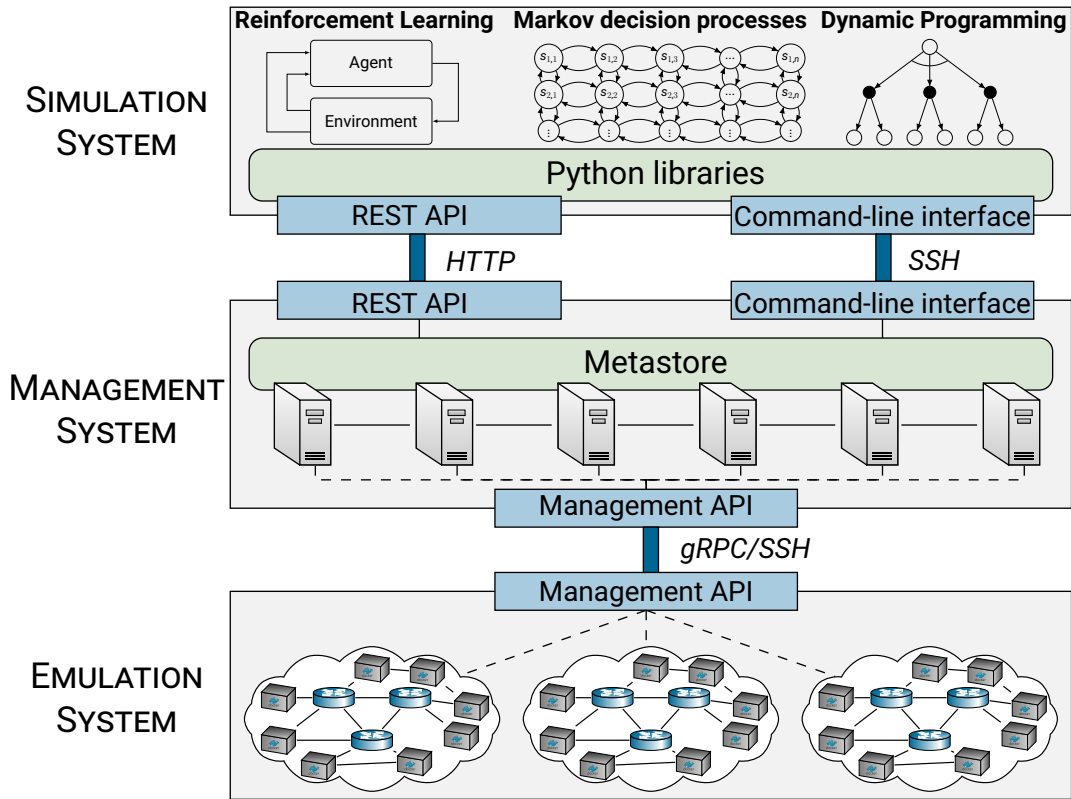


Figure 2: Architecture of CSLE; it consists of an emulation system (bottom part), a management system (middle part), and a simulation system (upper part); the three systems are connected through APIs and a shared metastore.

In the middle of Fig. 2 is the management system, which orchestrates the overall execution of the framework and stores metadata in a distributed database called the metastore. It has three interfaces. One interface is a

GRPC API to the emulation system, which emulates IT infrastructures using hardware and network virtualization [10]. The other two interfaces—a web interface and a CLI—are to the simulation system, which simulates Markov decision processes and runs reinforcement learning workloads.

Users access CSLE in three ways: (1) through software functions using PYTHON libraries of the simulation system; (2) through a web browser using the web interface; and (3) through a terminal using the CLI.

Details about each component of CSLE are given in the sections below (§2.2-2.7).

2.2 The Management System

The management system is the central component of CLSE and manages the overall execution of the framework. It is a distributed system that consists of $N \geq 1$ physical servers connected through an IP network. One of the servers is designated to be the “leader,” and the other servers are “workers”. Workers can perform local management actions but not actions that affect the overall system state. These actions are routed to the leader, who applies them sequentially to ensure consistent updates to the system state.

The leader is elected using the leader election protocol in [48], which uses the metastore for coordination (see Fig. 3). A new leader is elected by a quorum whenever the current leader fails or becomes unresponsive. This means that the management system tolerates up to $N/2 - 1$ failing servers.

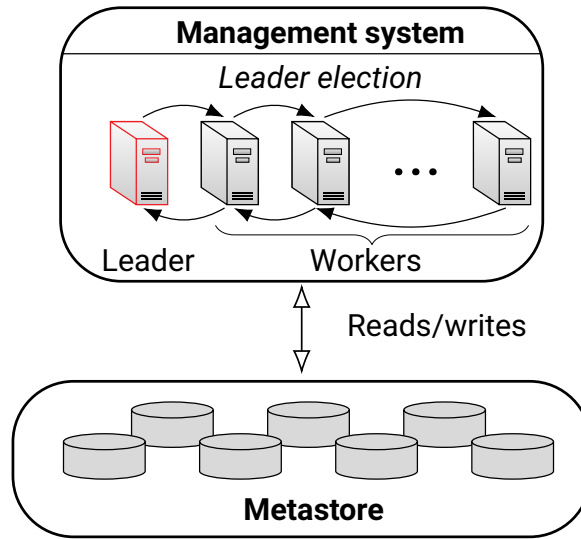


Figure 3: Architecture diagram of the management system; it is a distributed system with a designated leader, which is elected using a leader election algorithm that uses the metastore for coordination.

Each server of the management system executes the following services: (1) an instance of the emulation system; (2) a replica of the metastore; (3) an instance of the simulation system; (4) an HTTP server; and (5), a set of monitoring services, namely PROMETHEUS [54], GRAFANA [12], CADVISOR [9], and NODE EXPORTER [53] (see Fig. 4).

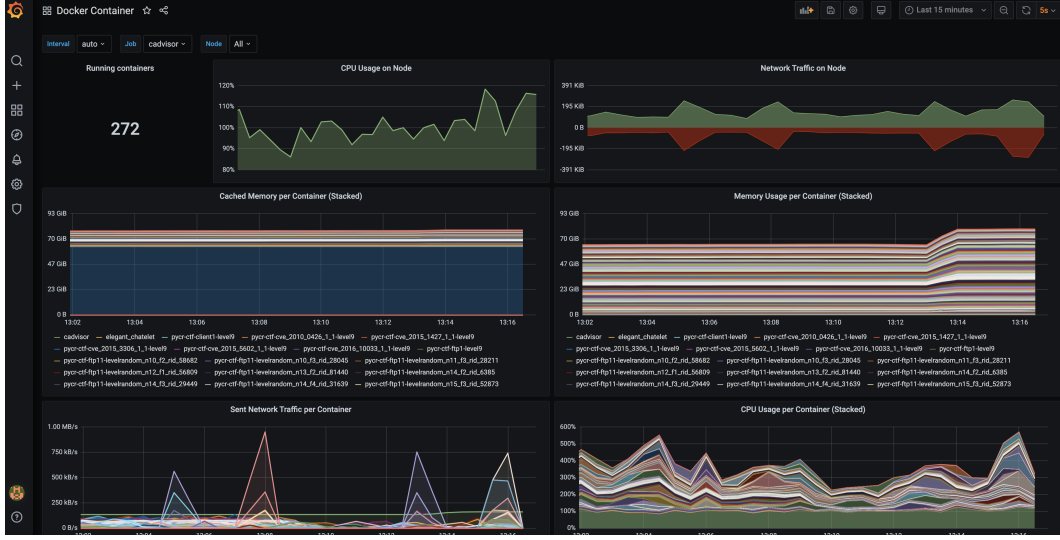


Figure 4: A GRAFANA dashboard for monitoring a physical server of the management system; CADVISOR and NODE EXPORTER are used to push system metrics to PROMETHEUS, which stores the metrics in a time-series database, whose data is visualized with GRAFANA dashboards.

2.3 The Metastore

The metastore is a distributed relational database based on POSTGRESQL [62] and CITUS [5]. It stores metadata and management information about CSLE. This data includes metadata about emulated IT infrastructures, metadata about reinforcement learning workloads, metadata about management users, metadata about simulation environments, and management information about virtual containers and networks (see Table 1 for a complete list).

The metastore is divided into N replicas. Each physical server of the management system stores one replica. Reads and writes to the metastore are distributed evenly across all replicas to maximize throughput. A quorum-based two-phase commit (2PC) scheme is used to coordinate writes and achieve consensus among replicas. This means that the metastore tolerates $N/2 - 1$ failing replicas.

Table name	Description	Columns
emulations	Metadata about emulations.	id, name, config
emulations_traces	Attack and system traces.	id, emulation_name, trace
emulation_statistics	Count statistics of system metrics.	id, emulation_name, statistics
simulation_traces	Simulation traces.	id, gym_env, trace
emulation_simulation_traces	Mapping between emulation & simulation traces.	id, emulation_trace, simulation_trace
emulation_images	Pictures of emulation topologies.	id, emulation_name, image
simulations	Metadata about simulations.	id, name, config
experiment_executions	Metadata about experiments.	id, execution, simulation_name, emulation_name
simulation_images	Pictures of simulations.	id, simulation_name, image
multi_threshold_stopping_policies	Threshold-based policies.	id, policy, simulation_name
training_jobs	Training jobs.	id, config, simulation_name, emulation_name, pid
system_identification_jobs	System identification jobs.	id, config, emulation_name, pid
ppo_policies	PPO policies.	id, policy, simulation_name
system_models	System models.	id, emulation_name, system_model
data_collection_jobs	Data collection jobs.	id, config, emulation_name, pid
gaussian_mixture_system_models	Gaussian mixture system models.	id, model, emulation_name, emulation_statistic_id
tabular_policies	Tabular policies.	id, policy, simulation_name
alpha_vec_policies	Alpha vector policies.	id, policy, simulation_name
dqn_policies	DQN policies.	id, policy, simulation_name
fnw_softmax_policies	Feed-forward neural network policies.	id, policy, simulation_name
vector_policies	Vector policies.	id, policy, simulation_name
emulation_executions	Emulation executions.	ip_first_octet, emulation_name, info
empirical_system_models	Empirical system models.	id, model, emulation_name, emulation_statistic_name
mcmc_system_models	MCMC system models.	id, model, emulation_name, emulation_statistic_name
gp_system_models	Gaussian process system models.	id, model, emulation_name, emulation_statistic_id
management_users	Management users.	id, username, password, email, first_name, etc.
session_tokens	Session tokens.	token, timestamp, username
traces_datasets	Traces datasets.	id, name, description, etc.
statistics_datasets	Statistics datasets.	id, name, description, etc.
linear_threshold_stopping_policies	Linear threshold policies.	id, name, description, etc.

Table 1: Metastore tables in version 0.10.0 of CSLE.

2.4 The Emulation System

The emulation system uses a virtualization layer provided by DOCKER [46] containers and virtual networks to emulate IT infrastructures. An emulated IT infrastructure is defined by an *emulation configuration*, which includes the properties listed in Table 2. The set of configurations supported by the emulation system is stored in the metastore and can be seen as a *configuration space*, which defines the class of infrastructures that can be emulated (see Fig. 5).

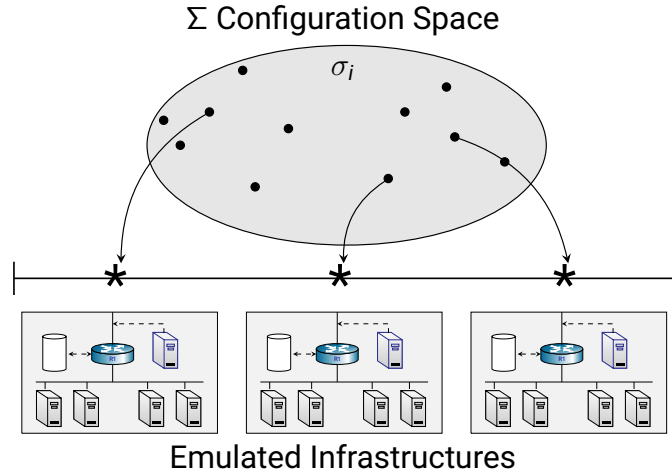


Figure 5: The configuration space of the emulation system defines the class of infrastructures that the emulation system can emulate.

Configuration property	Description
name	The name of the emulation.
containers_config	Configuration of each container (ip, hostname, resources, etc.).
users_config	Configuration of user accounts on each container.
flags_config	Configuration of flags on each container.
vuln_config	Configuration of vulnerabilities on each container.
topology_config	Configuration of the emulation's network topology.
traffic_config	Configuration of traffic generators and client population.
resources_config	Configuration of physical resources (CPU, memory, etc.).
services_config	Configuration of services running on each container.
descr	Description of the emulation.
static_attacker_sequences	Configuration of pre-defined attacker sequences.
ovs_config	Configuration of Open vSwitches (virtual OPENflow switches).
sdn_controller_config	Configuration of SDN controllers.
host_manager_config	Configuration of host managers.
snort_ids_manager_config	Configuration of SNORT IDS managers.
ossec_ids_manager_config	Configuration of OSSEC IDS managers.
docker_stats_manager_config	Configuration of DOCKER stats managers.
beats_config	Configuration of FILEBEATS, PACKETBEATS, etc.
elk_config	Configuration of the ELK stack in the management network.
level	Level of the emulation.
version	Version of the emulation.
execution_id	ID of the execution that the emulation belongs to (if any).
csle_collector_version	Version of the csle-collector library.
csle_ryu_version	Version of the csle-ryu library.

Table 2: Properties of an emulation configuration.

Users of CSLE can create new emulation configurations and insert them into the metastore. CSLE also includes a set of pre-installed configurations

(see Table 3). One of these pre-installed configurations is `csle-level19-003` (as an example), whose topology is shown in Fig. 6 and whose configuration is listed in Table 4.

Emulation configuration	Description
<code>csle-level11-0.10.0</code>	Emulation with 7 components, 3 flags, password vulnerabilities, no IDS.
<code>csle-level12-0.10.0</code>	Emulation with 13 components, 6 flags, password vulnerabilities, no IDS.
<code>csle-level13-0.10.0</code>	Emulation with 34 components, 6 flags, password vulnerabilities, no IDS.
<code>csle-level14-0.10.0</code>	Emulation with 7 components, 3 flags, password vulnerabilities, IDS.
<code>csle-level15-0.10.0</code>	Emulation with 13 components, 6 flags, password vulnerabilities, IDS.
<code>csle-level16-0.10.0</code>	Emulation with 34 components, 6 flags, password vulnerabilities, IDS.
<code>csle-level17-0.10.0</code>	Emulation with 7 components, 3 flags, password & RCE vulnerabilities, IDS.
<code>csle-level18-0.10.0</code>	Emulation with 13 components, 6 flags, password & RCE vulnerabilities, IDS.
<code>csle-level19-0.10.0</code>	Emulation with 34 components, 6 flags, password & RCE vulnerabilities, IDS.
<code>csle-level110-0.10.0</code>	Emulation with 16 components, 12 flags, password & RCE vulnerabilities, IDS.
<code>csle-level111-0.10.0</code>	Emulation with 36 components, 6 flags, password & RCE vulnerabilities, IDS.
<code>csle-level112-0.10.0</code>	Emulation with 7 components, 3 flags, password & RCE vulnerabilities, IDS, SDN.
<code>csle-level113-0.10.0</code>	Emulation with 64 components, 6 flags, password & RCE vulnerabilities, IDS, SDN.
<code>csle-level114-0.10.0</code>	Emulation with 17 components, 12 flags, password & RCE vulnerabilities, IDS, SDN.

Table 3: Emulation configurations in version 0.10.0 of csLE.

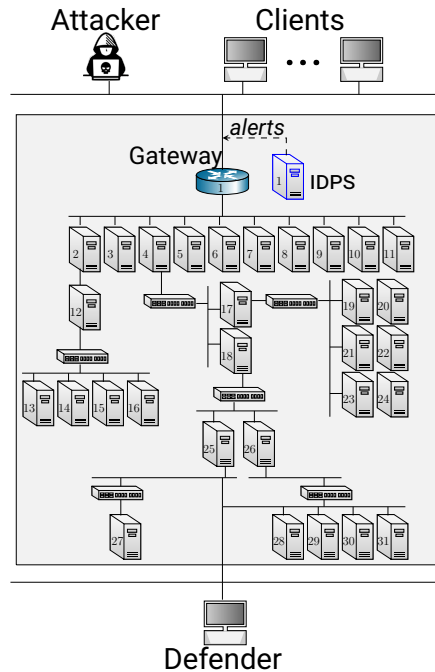


Figure 6: Topology of the emulation configuration `csle-level19-0.10.0`.

<i>ID (s)</i>	<i>OS:Services:Exploitable Vulnerabilities</i>
N_1	UBUNTU20:SNORT(community ruleset v2.9.17.1),SSH:-
N_2	UBUNTU20:SSH,HTTP ERL-PENGINE,DNS:SSH-pw
N_4	UBUNTU20:HTTP flask,TELNET,SSH:TELNET-pw
N_{10}	UBUNTU20:FTP,MONGODB,SMTP,TOMCAT,TS3,SSH:FTP-pw
N_{12}	Debian10.2:TS3,TOMCAT,SSH:CVE-2010-0426,SSH-pw
N_{17}	WHEEZY:APACHE2,SNMP,SSH:CVE-2014-6271
N_{18}	Deb10.2:IRC,APACHE2,SSH:SQL injection
N_{22}	Deb10.2:PROFTPD,SSH,APACHE2,SNMP:CVE-2015-3306
N_{23}	Deb10.2:APACHE2,SMTP,SSH:CVE-2016-10033
N_{24}	Deb10.2:SSH:CVE-2015-5602,SSH-pw
N_{25}	Deb10.2: ELASTICSEARCH,APACHE2,SSH,SNMP:CVE-2015-1427
N_{27}	Deb10.2:Samba,NTP,SSH:CVE-2017-7494
N_3, N_{11}, N_5-N_9	UBUNTU20:SSH,SNMP,POSTGRESQL,NTP:-
$N_{13-16}, N_{19-21}, N_{26}, N_{28-31}$	UBUNTU20:NTP, IRC, SNMP, SSH, POSTGRESQL:-

Table 4: Configuration of the emulation configuration `csle-level9-0.10.0`, whose topology is shown in Fig. 6.

An *emulation execution* consists of a set of running containers and virtual networks, which are defined by an associated *emulation configuration*.

Creating an execution involves two main tasks of the emulation system. The first task is to replicate relevant parts of the physical infrastructure that is emulated, such as physical resources, operating systems, network interfaces, and network conditions. The second task is instrumenting the emulated infrastructure with monitoring and management capabilities. Since the emulation execution will be used to evaluate reinforcement learning agents, it must be possible to monitor system metrics in real-time and to perform control actions prescribed by reinforcement learning agents.

In the following sections (§2.4.1-2.4.7), we describe how the emulation system emulates physical hosts, network links, switches, network conditions, client populations, attackers, and defenders, respectively. In the subsequent three sections, we describe the monitoring and management capabilities of the emulation system (§2.4.8-2.4.10). Lastly, in §2.4.11, we describe the process of creating an emulation execution from an emulation configuration.

2.4.1 Emulating Physical Hosts

Physical hosts are emulated with DOCKER containers. A container is a lightweight executable package that includes everything needed to emulate the host: a runtime system, code, system tools, system libraries, and configurations. Multiple containers can run on the same physical host and share the same

operating system kernel, in which case their resources and processes are isolated using cgroups⁴ (see Fig. 7).

The CSLE DOCKER images are listed in Table 5 and include images for emulating application servers, Software-Defined Networking (SDN) controllers, clients, attackers, Intrusion Detection and Prevention Systems (IDPSS), and storage systems. Detailed configuration of each image is available at: <https://hub.docker.com/r/kimham/>.

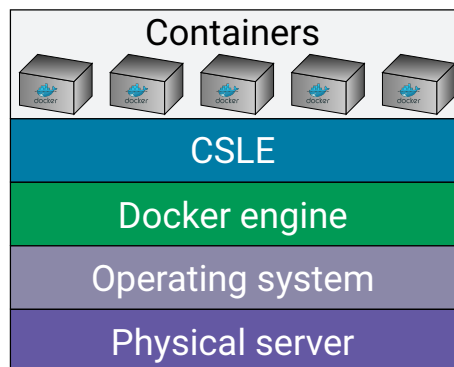


Figure 7: Software and hardware stack of a physical server that runs CSLE; closest to the hardware is the operating system; on top of the operating system is the DOCKER engine, which provides the base for running CSLE and virtual containers.

⁴<https://man7.org/linux/man-pages/man7/cgroups.7.html>

<i>Name</i>	<i>Description</i>	<i>OS</i>
cve_2010_0426_1	An image with the CVE-2010-0426 vulnerability.	Debian:Deb10.2
cve_2015_1427_1	An image with the CVE-2015-1427 vulnerability.	Debian:Deb10.2
cve_2015_3306_1	An image with the CVE-2015-3306 vulnerability.	Debian:Deb10.2
cve_2015_5602_1	An image with the CVE-2015-5602 vulnerability.	Debian:Deb10.2
cve_2016_10033_1	An image with the CVE-2016-10033 vulnerability.	Debian:Deb10.2
hacker_kali_1	An image with tools for penetration testing.	Kali
samba_1	An image with the CVE-2017-7494 vulnerability.	Debian:Deb10.2
samba_2	An image with the CVE-2017-7494 vulnerability.	Debian:Deb10.2
shellshock_1	An image with the CVE-2014-6271 vulnerability.	Debian:WHEEZY
sql_injection_1	An image with SQL-injection vulnerability.	Debian:10.2
ftp_1	An image with an FTP server.	UBUNTU:14
ftp_2	An image with an FTP server.	UBUNTU:14
ssh_1	An image with a SSH server.	UBUNTU:20
ssh_2	An image with a SSH server.	UBUNTU:20
ssh_3	An image with a SSH server.	UBUNTU:20
telnet_1	An image with a TELNET server.	UBUNTU:20
telnet_2	An image with a TELNET server.	UBUNTU:20
telnet_3	An image with a TELNET server.	UBUNTU:20
honeypot_1	An image with various honeypot services.	UBUNTU:20
honeypot_2	An image with various honeypot services.	UBUNTU:20
router_1	An image with the SNORT IDPS.	UBUNTU:20
router_2	An image with the SNORT IDPS.	UBUNTU:20
client_1	A blank image.	UBUNTU:20
blank_1	A blank image.	UBUNTU:20
blank_2	A blank image.	UBUNTU:22
blank_3	A blank image.	UBUNTU:24
pengine_exploit_1	An image with the pengine exploit.	UBUNTU:20
ovs_1	An image with the OVS virtual switch.	UBUNTU:20
ryu_1	An image with the RYU SDN controller.	UBUNTU:20
elk_1	An image with the ELK stack.	UBUNTU:20
kafka_1	An image with KAFKA.	UBUNTU:20
spark_1	An image with a SPARK server.	UBUNTU:22

Table 5: DOCKER images in version 0.10.0 of CSLE.

2.4.2 Emulating Physical Network Links

Network connectivity is emulated by virtual links implemented by Linux bridges⁵. Network isolation between emulated hosts and emulated switches on the same physical host is achieved through network namespaces⁶, which create logical copies of the physical host's network stack.

Each emulation execution has a dedicated virtual subnetwork with the

⁵<https://www.kernel.org/doc/html/v5.17/networking/bridge.html>

⁶https://man7.org/linux/man-pages/man7/network_namespaces.7.html

subnet mask:

```
executionID.emulationID.0.0/16
```

where `executionID` identifies the execution and `emulationID` identifies the emulation configuration. This addressing scheme allows several executions of the same emulation configuration to execute concurrently on the same physical servers without overlapping address spaces.

In the case that an emulated network spans multiple physical servers of the management system, VXLAN connections are used. These connections tunnel the emulated network traffic over the physical network (see Fig. 8). In other words, the physical network that connects the servers of the management system provides a substrate network, on top of which the emulation system deploys virtual networks.

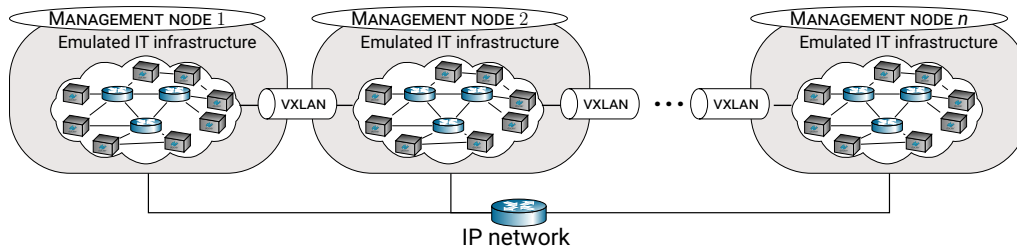


Figure 8: Architecture diagram of a distributed emulation execution; servers of the management system are connected through a physical IP network, over which virtual networks are overlayed using VXLAN tunnels.

2.4.3 Emulating Physical Switches

Physical switches are emulated with DOCKER containers that run Open vSwitch [52] (ovs) version 2.16 (see Fig. 9). The emulated switches connect to an SDN controller using the OPENflow protocol version 1.3 [45] over a secure TLS tunnel. The SDN controller is emulated by a container in the management network (see §2.4.8 for more details about the management network). (Since the switches are programmed through flow tables, they can act either as classical L2 switches or as routers, depending on the flow table configurations.)

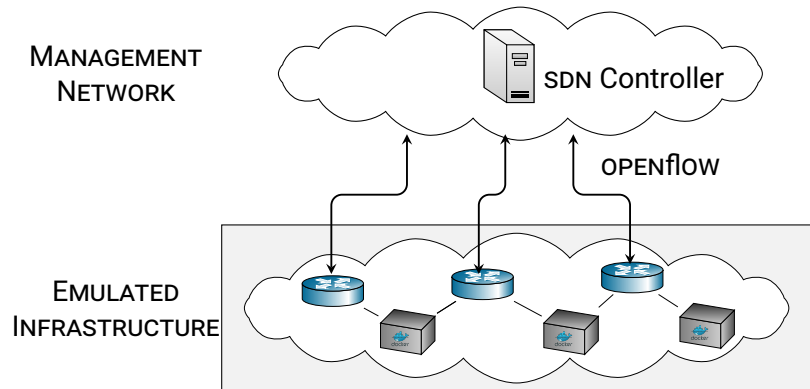


Figure 9: Physical switches in CSLE are emulated as Open vSwitch (ovs) switches [52]; each emulated switch runs the OPENflow protocol and can be configured via a Software-Defined Networking (SDN) controller.

2.4.4 Emulating Network Conditions

Network conditions of virtual links are configured using the NetEm module in the Linux kernel [35], which allows fine-gained configuration of bit rates, packet delays, packet loss probabilities, jitter, and packet reordering probabilities (see Fig. 10).

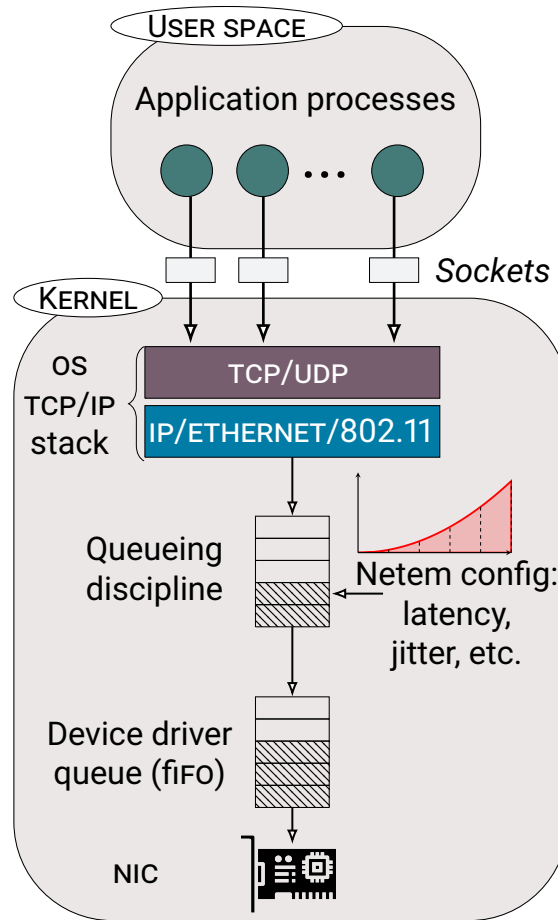


Figure 10: Method to emulate network conditions with the Netem module in the Linux kernel; Netem uses a dedicated queueing discipline in the Linux networking stack and emulates bit rates, latencies, packet loss probabilities, packet reordering probabilities, and jitter according to a predefined configuration.

2.4.5 Emulating Client Populations

Client populations in CSLE are emulated by processes that run inside DOCKER containers and interact with emulated hosts through various network protocols, e.g., HTTP, SSH, and DNS. The clients select functions from the list in [Table 6](#) according to some probability distribution.

<i>Function</i>	<i>Description</i>
HTTP	Download web pages and use REST APIs.
SSH	Connect to emulated components using SSH.
SNMP	Perform network management operations through SNMP.
ICMP	Ping emulated components.
IRC	Communicate with IRC servers.
POSTGRESQL	Communicate with POSTGRESQL servers.
FTP	Download and upload files to FTP servers.
DNS	Make DNS queries.
TELNET	Connect to emulated components using TELNET.

Table 6: Network functions that emulated clients may invoke.

Client arrivals are emulated by a Poisson process with exponentially distributed service times (see Fig. 11). This process may be stationary or non-stationary, e.g., it may be sine-modulated to model periodic load patterns. The duration of a time step in an emulation is defined by its configuration; a typical duration is 30 seconds.

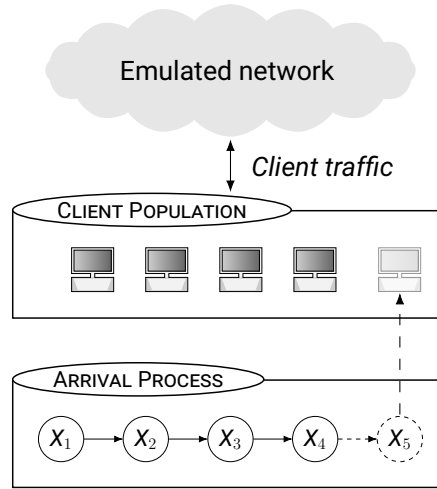


Figure 11: Clients are emulated by processes that invoke network functions of emulated hosts; client arrivals and service times are emulated by a Poisson process with exponentially distributed service times.

2.4.6 Emulating Attackers

Attackers are emulated by automated programs that select actions from a pre-defined set. The actions are selected according to an *attacker strategy*, which may depend on the system metrics of the emulation. (These metrics

are collected in real-time by monitoring agents (see §2.4.9 for details).). The attacker commands supported in version 0.10.0 of CSLE are listed in Table 7.

<i>Type</i>	<i>Actions</i>
Reconnaissance	TCP-SYN scan, UDP port scan, TCP NULL scan, TCP XMAS scan, TCP FIN scan, ping-scan, TCP connection scan, TCP-xmas-tree scan, OS-detection scan "Vulscan" vulnerability scanner "Vulners" vulnerability scanner NIKTO web scan, masscan firewall walk, HTTP-enum scan finger scan
Brute-force attack	TELNET, SSH, FTP, CASSANDRA, IRC, MONGODB, MYSQL, SMTP, POSTGRES
Exploit	CVE-2017-7494, CVE-2015-3306, CVE-2010-0426, CVE-2015-5602, CVE-2014-6271, CVE-2016-10033 CVE-2015-1427, SQL Injection

Table 7: Attacker commands supported in version 0.10.0 of CSLE.

2.4.7 Emulating Defenders

Defender actions are emulated by executing system commands on emulated hosts. These commands are invoked through a management API based on GRPC [10] and SSH (see §2.4.10). Similar to the attacker actions, the defender actions are prescribed by a *defender strategy*, which may depend on system metrics collected by monitoring agents (see §2.4.9). The defender actions supported in version version 0.10.0 of CSLE are listed in Table 8.

<i>Index</i>	<i>Action</i>
1	Revoke user certificates.
2	Blacklist IPs.
3 – 37	Drop traffic that generates IDPS alerts of priority 1 – 34.
38	Block gateway.
39	Server migration between different zones of the infrastructure.
40	Traffic redirect from one server to another.
41	Server isolation.
42	Deploying new security functions (e.g., firewalls and IDPSs).
43	Server shutdown.

Table 8: Defender commands supported in version 0.10.0 of CSLE.

2.4.8 The Management Network

Each emulation execution has a dedicated virtual subnetwork, called the *management network*, which spans the following address space:

`executionID.emulationID.253.0/24`

This network interfaces directly with the management system and also has an interface to each emulated component. The purpose of this network is to connect management systems to the emulated devices, e.g., storage systems, monitoring systems, and SDN controllers (see Fig. 12). The reason for using a dedicated management network instead of carrying management traffic on the network that carries device-to-device traffic is to avoid interference and to simplify control of the execution [4].

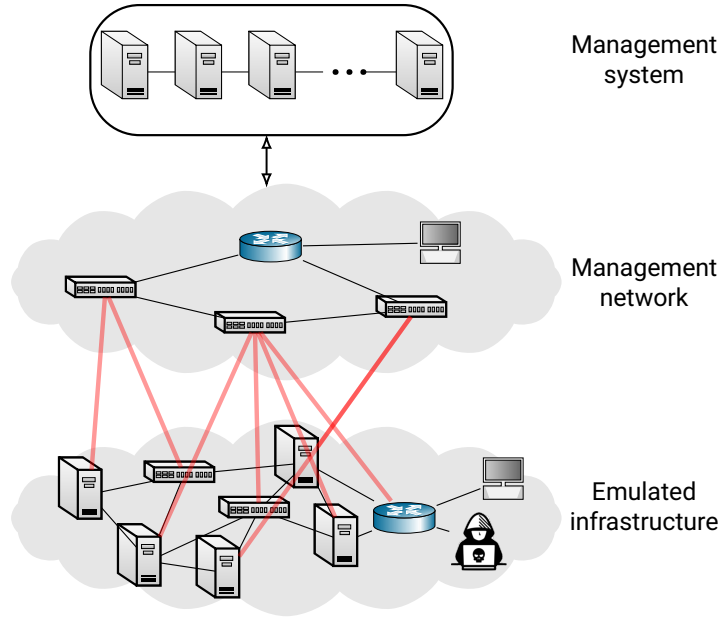


Figure 12: The management network is a dedicated network used for the operation of the emulated infrastructure.

2.4.9 Monitoring Agents

Each emulated device runs a *monitoring agent* that reads the local metrics of the device and pushes those metrics to a distributed KAFKA queue [40], which is deployed inside the management network (see Fig. 13). The metrics collected at every time step are listed in Tables 11-13.

The data in the KAFKA queue is organized in a set of *topics* (see Table 9), each of which stores a sequence of records in a first-in-first-out (fifO) order for a pre-determined time window. These topics are distributed over a set of partitions, each of which is stored on a server in the KAFKA cluster.

The topics are consumed by a set of data pipelines implemented with SPARK [65], which process the data and write the results to PRESTO [58] and ELASTICSEARCH [11] for persistent storage and search, respectively. These storage systems are then consumed by various downstream applications, e.g., dashboard applications and machine learning applications. The processed data is also input to reinforcement learning agents, which use it to decide on control actions.

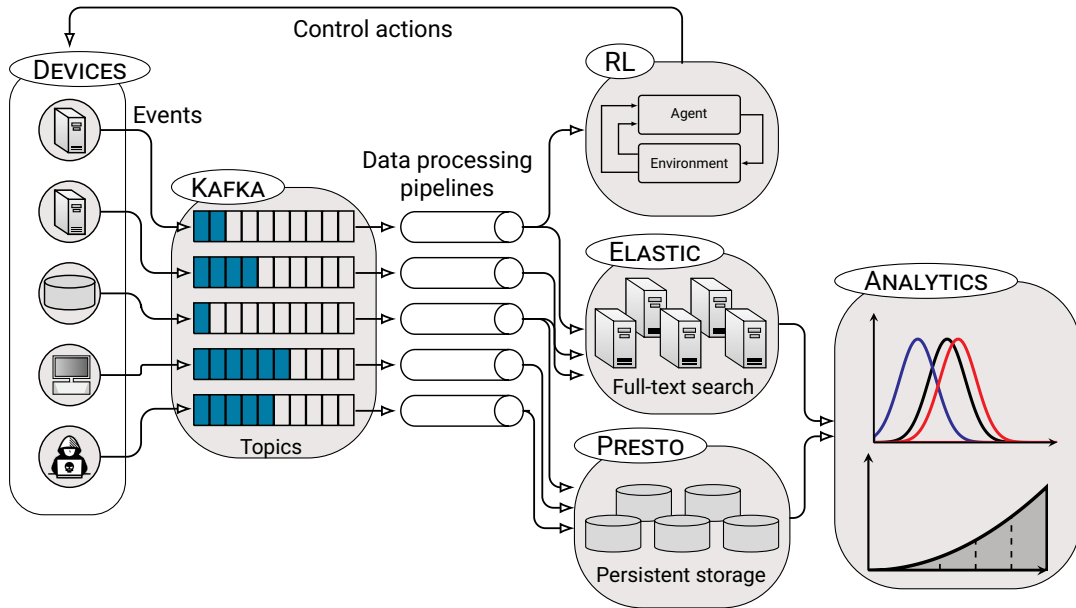


Figure 13: The monitoring system of CSLE; monitoring agents running on emulated devices push metric data to a distributed KAFKA queue; the data in this queue is consumed by data pipelines that process the data and write to PRESTO and ELASTICSEARCH for persistent storage and search, respectively; the processed data is used by Reinforcement Learning (RL) agents to decide on control actions, and by downstream applications to create dashboards and statistical models.

Topic	Description
client_population	Statistics of the client population (e.g., number of clients per time step).
snort_ids_log	Information about SNORT IDS alerts per time-step.
ossec_ids_log	Information about OSSEC IDS alerts per time-step.
host_metrics	Information about system metrics per host.
docker_stats	Resource statistics of docker containers.
docker_host_stats	Resource statistics of host servers.
openflow_flow_stats	Statistics of flows.
openflow_port_stats	Statistics of ports.
openflow_flow_agg_stats	Aggregate flow statistics.
avg_openflow_flow_stats_per_switch	Aggregate flow statistics per switch.
avg_openflow_port_stats_per_switch	Aggregate port statistics per switch.
attacker_actions	Attacker actions in the emulation.
defender_actions	Defender actions in the emulation.

Table 9: KAFKA topics in version 0.10.0 of CSLE.

<i>Metric</i>	<i>Description</i>
num_clients	Number of active clients.
clients_arrival_rate	Arrival rate of clients.
pids	Number of pids per container.
cpu_percent	CPU utilization per container.
mem_current	The total memory usage per container.
mem_total	The memory limit per container.
mem_percent	The memory utilization per container.
blk_read	Number of blocks read per container.
blk_write	Number of blocks written per container.
net_rx	Number of received network bytes per container.
net_tx	Number of transmitted network bytes per container.
avg_pids	Average number of pids.
avg_cpu_percent	Average CPU utilization.
avg_mem_current	Average total memory usage.
avg_mem_total	Average memory limit.
avg_mem_percent	Average memory utilization.
avg_blk_read	Average number of blocks read.
avg_blk_write	Average number of blocks written.
avg_net_rx	Average number of received network bytes.
avg_net_tx	Average number of transmitted network bytes.
num_logged_in_users	Number of logged-in users per container.
num_failed_login_attempts	Number of failed logins per container.
num_open_connections	Number of open connections per container.
num_login_events	Number of login events per container.
num_processes	Number of processes per container.
num_users	Number of users per container.
avg_num_logged_in_users	Average number of logged-in users per container.
avg_num_failed_login_attempts	Average number of failed logins per container.
avg_num_open_connections	Average number of open connections per container.
avg_num_login_events	Average number of login events per container.
avg_num_processes	Average number of processes per container.
avg_num_users	Average number of users per container.

Table 10: Metrics collected per time-step by the monitoring system in version 0.10.0 of CSLE (1/4).

<i>Metric</i>	<i>Description</i>
defender_action_id	Defender action ID.
defender_action_name	Defender action name.
defender_action_cmds	List of defender action commands.
defender_action_type	Defender action type.
defender_action_descr	Defender action description.
defender_action_outcome	Defender action outcome.
defender_action_execution_time	Defender action execution time.
attacker_action_id	Attacker action ID.
attacker_action_name	Attacker action name.
attacker_action_cmds	List of attacker action commands.
attacker_action_type	Attacker action type.
attacker_action_descr	Attacker action description.
attacker_action_outcome	Attacker action outcome.
attacker_action_execution_time	Attacker action execution time.
attacker_action_vulnerability	Attacker action vulnerability.
snort_priority_alerts	List of the number of SNORT alerts of priorities 1-4.
snort_class_alerts	List of number of SNORT alerts of classes 1-34.
snort_total_alerts	Total number of SNORT alerts.
snort_alerts_weighted_by_priority	Number of SNORT alerts weighted by priority.
ossec_level_alerts	Number of OSSEC IDS alerts of level 1-16 per container.
ossec_group_alerts	Number of OSSEC IDS alerts of group 1-12 per container.
ossec_total_alerts	The total number of OSSEC IDS alerts per container.
ossec_alerts_weighted_by_priority	Total number of OSSEC IDS alerts weighted by priority per container.
avg_ossec_level_alerts	Average number of OSSEC IDS alerts of level 1-16.
avg_ossec_group_alerts	Average number of OSSEC IDS alerts of group 1-12.
avg_ossec_total_alerts	Average number of OSSEC IDS alerts.
avg_ossec_alerts_weighted_by_priority	Average number of OSSEC IDS alerts weighted by priority.

Table 11: Metrics collected per time-step by the monitoring system in version 0.10.0 of CSLE (2/4).

<i>Metric</i>	<i>Description</i>
flow_num_packets	Number of packets per flow.
flow_num_bytes	Number of transceived bytes per flow.
flow_duration	Duration per flow.
flow_in_port	Input port per flow.
flow_out_port	Output port per flow.
flow_dst_mac_address	Destination MAC address per flow.
flow_datapath_id	Datapath id per flow.
avg_flow_num_bytes	Average number of transceived bytes per flow.
avg_flow_duration	Average duration per flow.
of_port_datapath_id	OPENflow switch port datapath id.
of_port_num_received_packets	Number of received packets per OPENflow port.
of_port_num_received_bytes	Number of received bytes per OPENflow port.
of_port_num_received_errors	Number of received errors per OPENflow port.
of_port_num_transmitted_packets	Number of transmitted packets per OPENflow port.
of_port_num_transmitted_bytes	Number of transmitted bytes per OPENflow port.
of_port_num_transmitted_errors	Number of transmitted errors per OPENflow port.
of_port_num_received_dropped	Number of received dropped packets per OPENflow port.
of_port_num_transmitted_dropped	Number of transmitted dropped packets per OPENflow port.
of_port_num_received_frame_errors	Number of received frame errors per OPENflow port.
of_port_num_received_overrun_errors	Number of received overrun errors per OPENflow port.
of_port_num_received_crc_errors	Number of received CRC errors per OPENflow port.
of_port_num_collisions	Number of received collisions per OPENflow port.
of_port_duration_seconds	Duration uptime per OPENflow port.
avg_of_port_num_received_packets	Average OPENflow port number of received packets.
avg_of_port_num_received_bytes	Average OPENflow port number of received bytes.
avg_of_port_num_received_errors	Average OPENflow port number of received errors.
avg_of_port_num_transmitted_packets	Average OPENflow port number of transmitted packets.
avg_of_port_num_transmitted_bytes	Average OPENflow port number of transmitted bytes.
avg_of_port_num_transmitted_errors	Average OPENflow port number of transmitted errors.
avg_of_port_num_received_dropped	Average OPENflow port number of received dropped packets.
avg_of_port_num_transmitted_dropped	Average OPENflow port number of transmitted dropped packets.
avg_of_port_num_received_frame_errors	Average OPENflow port number of received frame errors.
avg_of_port_num_received_overrun_errors	Average OPENflow port number of received overrun errors.
avg_of_port_num_received_crc_errors	Average OPENflow port number of received CRC errors.
avg_of_port_num_collisions	Average OPENflow port number of received collisions.
avg_of_port_duration_seconds	Average OPENflow port duration uptime.

Table 12: Metrics collected per time-step by the monitoring system in version 0.10.0 of CSLE (3/4).

Metric	Description
switch_flow_num_packets	Number of packets per flow per switch.
switch_flow_num_bytes	Number of transceived bytes per flow per switch.
switch_flow_duration	Duration per flow per switch.
switch_flow_in_port	Input port per flow per switch.
switch_flow_out_port	Output port per flow per switch.
switch_flow_dst_mac_address	Destination MAC address per flow per switch.
switch_flow_datapath_id	Datapath id per flow per switch.
switch_avg_flow_num_bytes	Average number of transceived bytes per flow per switch.
switch_avg_flow_duration	Average duration per flow per switch.
switch_of_port_datapath_id	OPENflow switch port datapath id.
switch_of_port_num_received_packets	Number of received packets per OPENflow port per switch.
switch_of_port_num_received_bytes	Number of received bytes per OPENflow port per switch.
switch_of_port_num_received_errors	Number of received errors per OPENflow port per switch.
switch_of_port_num_transmitted_packets	Number of transmitted packets per OPENflow port per switch.
switch_of_port_num_transmitted_bytes	Number of transmitted bytes per OPENflow port per switch.
switch_of_port_num_transmitted_errors	Number of transmitted errors per OPENflow port per switch.
switch_of_port_num_received_dropped	Number of received dropped packets per OPENflow port per switch.
switch_of_port_num_transmitted_dropped	Number of transmitted dropped packets per OPENflow port per switch.
switch_of_port_num_received_frame_errors	Number of received frame errors per OPENflow port per switch.
switch_of_port_num_received_overrun_errors	Number of received overrun errors per OPENflow port per switch.
switch_of_port_num_received_crc_errors	Number of received CRC errors per OPENflow port per switch.
switch_of_port_num_collisions	Number of received collisions per OPENflow port per switch.
switch_of_port_duration_seconds	Duration uptime per OPENflow port per switch.
switch_avg_of_port_num_received_packets	Average OPENflow port number of received packets.
switch_avg_of_port_num_received_bytes	Average OPENflow port number of received bytes.
switch_avg_of_port_num_received_errors	Average OPENflow port number of received errors.
switch_avg_of_port_num_transmitted_packets	Average OPENflow port number of transmitted packets.
switch_avg_of_port_num_transmitted_bytes	Average OPENflow port number of transmitted bytes.
switch_avg_of_port_num_transmitted_errors	Average OPENflow port number of transmitted errors.
switch_avg_of_port_num_received_dropped	Average OPENflow port number of received dropped packets.
switch_avg_of_port_num_transmitted_dropped	Average OPENflow port number of transmitted dropped packets.
switch_avg_of_port_num_received_frame_errors	Average OPENflow port number of received frame errors.
switch_avg_of_port_num_received_overrun_errors	Average OPENflow port number of received overrun errors.
switch_avg_of_port_num_received_crc_errors	Average OPENflow port number of received CRC errors.
switch_avg_of_port_num_collisions	Average OPENflow port number of received collisions.
switch_avg_of_port_duration_seconds	Average OPENflow port duration uptime.

Table 13: Metrics collected per time-step by the monitoring system in version 0.10.0 of CSLE (4/4).

2.4.10 Management Agents

In addition to the monitoring agent, each emulated device runs a *management agent*, which is connected to the management network (described in §2.4.8) and exposes a GRPC API [10] for management actions (see Fig. 14). The management system invokes this API to perform management operations on behalf of CSLE users and reinforcement learning agents, e.g., restarting services, rotating log files, or updating firewall configurations. The management functions available in the API are listed in Tables 14-16.

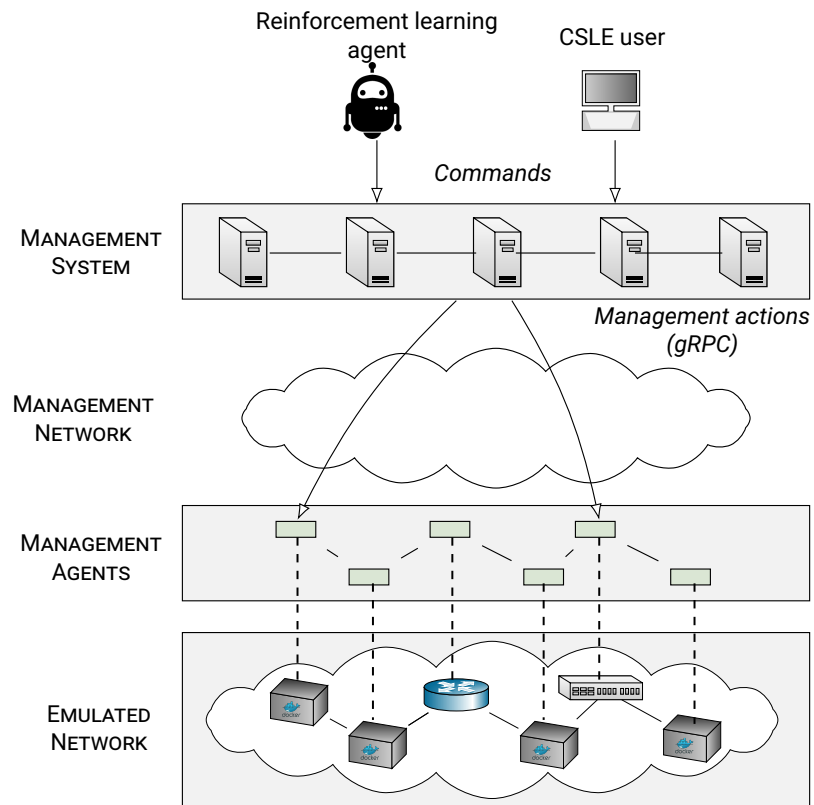


Figure 14: The management setup in CSLE; each emulated device runs a management agent that is connected to the management network and exposes a gRPC API for management actions, which is invoked by the management system on behalf of CSLE users and reinforcement learning agents.

<i>Remote procedure call</i>	<i>Description</i>
<code>getClients()</code>	Gets the number of clients.
<code>stopClients()</code>	Stops the client population.
<code>startClients()</code>	Starts the client population.
<code>startProducer()</code>	Starts the KAFKA producer thread of the client population.
<code>stopProducer()</code>	Stops the KAFKA producer thread of the client population.
<code>getDockerStatsMonitorStatus()</code>	Gets the status of the DOCKER statsmanager.
<code>stopDockerStatsMonitor()</code>	Stops the DOCKER statsmanager.
<code>startDockerStatsMonitor()</code>	Starts the DOCKER statsmanager.
<code>getElkStatus()</code>	Gets the status of the ELK stack.
<code>stopElk()</code>	Stops the ELK stack.
<code>startElk()</code>	Starts the ELK stack.
<code>stopElastic()</code>	Stops ELASTICSEARCH.
<code>startElastic()</code>	Starts ELASTICSEARCH.
<code>stopLogstash()</code>	Stops LOGSTASH.
<code>startLogstash()</code>	Starts LOGSTASH.
<code>stopKibana()</code>	Stops KIBANA.
<code>startKibana()</code>	Starts KIBANA.
<code>stopHostMonitor()</code>	Stops the host monitor of a particular host.
<code>startHostMonitor()</code>	Starts the host monitor of a particular host.
<code>getHostStatus()</code>	Gets the status of a host.
<code>getHostMetric()</code>	Gets a list of metrics of a given host.
<code>stopFilebeat()</code>	Stops FILEBEAT on a given host.
<code>startFilebeat()</code>	Starts FILEBEAT on a given host.
<code>configFilebeat()</code>	Configures FILEBEAT on a given host.
<code>stopPacketbeat()</code>	Stops PACKETBEAT on a given host.
<code>updateFirewall()</code>	Updates the firewall configuration on a given host.
<code>createUserAccount()</code>	Creates a user account on a given host.
<code>revokeUserCertificates()</code>	Revokes user certificates on a given host.

Table 14: Remote procedure calls (RPCs) in the GRPC API exposed by management agents in CSLE version 0.10.0 (1/3).

<i>Remote procedure call</i>	<i>Description</i>
startPacketbeat()	Starts PACKETBEAT on a given host.
configPacketbeat()	Configures PACKETBEAT on a given host.
stopMetricbeat()	Stops METRICBEAT on a given host.
startMetricbeat()	Starts METRICBEAT on a given host.
configMetricbeat()	Configures METRICBEAT on a given host.
stopHeartbeat()	Stops HEARTBEAT on a given host.
startHeartbeat()	Starts HEARTBEAT on a given host.
configHeartbeat()	Configures HEARTBEAT on a given host.
getKafkaStatus()	Gets the status of the KAFKA cluster.
stopKafka()	Stops the KAFKA cluster.
startKafka()	Starts the KAFKA cluster.
createTopic()	Creates a topic in KAFKA.
deleteTopic()	Deletes a topic in KAFKA.
getOSSECIdsAlerts()	Gets the list of OSSEC IDS alerts of a given host.
stopOSSECIdsMonitor()	Stops the OSSEC IDS monitor of a given host.
startOSSECIdsMonito()	Starts the OSSEC IDS monitor of a given host.
stopOSSECIds()	Stops the OSSEC IDS on a given host.
startOSSECIds()	Starts the OSSEC IDS on a given host.

Table 15: Remote procedure calls (RPCs) in the GRPC API exposed by management agents in CSLE version 0.10.0 (2/3).

<i>Remote procedure call</i>	<i>Description</i>
getOSSECIdsMonitorStatus()	Gets the status of the OSSEC IDS monitor of a given host.
getRyuStatus()	Gets the status of the RYU SDN manager.
stopRyu()	Stops the RYU SDN manager.
startRyu()	Starts the RYU SDN manager.
stopRyuMonitor()	Stops the RYU monitor.
startRyuMonitor()	Starts the RYU monitor.
getSnortIdsAlerts()	Gets the list of SNORT IDS alerts.
stopSnortIdsMonitor()	Stops the SNORT IDS monitor.
startSnortIdsMonitor()	Starts the SNORT IDS monitor.
getSnortIdsMonitorStatus()	Gets the status of the SNORT IDS monitor.
stopSnortIds()	Stops the SNORT IDS.
startSnortIds()	Starts the SNORT IDS.
getTrafficStatus()	Gets the status of the traffic generator of a given host.
stopTraffic()	Stops the traffic generator on a host.
startTraffic()	Starts the traffic generator on a host.
updateSnortConfig()	Updates the SNORT configuration on a given host.
updateOssecConfig()	Updates the OSSEC IDS configuration on a given host.
updateRyuConfig()	Updates the RYU configuration on a given host.

Table 16: Remote procedure calls (RPCs) in the GRPC API exposed by management agents in CSLE version 0.10.0 (3/3).

2.4.11 Starting an Emulation Execution

An emulation execution corresponds to running DOCKER containers and virtual networks. Starting an execution takes around 10-15 minutes. To start an execution, the emulation system first reads the emulation configuration from the metastore and starts the containers that are described in the configuration. Once the containers have started, the emulation system applies the emulation configuration. This step involves executing a sequence of Remote Procedure Calls (RPCs) through the GRPC API described in §2.4.10, e.g., RPCs to create user accounts, to start services, and to configure services (see Fig. 15).

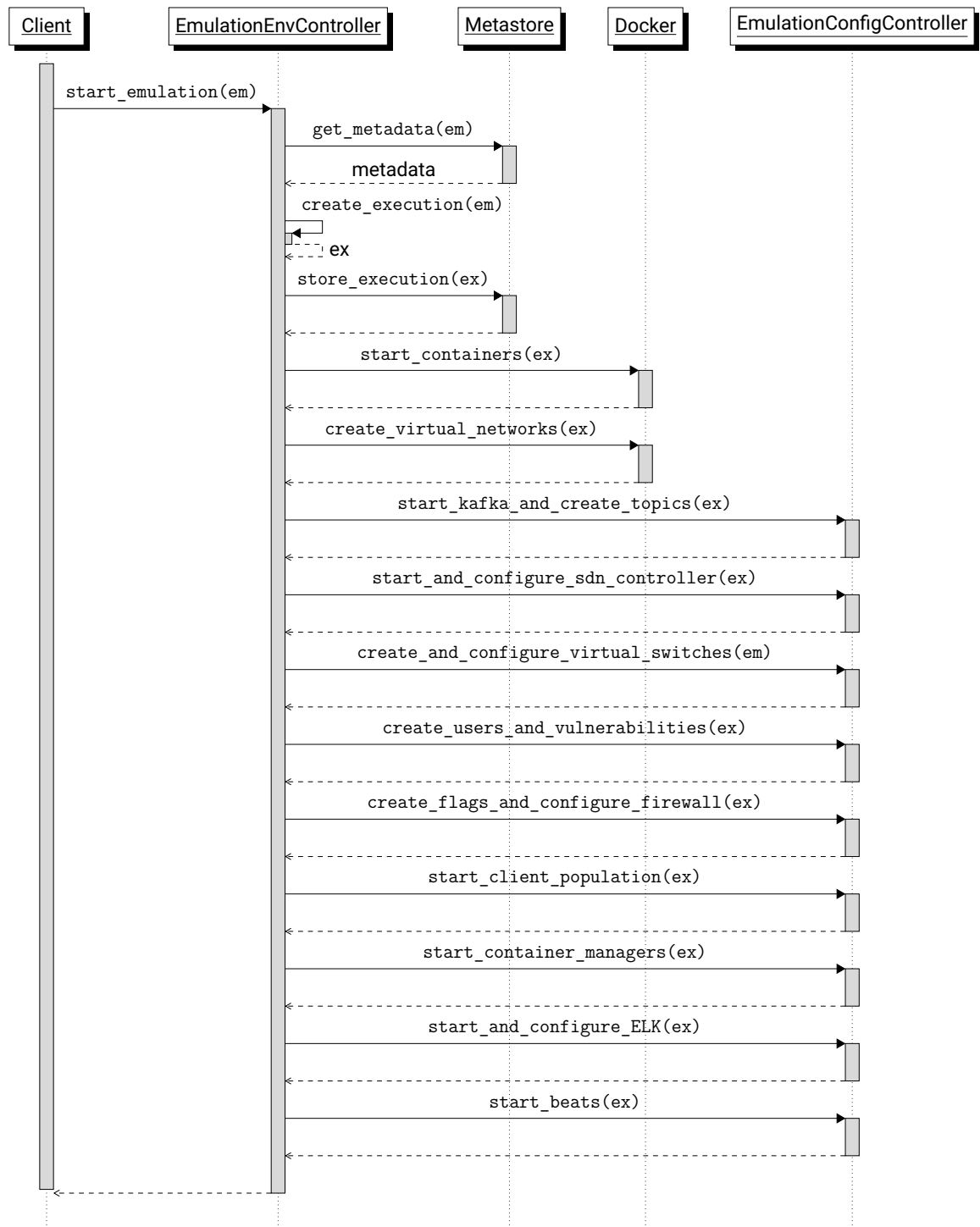


Figure 15: Sequence diagram of the emulation system's procedure to start an emulation execution.

2.5 The Simulation System

The simulation system uses the data collected from the emulation system to instantiate simulations, which are used to find effective defender strategies through reinforcement learning and other optimization techniques (see Fig. 16). It consists of simulation configurations and PYTHON libraries that implement numerical algorithms for computing defender strategies, for simulating Markov decision processes, and for system identification.

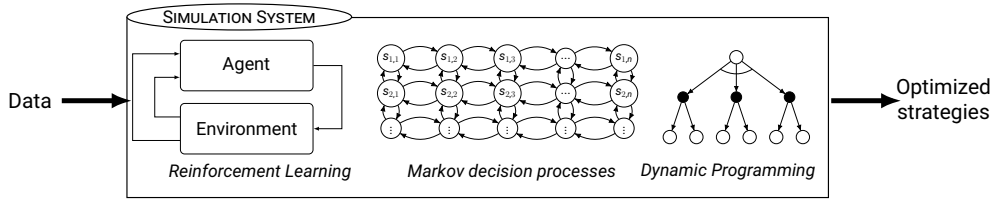


Figure 16: Overview of the simulation system; it uses data collected from emulated IT infrastructures to instantiate simulations of Markov decision processes (MDPs) and to optimize defender strategies through reinforcement learning and other optimization methods.

2.5.1 Simulation Environments

The simulation environments are listed in Table 17. Similar to an emulation execution, which is defined by an emulation configuration, a simulation is defined by a *simulation configuration*, which includes the set of properties listed in Table 18. Each simulation environment is implemented in PYTHON, and the configuration files are stored in the metastore. Simulations are typically based either on Markov decision processes or game-theoretic models.

Name	Description
Optimal stopping game	Zero-sum one-sided partially observed stochastic game in [27].
Optimal stopping MDP	MDP based on the optimal stopping formulation in [22].
Optimal stopping POMDP	POMDP based on the optimal stopping formulation in [22].
Local intrusion response game	Zero-sum partially observed stochastic game with public observations.
Local intrusion response POMDP attacker	POMDP for the attacker in the local intrusion response game.
Local intrusion response POMDP defender	POMDP for the defender in the local intrusion response game.
Workflow intrusion response game	Zero-sum partially observed stochastic game with public observations.
Workflow intrusion response POMDP attacker	POMDP for the attacker in the workflow intrusion response game.
Workflow intrusion response POMDP defender	POMDP for the defender in the workflow intrusion response game.
Intrusion recovery POMDP	POMDP intrusion recovery.
CMDP for replication factor control	CMDP for replication factor control
APT game	Zero-sum one-sided partially observed stochastic APT game.
APT MDP	MDP formulation of APT from the attacker's perspective.
APT POMDP	POMDP formulation of APT from the defender's perspective.

Table 17: Simulation environments supported in version 0.10.0 of CSLE.

<i>Configuration property</i>	<i>Description</i>
name	Name of the simulation environment.
gym_env_name	Name of the OpenAI gym environment.
descr	Description of the simulation environment.
simulation_env_input_config	Input configuration to the simulation.
players_config	Players configuration of the simulation.
state_space_config	State space configuration of the simulation.
joint_action_space_config	Joint action space configuration of the simulation.
joint_observation_space_config	Joint observation space config of the simulation.
time_step_type	Time-step type of the simulation
reward_function_config	Reward function configuration of the of the simulation.
transition_operator_config	Transition operator configuration of the simulation.
observation_function_config	Observation function configuration of the simulation.
emulation_statistic_id	Id of the emulation statistic.
initial_state_distribution_config	Initial state distribution configuration of the simulation.
version	Version of the simulation environment.
env_parameters_config	Parameters that are not part of the state but that the policy depends on.
plot_transition_probabilities	Boolean parameter whether to plot transition probabilities.
plot_observation_function	Boolean parameter whether to plot the observation function.
plot_reward_function	Boolean parameter whether to plot the reward function.

Table 18: Properties of a simulation environment configuration.

2.5.2 Numerical Algorithms

The numerical algorithms for strategy optimization and system identification are listed in [Table 19](#). These algorithms are implemented in PYTHON and are based on PYTORCH [2], NUMPY [33], PULP [61], CVXPY [6], OPENSPIEL [41], STABLE-BASELINES3 [55], EMUKIT [50], and CMA-PYTHON [32]. A code example of running the Q-learning algorithm is shown below:

```

1 import csle_common.constants.constants as constants
2 from csle_common.dao.training.experiment_config import
  ↳ ExperimentConfig
3 from csle_common.metastore.metastore_facade import
  ↳ MetastoreFacade
4 from csle_common.dao.training.agent_type import AgentType
5 from csle_common.dao.training.hparam import HParam
6 from csle_common.dao.training.player_type import PlayerType
7 from csle_agents.agents.q_learning.q_learning_agent import
  ↳ QLearningAgent
8 import csle_agents.constants.constants as agents_constants
9 # Select simulation configuration
10 simulation_env_config = MetastoreFacade.get_simulation_by_name(
11 "csle-stopping-mdp-attacker-002")
12 # Setup experiment with hyperparameters
13 experiment_config = ExperimentConfig(output_dir="..",
  ↳ title="Q-learning test", random_seeds=[..],
  ↳ agent_type=AgentType.Q_LEARNING, hparams={..},
  ↳ player_type=PlayerType.ATTACKER, player_idx=1)
14 agent =
  ↳ QLearningAgent(simulation_env_config=simulation_env_config,
  ↳ experiment_config=experiment_config, save_to_metastore=True)
15 # Run the algorithm
16 experiment_execution = agent.train()
17 # Save the results and the learned policies
18 MetastoreFacade.save_experiment_execution(experiment_execution)
19 for policy in experiment_execution.result.policies.values():
20     MetastoreFacade.save_tabular_policy(tabular_policy=policy)

```

Listing 2: Example of strategy optimization through the Q-learning algorithm in CSLE.

<i>Name</i>	<i>Algorithm type</i>
Bayesian optimization	Black-box optimization
Cross-entropy method	Black-box optimization
Simulated annealing	Black-box optimization
CMA-ES	Evolutionary computation
Nelder-mead	Black-box optimization
Particle swarm	Black-box optimization
Differential evolution	Evolutionary computation
Deep Q-network (DQN)	Reinforcement learning
Fictitious Self-Play	Computational game theory
Heuristic Search Value Iteration (HSVI)	Dynamic programming
Heuristic Search Value Iteration (HSVI) for one-sided game POSGs	Computational game theory
Kiefer Wolfowitz	Stochastic approximation
Linear programming	Computational game theory/Markov decision theory
Policy iteration	Dynamic programming
Value iteration	Dynamic programming
Proximal Policy Optimization (PPO)	Reinforcement learning
Q-learning	Reinforcement learning
Random search	Black-box optimization
REINFORCE	Reinforcement learning
SARSA	Reinforcement learning
Shapley iteration	Computational game theory
Sondik's value iteration	Dynamic programming
T-FP	Reinforcement learning
T-SPSA	Reinforcement learning
Expectation maximization	System identification
POMCP	POMDP planning
Phasic Policy Gradient (PPG)	Reinforcement learning

Table 19: Numerical algorithms supported in version 0.10.0 of CSLE.

2.6 The Web Interface

The web interface of CSLE can be used to view management information and to request management operations (see Fig. 17). It can also be used to monitor emulations in real-time (see Fig. 18 and Fig. 19), to start or stop services (see Fig. 20), to monitor reinforcement learning workloads (see Fig. 21), to access terminals of emulated components (see Fig. 22), and to examine security policies (see Fig. 2.6.1).

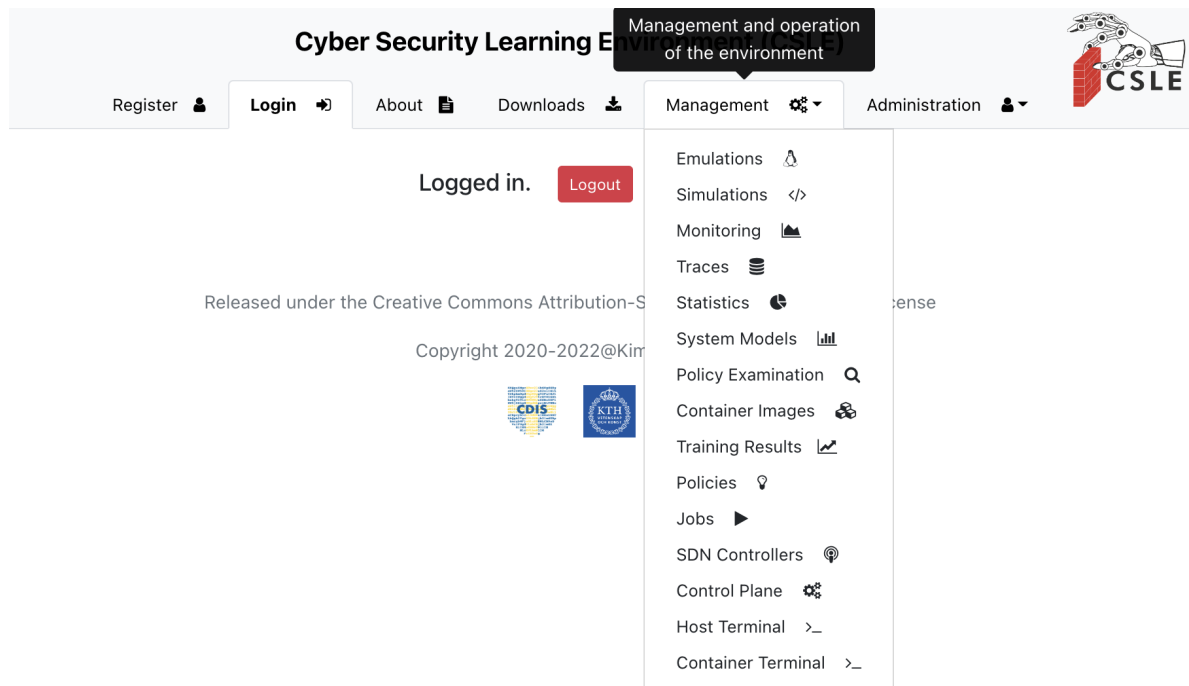


Figure 17: The web interface of CSLE; this interface shows management information and can be used to request management operations.



Figure 18: The monitoring page of the web interface; this page allows a user to view system metrics of a given emulation execution in real-time.

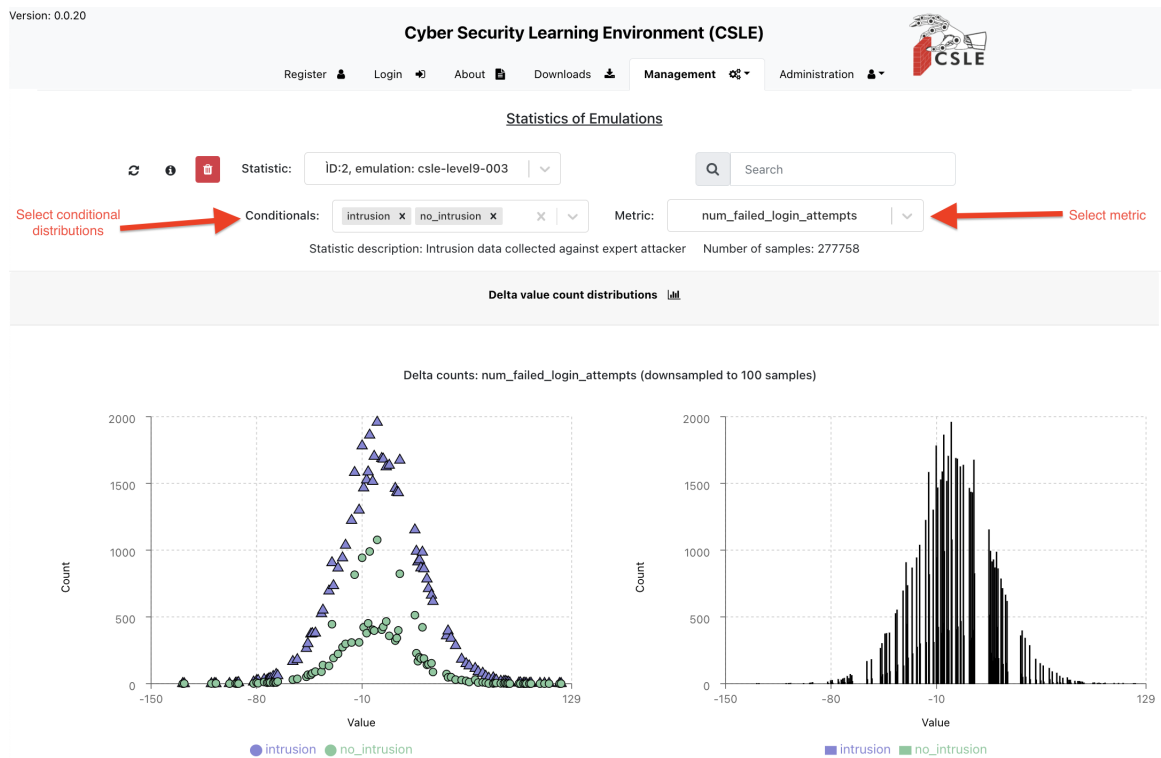


Figure 19: The statistics page of the web interface; this page allows a user to view empirical statistics of emulated infrastructures, e.g., empirical distributions of system metrics.

Version: 0.0.20

Cyber Security Learning Environment (CSLE)



Register
Login
About
Downloads
Management
Administration

Emulations Control Plane

Execution:
ID:15, emulation: csle-level9-003
Search

Execution:

ID: 15, name: csle-level9-003

[Docker container statuses](#)


Stop all containers:
Start all containers:

Name	Image	Os	IPs	Status	Actions
csle_client_1_1-level9-15	csle_client_1	ubuntu	15.9.1.254, 15.9.253.254	Running	  
csle_ftp_1_1-level9-15	csle_ftp_1	ubuntu	15.9.2.79, 15.9.253.79	Running	  
csle_hacker_kali_1_1-level9-15	csle_hacker_kali_1	kali	15.9.1.191, 15.9.253.191	Running	  
csle_honeypot_1_1-level9-15	csle_honeypot_1	ubuntu	15.9.2.21, 15.9.253.21	Running	  
csle_router_2_1-level9-15	csle_router_2	ubuntu	15.9.2.10, 15.9.1.10, 15.9.253.10	Running	  
csle_ssh_1_1-level9-15	csle_ssh_1	ubuntu	15.9.2.2, 15.9.3.2, 15.9.253.2	Running	  
csle_samba_2_1-level9-15	csle_samba_2	debian	15.9.2.3, 15.9.4.3, 15.9.253.3	Running	  

Figure 20: The control plane of the web interface; this page allows a user to request management operations, e.g., starting and stopping emulated components or services.

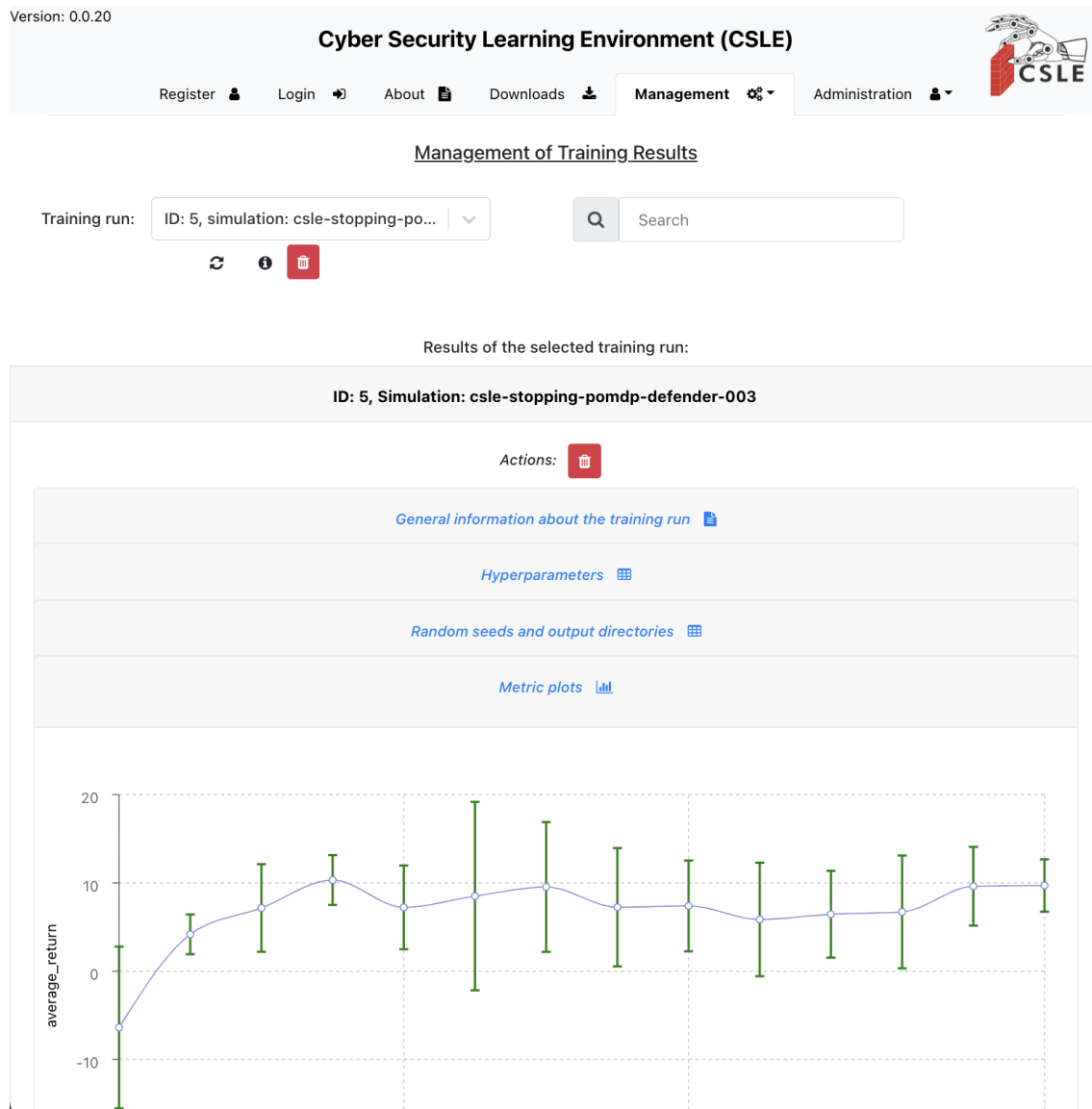


Figure 21: The training results page of the web interface; this page allows a user to view the results of reinforcement learning experiments.

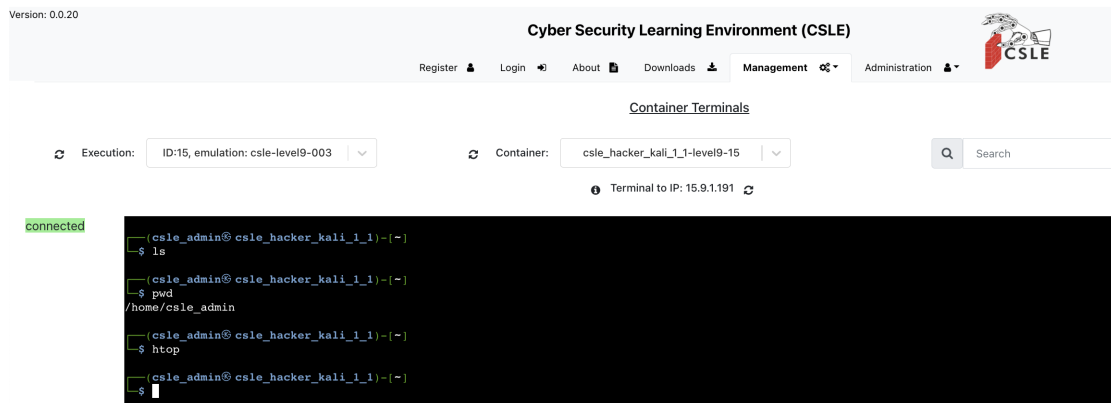


Figure 22: The container-terminal page of the web interface; this page allows a user to execute arbitrary bash commands inside a container of an emulated infrastructure.

2.6.1 The Policy Examination System

The policy examination system allows a user to traverse episodes of Markov decision processes in a controlled manner and to track the actions triggered by security policies. Like a software debugger, a user can continue or halt an episode at any time step and inspect parameters and probability distributions of interest. The system enables insight into the structure of a given policy and the behavior of a policy in edge cases. It is integrated with the rest of the web interface and can be accessed as a regular web page (see Fig. 23).

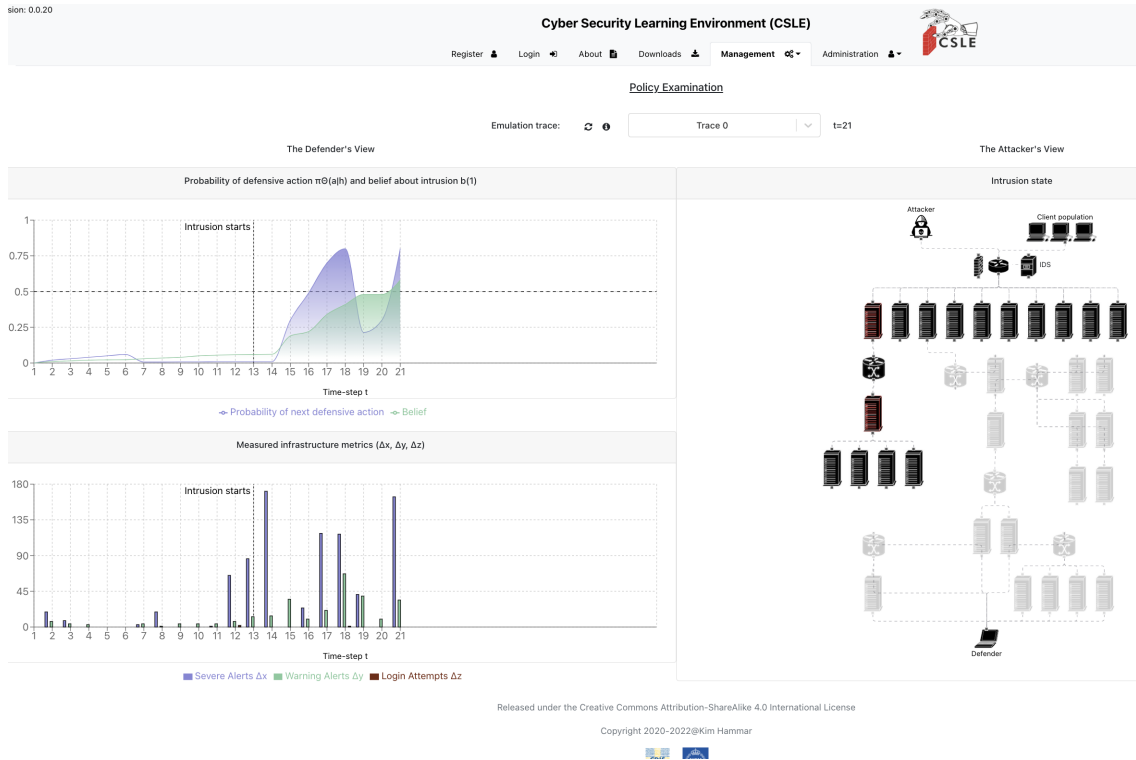


Figure 23: The demonstration view of the policy examination system; the system allows a user to interactively step through an episode of a Markov decision process.

Figure 23 pictures the main user interface of the policy examination system. The left part of the interface shows the defender's view, and the right part shows the attacker's view. The plot on the upper left shows the defender's belief about the state of the infrastructure and the probability that the defender will take action at each step. The plot on the lower left shows the distribution of infrastructure metrics that the defender observes. The graphic on the right shows the attacker's view, depicted as an overlay on the IT infrastructure's topology.

The policy examination system provides insight into the structure of the defender policy and its behavior in edge cases. It can be used to observe correlations among the attacker's actions, the infrastructure metrics, and the actions that the defender policy prescribes. It can also be used to examine which actions of the attacker are difficult for the defender to detect or trigger actions by the defender.

2.6.2 Implementation

The web interface is implemented by a web application written in JAVASCRIPT [7] and the REACT framework [8], which executes on the management system. Each physical server of the management system runs a Flask web server [51] that exposes an HTTP REST API (the REST API is detailed in the subsequent section). In front of these web servers is a NGINX [56] reverse proxy that load balances client requests, decrypts incoming HTTPS traffic, and encrypts outgoing HTTP traffic using TLS (see Fig. 24).

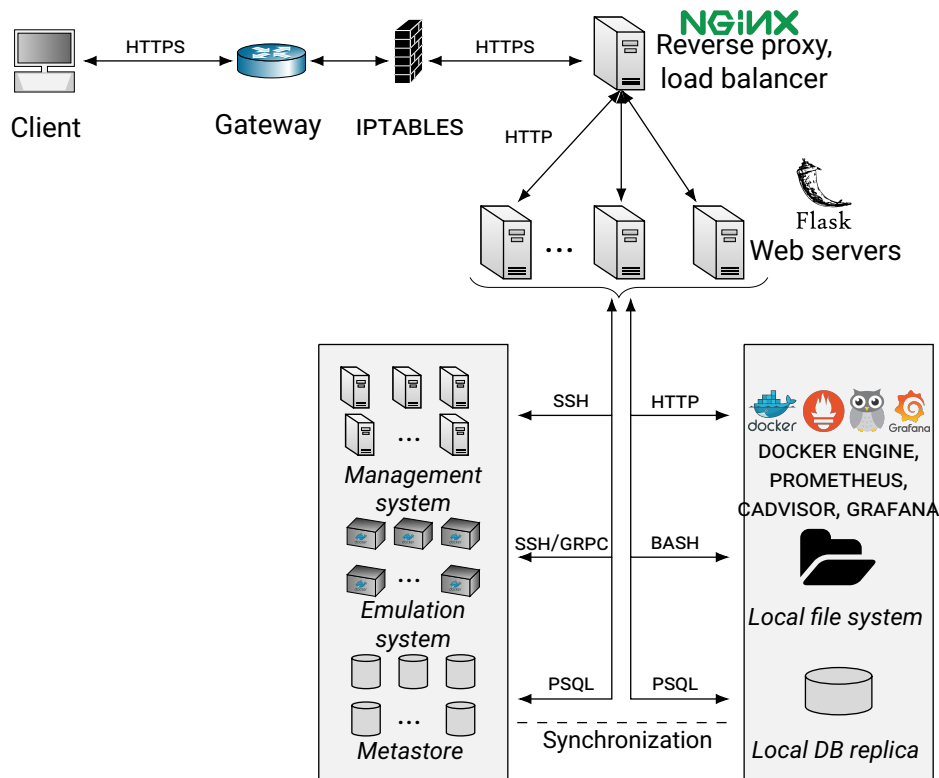


Figure 24: Architecture diagram of the CSLE web application; a NGINX reverse proxy load balances requests to a group of Flask web servers, which expose HTTP REST APIs; these APIs can be used to view management information and to request management operations.

The web-based terminal emulation shown in Fig. 22 is implemented as follows. To create a web-based terminal session, the web client's browser first opens a web socket to one of the HTTP servers. The server then opens an SSH tunnel to the emulated device and creates a terminal session. This tunnel is then used to pipe commands and terminal responses between the client and the terminal (see Fig. 25).

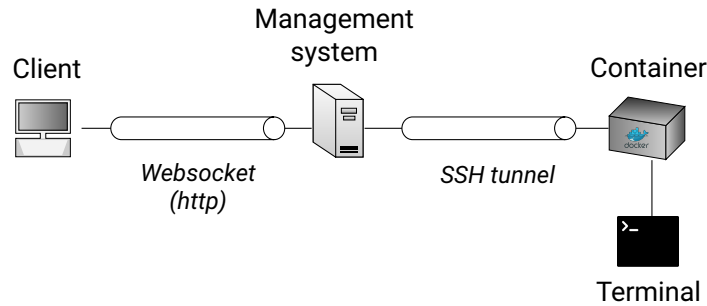


Figure 25: Architecture diagram of the web-based terminal emulation in CSLE; the web application in the client's browser opens a web socket to an HTTP server running on one of the physical servers of the management system; the web server then pipes the web socket to an SSH tunnel that connects to a terminal of the emulated component.

2.6.3 The REST API

The endpoints of the REST API are listed in Tables 20-24. An example HTTP request and response is given below.

```

1 curl https://csle.dev/emulations?ids=true\&token=<my_token>
2 [{"emulation":"csle-level1-0.10.0","id":1,"running":false},
3 {"emulation":"csle-level2-0.10.0","id":2,"running":false},
4 {"emulation":"csle-level3-0.10.0","id":3,"running":false},
5 {"emulation":"csle-level4-0.10.0","id":4,"running":false},
6 {"emulation":"csle-level5-0.10.0","id":5,"running":false},
7 {"emulation":"csle-level6-0.10.0","id":6,"running":false},
8 {"emulation":"csle-level7-0.10.0","id":7,"running":false},
9 {"emulation":"csle-level8-0.10.0","id":8,"running":false},
10 {"emulation":"csle-level9-0.10.0","id":9,"running":true},
11 {"emulation":"csle-level10-0.10.0","id":10,"running":false},
12 {"emulation":"csle-level11-0.10.0","id":11,"running":false},
13 {"emulation":"csle-level12-0.10.0","id":12,"running":false}]

```

Listing 3: An HTTP request for the `emulations` resource of the CSLE REST API (line 1) and the corresponding HTTP response (lines 2-13); the request includes the flag `ids=true`, which means that the response only includes the identifiers, names, and statuses of the emulations, rather than the full configurations; the request is performed with `curl` [60].

Resource	Method
/emulations	GET, DELETE
/emulations?ids=true&token=<token>	GET, DELETE
/emulations/<em_id>?token=<token>	GET, DELETE, POST
/emulations/<em_id>/executions?token=<token>	GET, DELETE
/emulations/<em_id>/executions/<exec_id>?token=<token>	GET, DELETE
/emulations/<em_id>/executions/<exec_id>/monitor/<min>?token=<token>	GET
/simulations?token=<token>	GET, DELETE
/simulations?ids=true&token=<token>	GET, DELETE
/simulations/<simulation_id>?token=<token>	GET, DELETE
/cadvisor?token=<token>	POST, GET
/nodeexporter?token=<token>	POST, GET
/grafana?token=<token>	POST, GET
/prometheus?token=<token>	POST, GET
/alpha-vec-policies?token=<token>	GET, DELETE
/alpha-vec-policies?ids=true&token=<token>	GET, DELETE
/alpha-vec-policies/<policy_id>?token=<token>	GET, DELETE
/dqn-policies?token=<token>	GET, DELETE
/dqn-policies?ids=true&token=<token>	GET, DELETE
/dqn-policies/<policy_id>?token=<token>	GET, DELETE
/ppo-policies?token=<token>	GET, DELETE
/ppo-policies?ids=true&token=<token>	GET, DELETE
/ppo-policies/<policy_id>?token=<token>	GET, DELETE
/vector-policies?token=<token>	GET, DELETE
/vector-policies?ids=true&token=<token>	GET, DELETE
/vector-policies/<policy_id>?token=<token>	GET, DELETE
/tabular-policies?token=<token>	GET, DELETE
/tabular-policies?ids=true&token=<token>	GET, DELETE
/tabular-policies/<policy_id>?token=<token>	GET, DELETE
/multi-threshold-policies?token=<token>	GET, DELETE
/multi-threshold-policies?ids=true&token=<token>	GET, DELETE
/multi-threshold-policies/<policy_id>?token=<token>	GET, DELETE
/linear-threshold-policies?token=<token>	GET, DELETE
/linear-threshold-policies?ids=true&token=<token>	GET, DELETE
/linear-threshold-policies/<policy_id>?token=<token>	GET, DELETE
/fnn-w-softmax-policies?token=<token>	GET, DELETE
/fnn-w-softmax-policies?ids=true&token=<token>	GET, DELETE
/fnn-w-softmax-policies/<policy_id>?token=<token>	GET, DELETE
/training-jobs?token=<token>	GET, DELETE
/training-jobs?ids=true&token=<token>	GET, DELETE
/training-jobs/<job_id>?token=<token>	GET, DELETE, POST
/data-collection-jobs?token=<token>	GET, DELETE
/data-collection-jobs?ids=true&token=<token>	GET, DELETE
/data-collection-jobs/<job_id>?token=<token>	GET, DELETE, POST
/system-identification-jobs?token=<token>	GET, DELETE
/system-identification-jobs?ids=true&token=<token>	GET, DELETE
/system-identification-jobs/<job_id>?token=<token>	GET, DELETE

Table 20: REST API resources in version 0.10.0 of CSLE (1/5).

Resource	Method
/emulation-traces?token=<token>	GET, DELETE
/emulation-traces?ids=true&token=<token>	GET, DELETE
/emulation-traces/<trace_id>?token=<token>	GET, DELETE
/simulation-traces?token=<token>	GET, DELETE
/simulation-traces?ids=true&token=<token>	GET, DELETE
/simulation-traces/<trace_id>?token=<token>	GET, DELETE
/emulation-simulation-traces?token=<token>	GET, DELETE
/emulation-simulation-traces?ids=true&token=<token>	GET, DELETE
/emulation-simulation-traces/<trace_id>?token=<token>	GET, DELETE
/images?token=<token>	GET
/file?token=<token>	POST
/experiments?token=<token>	GET, DELETE
/experiments?ids=true&token=<token>	GET, DELETE
/experiments/<trace_id>?token=<token>	GET, DELETE
/sdn-controllers?token=<token>	GET, DELETE
/sdn-controllers?ids=true&token=<token>	GET, DELETE
/emulation-statistics?token=<token>	GET, DELETE
/emulation-statistics?ids=true&token=<token>	GET, DELETE
/emulation-statistics/<trace_id>?token=<token>	GET, DELETE
/gaussian-mixture-system-models?token=<token>	GET, DELETE
/gaussian-mixture-system-models?ids=true&token=<token>	GET, DELETE
/gaussian-mixture-system-models/<trace_id>?token=<token>	GET, DELETE
/empirical-system-models?token=<token>	GET, DELETE
/empirical-system-models?ids=true&token=<token>	GET, DELETE
/empirical-system-models/<trace_id>?token=<token>	GET, DELETE
/mcmc-system-models?token=<token>	GET, DELETE
/mcmc-system-models?ids=true&token=<token>	GET, DELETE
/mcmc-system-models/<trace_id>?token=<token>	GET, DELETE
/gp-system-models?token=<token>	GET, DELETE
/gp-system-models?ids=true&token=<token>	GET, DELETE
/gp-system-models/<model_id>?token=<token>	GET, DELETE
/system-models?ids=true&token=<token>	GET
/login	POST
/traces-datasets?token=<token>	GET, DELETE
/traces-datasets?ids=true&token=<token>	GET, DELETE
/traces-datasets/<traces_datasets_id>?token=<token>	GET, DELETE
/traces-datasets/<traces_datasets_id>?token=<token>&download=true	GET
/server-cluster?token=<token>	GET
/pgadmin?token=<token>	GET,POST
/postgresql?token=<token>	GET,POST
/docker?token=<token>	GET,POST
/nginx?token=<token>	GET,POST
/flask?token=<token>	GET,POST
/clusterstatuses?token=<token>	GET
/logs/nginx?token=<token>	POST
/logs/postgresql?token=<token>	POST
/logs/docker?token=<token>	POST
/logs/flask?token=<token>	POST
/logs/clustermanager?token=<token>	POST

Table 21: REST API resources 54 version 0.10.0 of CSLE (2/5).

Resource	Method
/statistics-datasets?token=<token>	GET, DELETE
/statistics-datasets?ids=true&token=<token>	GET, DELETE
/statistics-datasets/<statistics_datasets_id>?token=<token>	GET, DELETE
/statistics-datasets/<statistics_datasets_id>?token=<token>&download=true	GET
/emulation-executions?token=<token>	GET, DELETE
/emulation-executions?ids=true&token=<token>	GET, DELETE
/emulation-executions/<exec_id>?token=<token>	GET, DELETE
/emulation-executions/<exec_id>/info?token=<token>&emulation=	GET
/emulation-executions/<exec_id>/docker-stats-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/docker-stats-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/client-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/client-population?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/client-producer?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/kafka-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/kafka?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/snort-ids-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/snort-ids-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/snort-ids?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/ossec-ids-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/ossec-ids?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/ossec-ids-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/host-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/host-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/elk-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/elastic?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/logstash?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/kibana?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/container?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/elk-stack?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/traffic-generator?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/filebeat?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/packetbeat?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/metricbeat?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/ryu-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/ryu-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/ryu-controller?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/switches?token=<token>&emulation=	POST
/users?token=<token>	GET, DELETE, PUT
/users?ids=true&token=<token>	GET, DELETE
/users/<user_id>?token=<token>	GET, DELETE, PUT
/users/create	POST
/logs?token=<token>	GET
/logs/docker-stats-manager?token=<token>	POST
/logs/prometheus?token=<token>	POST

Table 22: REST API resources in version 0.10.0 of CSLE (3/5).

<i>Resource</i>	<i>Method</i>
/logs/grafana?token=<token>	POST
/logs/cadvisor?token=<token>	POST
/logs/pgadmin?token=<token>	POST
/logs/container?token=<token>	GET
/logs/node-exporter?token=<token>	POST
/logs/client-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/kafka-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/elk-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/ryu-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/traffic-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/snort-ids-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/ossec-ids-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/host-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/ossec-ids?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/snort-ids?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/kafka?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/elk-stack?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/ryu-controller?token=<token>&emulation=&executionid=<exec_id>	POST
/config?token=<token>	GET, PUT
/config/registration-allowed	GET
/version	GET
/about-page	GET
/login-page	GET
/register-page	GET
/emulation-statistics-page	GET
/emulations-page	GET
/images-page	GET
/downloads-page	GET
/jobs-page	GET
/monitoring-page	GET
/policies-page	GET
/policy-examination-page	GET
/sdn-controllers-page	GET
/control-plane-page	GET
/user-admin-page	GET
/system-admin-page	GET
/logs-admin-page	GET
/simulations-page	GET
/system-models-page	GET
/traces-page	GET
/training-page	GET
/container-terminal-page	GET
/container-terminal?token=<token>	Websockets
/create-emulation-page	GET
/create-emulation?token=<token>	POST

Table 23: REST API resources in version 0.10.0 of csLE (4/5).

<i>Resource</i>	<i>Method</i>
/emulation-executions/<exec_id>/five-g-core-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-core-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-cu-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-cu-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-du-manager?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-du-monitor?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-core?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-cu?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-du?token=<token>&emulation=	POST
/emulation-executions/<exec_id>/five-g-ue?token=<token>&emulation=	POST
/logs/five-g-core-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/five-g-cu-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/five-g-du-manager?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/five-g-core?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/five-g-cu?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/five-g-du?token=<token>&emulation=&executionid=<exec_id>	POST
/logs/five-g-ue?token=<token>&emulation=&executionid=<exec_id>	POST

Table 24: REST API resources in version 0.10.0 of csLE (5/5).

2.7 The Command-Line Interface (CLI)

The Command-Line Interface (CLI) allows a user to inspect the state of emulations and to execute management operations from the command line. The commands available in the CLI are listed in Tables 25-27. An example command and the resulting output are shown below.

```
1 kim@gpu2 ~> csle ls emulations --all
2 CSLE emulations:
3 csle-level9-0.10.0 [running]
4 csle-level1-0.10.0 [stopped]
5 csle-level2-0.10.0 [stopped]
6 csle-level3-0.10.0 [stopped]
7 csle-level4-0.10.0 [stopped]
8 csle-level5-0.10.0 [stopped]
9 csle-level6-0.10.0 [stopped]
10 csle-level7-0.10.0 [stopped]
11 csle-level8-0.10.0 [stopped]
12 csle-level10-0.10.0 [stopped]
13 csle-level11-0.10.0 [stopped]
14 csle-level12-0.10.0 [stopped]
```

Listing 4: The command `csle ls emulations --all` (line 1) and the output generated by the CSLE CLI (lines 2-14).

<i>Command</i>	<i>Description</i>
<code>csle attacker <execid></code>	Opens an attacker shell in a given execution.
<code>csle clean all</code>	Removes all containers, networks, emulations, etc.
<code>csle clean containers <execid></code>	Removes all DOCKER containers in a given execution.
<code>csle clean emulations <execid></code>	Removes all emulations in a given execution.
<code>csle clean emulation_traces <execid></code>	Removes all emulation traces in a given execution.
<code>csle clean simulation_traces <execid></code>	Removes all simulation traces in a given execution.
<code>csle clean emulation_statistics <execid></code>	Removes all emulation statistics in a given execution.
<code>csle clean emulation_executions <execid></code>	Removes all emulation executions in a given execution.
<code>csle em <emname></code>	Get status of a given emulation.
<code>csle em <emname> -host</code>	Get status of host managers in a given emulation.
<code>csle em <emname> -stats</code>	Get status of stats managers in a given emulation.
<code>csle em <emname> -kafka</code>	Get status of kafka managers in a given emulation.
<code>csle em <emname> -snortids</code>	Get status of snortids in a given emulation.
<code>csle em <emname> -clients</code>	Get status of clients in a given emulation.
<code>csle em <emname> -executions</code>	Get status of executions in a given emulation.
<code>csle em <emname> -executions</code>	Get status of executions in a given emulation.
<code>csle init</code>	Initializes CSLE and sets up management accounts.
<code>csle install emulations</code>	Install emulation environments.
<code>csle install simulations</code>	Install simulation environments.
<code>csle install <emname></code>	Install a given emulation.
<code>csle install <simname></code>	Install a given simulation.
<code>csle install derived images</code>	Install derived DOCKER images.
<code>csle install base images</code>	Install base DOCKER images.
<code>csle install metastore</code>	Install the metastore.
<code>csle install all</code>	Install everything.
<code>csle ls all</code>	List all information about entities.
<code>csle ls containers</code>	List all containers.
<code>csle ls networks</code>	List all networks.
<code>csle ls images</code>	List all DOCKER images.
<code>csle ls emulations</code>	List all emulations.
<code>csle ls environments</code>	List all environments.
<code>csle ls prometheus</code>	List status of PROMETHEUS.
<code>csle ls node_exporter</code>	List status of node exporter.
<code>csle ls cadvisor</code>	List status of cadvisor.
<code>csle ls statsmanager</code>	List status of statsmanager.
<code>csle ls flask</code>	List status of the REST API server.
<code>csle ls simulations</code>	List status of simulations.
<code>csle ls emulation_executions</code>	List status of emulation executions.
<code>csle ls <entity> -all</code>	List extended information of the given entity.
<code>csle ls <entity> -running</code>	Only list information about running entities.
<code>csle ls <entity> -stopped</code>	Only list information about stopped entities.
<code>csle rm <network></code>	Removes the network with the given name.
<code>csle rm <container></code>	Removes the container with the given name.
<code>csle rm <image></code>	Removes the DOCKER image with the given name.
<code>csle rm networks</code>	Removes all networks.
<code>csle rm images</code>	Removes all images.
<code>csle rm containers</code>	Removes all containers.
<code>csle rm help</code>	Lists all available commands.

Table 25: Commands available in the CSLE command-line interface (version 0.10.0) (1/3).

Command	Description
csle shell <containername>	Opens shell in a given container.
csle start prometheus -ip <ip>	Starts prometheus on a given IP.
csle start nginx -ip <ip>	Starts nginx on a given IP.
csle start postgresql -ip <ip>	Starts postgresql on a given IP.
csle start docker -ip <ip>	Starts the DOCKER engine on a given IP.
csle start clustermanager	Starts the cluster manager locally.
csle start node_exporter -ip <ip>	Starts node exporter on a given IP.
csle start grafana -ip <ip>	Starts grafana on a given IP.
csle start cadvisor -ip <ip>	Starts cadvisor on a given IP.
csle start flask -ip <ip>	Starts the REST API server on a given IP.
csle start <containername>	Starts the given container.
csle start <emulationname>	Starts the given emulation.
csle start <emulationname> -no_network	Starts the given emulation without virtual networks.
csle start <emulationname> -no_traffic	Starts the given emulation without traffic generators.
csle start <emulationname> -no_clients	Starts the given emulation without clients.
csle start <emulationname> -id	Starts the given emulation with execution id.
csle start all	Starts everything.
csle start all -id	Starts everything in an execution.
csle start statsmanager -ip <ip>	Starts the statsmanager on a given IP.
csle start <trainingjobid>	Starts trainingjob with a given id.
csle start <systemidjobid>	Starts system identification job with a given id.
csle start <image> <containername>	Starts a container with a given image and name.
csle start_traffic <emulationname> <executionid>	Starts the traffic and clients in execution.
csle statsmanager <port> <logdir> <logfile> <maxworkers>	Starts the statsmanager locally.
csle stop <emulationname> <execid>	Stops the emulation execution.
csle stop <prometheus> -ip <ip>	Stops prometheus on a given IP.
csle stop <cadvisor> -ip <ip>	Stops cadvisor on a given IP.
csle stop <grafana> -ip <ip>	Stops grafana on a given IP.
csle stop <flask> -ip <ip>	Stops the REST API server on a given IP.
csle stop <containername> <execid>	Stops the container.
csle stop <statsmanager> -ip <ip>	Stops the statsmanager on a given ip.
csle stop emulation_executions	Stops all emulation executions.
csle stop nginx -ip <ip>	Stops NGINX on the given IP.
csle stop docker -ip <ip>	Stops the DOCKER engine on the given IP.
csle stop postgresql -ip <ip>	Stops Postgresql on the given IP.
csle stop all	Stops everything that runs.
csle stop_traffic <emulationname> <execid>	Stops client population and traffic in execution.
csle systemidentificationjob <jobid>	Starts job.
csle trainingjob <jobid>	Starts job.
csle datacollectionjob <jobid>	Starts job.
csle uninstall emulations	Uninstalls emulation environments.
csle uninstall simulations	Uninstalls simulation environments.
csle uninstall <emname>	Uninstalls emulation.
csle uninstall <simname>	Uninstalls simulation.
csle uninstall derived_images	Uninstalls derived images.
csle uninstall base_images	Uninstalls base images.
csle uninstall metastore	Uninstalls metastore.
csle uninstall all	Uninstalls everything.

Table 26: Commands available in the CSLE command-line interface (version 0.10.0) (2/3).

Command	Description
csle start hostmanagers <emulationname> -id <emulationid> -ip <ip>	Starts host managers on the specified ip sever.
csle start hostmanager <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts a host manager on the specified container.
csle start clientmanager <emulationname> -id <emulationid> -ip <ip>	Starts a client manager on the specified ip sever.
csle start snortmanagers <emulationname> -id <emulationid> -ip <ip>	Starts snort managers on the specified ip sever.
csle start snortmanager <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts a snort manager on the specified container.
csle start elkmanager <emulationname> -id <emulationid> -ip <ip>	Starts an Elk manager on the specified ip sever.
csle start trafficmanagers <emulationname> -id <emulationid> -ip <ip>	Starts traffic managers on the specified ip sever.
csle start trafficmanager <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts a traffic manager on the specified container.
csle start kafkamanagers <emulationname> -id <emulationid> -ip <ip>	Starts a KAFKA manager on specified ip sever.
csle start ossecmanagers <emulationname> -id <emulationid> -ip <ip>	Starts Ossec managers on the specified ip sever.
csle start ossecmanager <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts an Ossec manager on the specified container.
csle start ryuamanager <emulationname> -id <emulationid> -ip <ip>	Starts a RVU manager on a specified ip sever.
csle start filebeats <emulationname> -id <emulationid> -ip <ip>	Starts FILEBEATS on the specified ip sever.
csle start filebeat <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts the FILEBEAT on the specified container.
csle start metricbeats <emulationname> -id <emulationid> -ip <ip>	Starts METRICBEATS on the specified ip sever.
csle start metricbeat <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts the METRICBEAT on the specified container.
csle start heartbeats <emulationname> -id <emulationid> -ip <ip>	Starts HEARTBEATS on the specified ip sever.
csle start heartbeat <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts the HEARTBEAT on the specified container.
csle start packetbeats <emulationname> -id <emulationid> -ip <ip>	Starts PACKETBEATS on the specified ip sever.
csle start packetbeat <emulationname> -id <emulationid> -ip <ip> -container_ip <containerip>	Starts the PACKETBEAT on the specified container.
csle stop hostmanagers <emulationname> <emulationid> -ip <ip>	Stops host managers on the specified ip sever.
csle stop hostmanager <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops a host manager on the specified container.
csle stop clientmanager <emulationname> <emulationid> -ip <ip>	Stops a client manager on the specified ip sever.
csle stop snortmanagers <emulationname> <emulationid> -ip <ip>	Stops snort managers on the specified ip sever.
csle stop snortmanager <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops a snort manager on the specified container.
csle stop elkmanager <emulationname> <emulationid> -ip <ip>	Stops an Elk manager on the specified ip sever.
csle stop trafficmanagers <emulationname> <emulationid> -ip <ip>	Stops traffic managers on the specified ip sever.
csle stop trafficmanager <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops a traffic manager on the specified container.
csle stop kafkamanagers <emulationname> <emulationid> -ip <ip>	Stops a KAFKA manager on specified ip sever.
csle stop ossecmanagers <emulationname> <emulationid> -ip <ip>	Stops Ossec managers on the specified ip sever.
csle stop ossecmanager <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops an Ossec manager on the specified container.
csle stop ryuamanager <emulationname> <emulationid> -ip <ip>	Stops a RVU manager on a specified ip sever.
csle stop filebeats <emulationname> <emulationid> -ip <ip>	Stops FILEBEATS on the specified ip sever.
csle stop filebeat <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops the FILEBEAT on the specified container.
csle stop metricbeats <emulationname> <emulationid> -ip <ip>	Stops METRICBEATS on the specified ip sever.
csle stop metricbeat <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops the METRICBEAT on the specified container.
csle stop heartbeats <emulationname> <emulationid> -ip <ip>	Stops HEARTBEATS on the specified ip sever.
csle stop heartbeat <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops the HEARTBEAT on the specified container.
csle stop packetbeats <emulationname> <emulationid> -ip <ip>	Stops PACKETBEATS on the specified ip sever.
csle stop packetbeat <emulationname> <emulationid> -ip <ip> -container_ip <containerip>	Stops the PACKETBEAT on the specified container.
csle ls hostmanagers <emulationname> -id <emulationid> -ip <ip>	Lists host managers on the specified ip sever.
csle ls clientmanager <emulationname> -id <emulationid> -ip <ip>	List the client manager on the specified ip sever.
csle ls snortmanagers <emulationname> -id <emulationid> -ip <ip>	Lists snort managers on the specified ip sever.
csle ls elkmanager <emulationname> -id <emulationid> -ip <ip>	Lists the Elk manager on the specified ip sever.
csle ls trafficmanagers <emulationname> -id <emulationid> -ip <ip>	Lists traffic managers on the specified ip sever.
csle ls kafkamanagers <emulationname> -id <emulationid> -ip <ip>	Lists the KAFKA manager on specified ip sever.
csle ls ossecmanagers <emulationname> -id <emulationid> -ip <ip>	Lists Ossec managers on the specified ip sever.
csle ls ryuamanager <emulationname> -id <emulationid> -ip <ip>	Lists the RVU manager on a specified ip sever.
csle ls filebeats <emulationname> -id <emulationid> -ip <ip>	Lists FILEBEATS on the specified ip sever.
csle ls metricbeats <emulationname> -id <emulationid> -ip <ip>	Lists METRICBEATS on the specified ip sever.
csle ls heartbeats <emulationname> -id <emulationid> -ip <ip>	Lists HEARTBEATS on the specified ip sever.
csle ls packetbeats <emulationname> -id <emulationid> -ip <ip>	Lists PACKETBEATS on the specified ip sever.
csle ls logfiles <emulationname> -id <emulationid> -ip <ip>	Lists log files on the specified server.
csle ls logfile <emulationname> -id <emulationid> -ip <ip> -logfile_name <logfile_name>	Lists a specified log file on the specified ip sever.
csle ls emulation_description <emulationname> -id <emulationid> -ip <ip>	Describes the specified emulation on specified ip sever.

Table 27: Commands available in the CSLE command-line interface (version 0.10.0) (3/3).

3 Getting Started

This section contains instructions for getting started with CSLE. It includes instructions for the installation and operation of CSLE, as well as instructions on how to uninstall CSLE. A video that walks through the installation process is available at https://www.youtube.com/watch?v=l_g3sRJwwhc.

3.1 Installing CSLE

This section contains instructions for installing CSLE using Ansible [34].

3.1.1 Prerequisites

To install and run CSLE you need at least one server or virtual machine that meets the following criteria:

- UBUNTU 18.04+;
- at least 16GB RAM (the exact amount of RAM necessary depends on the emulations to deploy; 16GB is sufficient for the smallest emulations, e.g., in the size of 5-10 containers);
- at least 2 CPUs;
- 50 GB of free hard disk space;
- outside Internet access;
- and a UNIX user account with `sudo` privileges.

Below are a list of dependencies that will be installed with CSLE (unless they are already installed):

- DOCKER 20.10.14+;
- PYTHON 3.9+;
- PYTHON libraries: `torch`, `numpy`, `gym`, `docker`, `paramiko`, `stable-baselines3`, `psycpg`, `pyglet`, `flask`, `click`, `waitress`, `scp`, `psutil`, `grpcio`, `grpcio-tools`, `scipy`, `confluent-kafka`, `requests`, `pyopenssl`, `sphinx`, `mypy`, `mypy-extensions`, `mypy-protobuf`, `types-PyYAML`, `types-protobuf`, `types-paramiko`, `types-requests`, `types-urllib3`, `flake8`, `flake8-rst-docstrings`, `pytest`, `gevent`, `eventlet`, `dnspython`, `csle-ryu-fork`, `gpytorch`, `pulp`, `Bayesian optimization`, `emukit`, `cma`, `pycryptodome`, `mitreattack-python`.

- POSTGRESQL 15+;
- CITUS 11.2+;
- build-essential, make, git, bzip2;
- PROMETHEUS 2.23+;
- NODE EXPORTER 1.0.1+;
- CADVISOR (any version);
- GRAFANA (any version);
- Node v16.13.1+;
- and `npm` v6.14.8+.

3.1.2 Installation Setup

The network setup for a minimal installation includes (i) a leader server; (ii) zero or more worker servers; and (iii) a user host, e.g., a laptop or workstation (see [Fig. 26.](#))

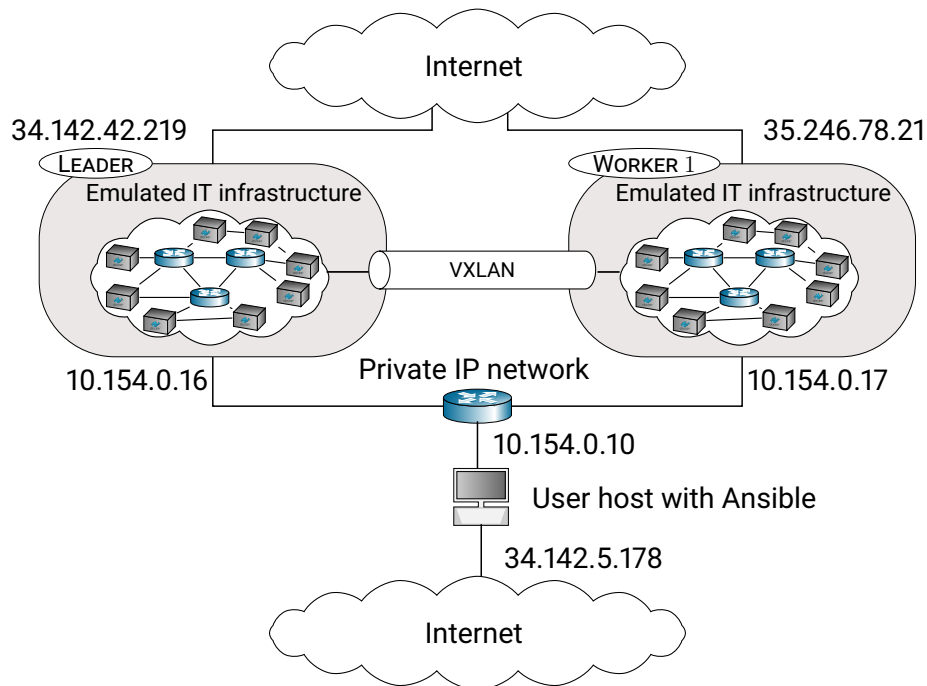


Figure 26: Network topology of a typical CSLE installation; the cluster consists of a leader and zero or more workers; the installation commands are executed from an external user host that runs Ansible [34]; the servers are connected through a private network as well as to the Internet; the user host has access to the private network.

Start by setting up SSH keys so that a) all servers (leader and workers) have SSH access to each other without requiring a password; and b) the user host from which the installation will be executed has SSH access to all servers. To do this, start by generating an SSH key pair on each host using the command `ssh-keygen`. After generating the keys, copy the public key of each host (e.g., `id_rsa.pub`) to the file `.ssh/authorized_keys` on the other hosts.

Next, clone the CSLE GIT repository on the user host by running the following command.

```
1 git clone https://github.com/Kim-Hammar/csle
```

Listing 5: Command to clone the CSLE GIT repository.

Now configure the installation by setting the parameters in the files `csle/ansible/inventory`

```
csle/ansible/groups_vars/all/variables.yml
```

In most cases the default parameters can be used. However, you must always configure the IP addresses of the servers in your cluster in `csle/ansible/inventory` as well as configure the variables `leader_ip`, `leader_public_ip`, `metastore_ip`, and `cluster_nodes` in the file `csle/ansible/groups_vars/all/variables.yml`.

After configuring the installation, install Ansible on the user host by running the command:

```
1 pip install ansible
```

Listing 6: Command to install Ansible [34].

Finally, start the automated installation by running the following commands on the user host:

```
1 cd csle/ansible; ansible-playbook --ask-become-pass install.yml
```

Listing 7: Command to run the automated installation of CSLE with Ansible [34].

The above command will ask for a `sudo` password to the servers. After entering the password, Ansible will install all components of CSLE on all servers in your cluster. This process usually takes around 40 minutes.

3.2 Manual Installation of CSLE

The recommended way to install CSLE is to use Ansible, as described above. This section describes an alternative, manual way of installing CSLE. Manual installation is suitable if you want to customize the installation.

The installation of CSLE can be divided in four main steps (see Fig. 27). The first step is “Installation setup”, which comprises installation configuration and installation of build tools. In the second step, the metastore and the simulation system are installed. Lastly, in the third and the fourth steps, the emulation system and the management system are installed, respectively.

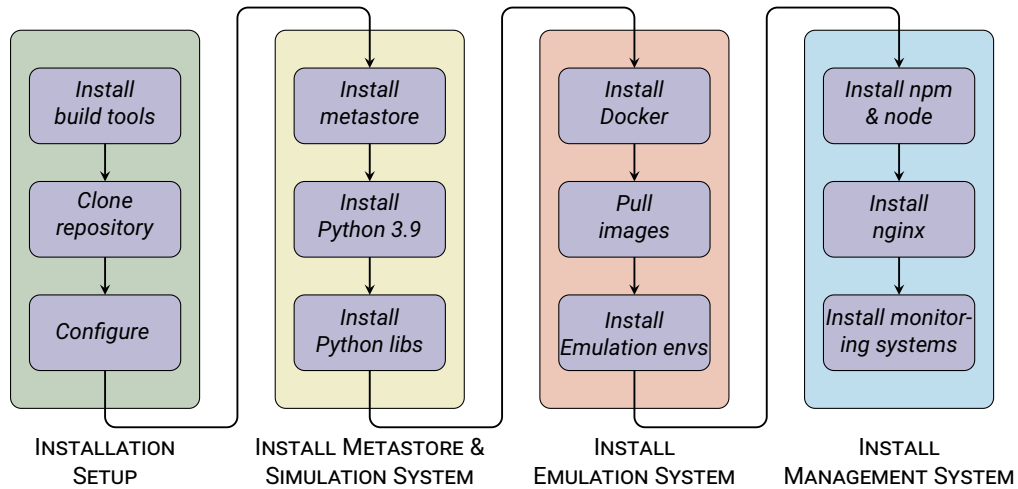


Figure 27: Steps to install CSLE.

3.2.1 Physical Resources

CSLE can be installed on a cluster of servers. All servers need to be connected through an IP network. One of the servers should be designated as the “leader” (see Fig. 3). The other servers are workers. (If you install CSLE on a single server, then that server is the master, and there are no workers.) If not stated otherwise, all commands in the installation instructions below should be executed on every server in the cluster. Certain commands should only be executed on the leader, which will be clearly indicated.

3.2.2 Installation setup

In this step of the installation, the source code of CSLE is downloaded, configuration parameters are set, and tools required in later steps of the installation are installed.

Start with installing the necessary build tools and version management tools by running the following commands:

```

1 sudo apt install build-essential
2 sudo apt install make
3 sudo apt install git
4 sudo apt install bzip2
5 sudo apt install nginx
6 wget
  ↪ https://repo.anaconda.com/archive/Anaconda3-2022.10-Linux-x86_64.sh
7 chmod u+rx Anaconda3-2022.10-Linux-x86_64.sh
8 ./Anaconda3-2022.10-Linux-x86_64.sh

```

Listing 8: Commands to install build tools and version management tools.

Next, clone the CSLE repository and setup environment variables by running the commands:

```

1 git clone https://github.com/Kim-Hammar/csle
2 export CSLE_HOME=/path/to/csle/ # for bash
3 set -gx CSLE_HOME "/path/to/csle" # for fish

```

Listing 9: Commands to clone the CSLE repository and setup environment variables.

Further, add the following line to `.bashrc` to set the environment variable `CSLE_HOME` permanently in the bash shell:

```

1 export CSLE_HOME=/path/to/csle/

```

Listing 10: Line to add to `.bashrc` to set the `CSLE_HOME` environment variable.

and add the following line to the fish configuration file to set the environment variable `CSLE_HOME` permanently in the fish shell:

```

1 set -gx CSLE_HOME "/path/to/csle"

```

Listing 11: Line to add to the fish configuration file to set the `CSLE_HOME` environment variable.

After performing the steps above, you should have the directory layout shown in Fig. 28.

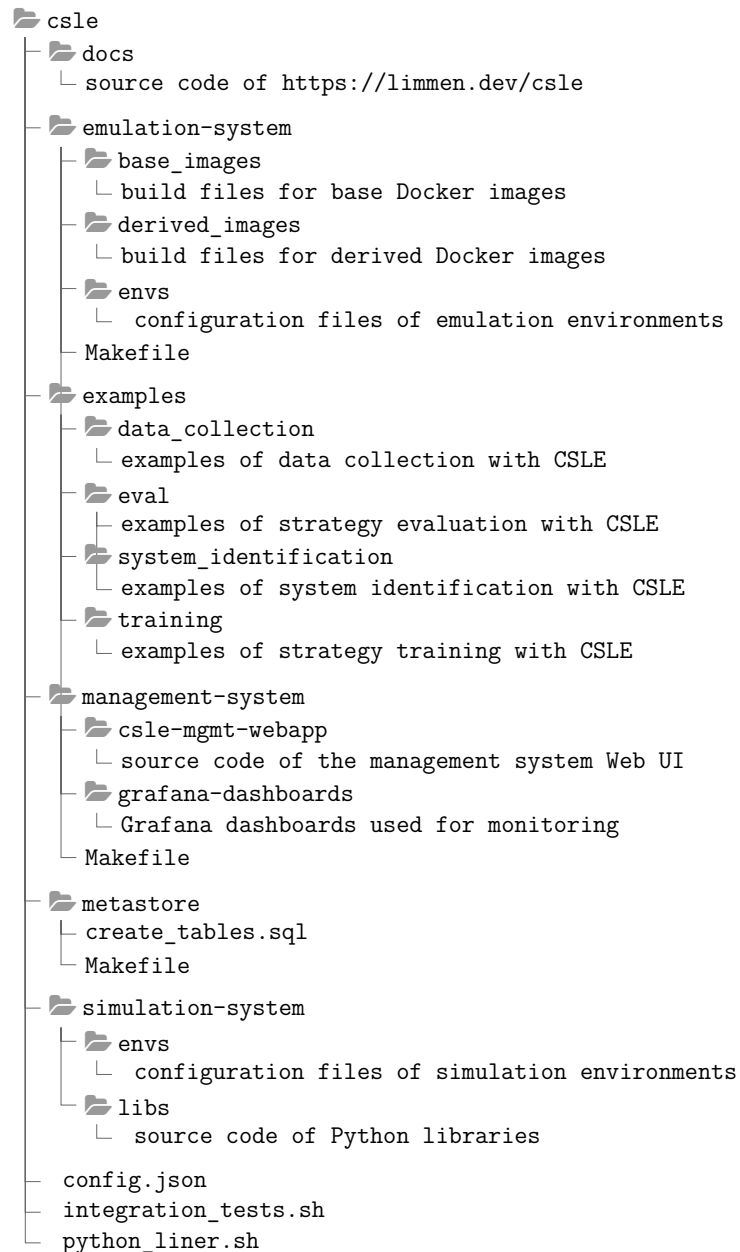


Figure 28: The directory layout of the CSLE code repository.

Next, create a directory to store PID files by running the following commands (change `<my_user>` to your username):

```
1 sudo mkdir /var/log/csle
2 sudo chmod -R u+rw /var/log/csle
3 sudo chown -R <my_user> /var/log/csle
```

Listing 12: Commands to set up the pid file directory of CSLE.

Similarly, create a directory to store log files by running the following commands (change <my_user> to your username):

```
1 mkdir /tmp/csle
2 sudo chmod -R u+rw /tmp/csle
3 sudo chown -R <my_user> /tmp/csle
```

Listing 13: Commands to set up the log file directory of CSLE.

Next, add the following line to the `sudoers` file using `sudo visudo` (change <my_user> to your username):

 **Warning:** take care when editing the `sudoers` file. If the `sudoers` file becomes corrupted, the system will be unusable.

```
1 <my_user> ALL = NOPASSWD: /usr/sbin/service docker stop,
  ↳ /usr/sbin/service docker start, /usr/sbin/service docker
  ↳ restart, /usr/sbin/service nginx stop, /usr/sbin/service
  ↳ nginx start, /usr/sbin/service nginx restart,
  ↳ /usr/sbin/service postgresql start, /usr/sbin/service
  ↳ postgresql stop, /usr/sbin/service postgresql restart,
  ↳ /bin/kill, /usr/bin/journalctl -u docker.service -n 100
  ↳ --no-pager -e
```

Listing 14: Line to add to the `sudoers` file.

By adding the above line to the `sudoers` file, CSLE can view logs and start and stop management services without requiring a password to be entered. (Note that the exact paths used above may differ on your system; verify the paths by running the command `whereis service,whereis journalctl`, etc.)

Next, set up SSH keys so that all servers (leader and workers) have SSH access to each other without requiring a password. To do this, generate an

ssh key pair on each server using the command `ssh-keygen`. After generating the keys, copy the public key of each server (e.g., `id_rsa.pub`) to the file `.ssh/authorized_keys` on the other servers.

Lastly, define the default username and password to the management system by editing the file: `csle/config.json`.

3.2.3 Installing the Metastore

The metastore is based on POSTGRESQL and CITUS. Installing the metastore thus corresponds to installing and configuring POSTGRESQL and CITUS.

To install POSTGRESQL v17 and the CITUS extension v13.0, run the following commands:

```
1 sudo apt install postgresql-17
2 curl https://install.citusdata.com/community/deb.sh >
   ↪ add-citus-repo.sh
3 sudo bash add-citus-repo.sh
4 sudo apt-get -y install postgresql-17-citus-13.0
5 sudo pg_conftool 17 main set shared_preload_libraries citus
6 sudo pg_conftool 17 main set listen_addresses '*'
```

Listing 15: Commands to install POSTGRESQL and the CITUS extension.

Verify the installed version of POSTGRESQL by running the command:

```
1 psql --version
```

Listing 16: Commands to verify the installed version of POSTGRESQL.

Next, set a password for the `postgres` user by running the commands:

```
1 sudo -u postgres psql # start psql session
2 psql> \password postgres # set postgres password
```

Listing 17: Commands to setup a password for the `postgres` user.

Next, set password authentication for the `postgres` user and allow remote connections by performing the following steps:

1. Open the file `"/etc/postgresql/<YOUR_VERSION>/main/pg_hba.conf"` and replace the existing content to match your IP addresses and desired security level:

```
1 local all postgres md5
2 host all all 127.0.0.1/32 trust
3 host all all ::1/128 trust
4 host all all 172.31.212.0/24 trust
```

Listing 18: Lines to add to `pg_hba.conf`; note: to allow external connections `127.0.0.1/32` can be changed to `0.0.0.0/0`.

2. Restart POSTGRESQL with the command:

```
1 sudo service postgresql restart
```

Listing 19: Command to restart POSTGRESQL.

3. Run the following command to have POSTGRESQL restarted automatically when the server is restarted:

```
1 sudo update-rc.d postgresql enable
```

Listing 20: Command to make POSTGRESQL start automatically when the server starts.

After completing the steps above, create the CSLE database and set up the CITUS extension by running the following command:

```
1 cd metastore; make db
```

Listing 21: Commands to create the CSLE database and setup the CITUS extension.

Next, edit the file `csle/metastore/create_cluster.sql` and configure IP addresses of the worker servers and of the leader. Then, **on the leader**, run the following commands to setup the CITUS cluster and create the tables:


```
1 cd metastore; make cluster
2 cd metastore; make tables
```

Listing 22: Commands to set up the CITUS cluster and create tables.

Next, update the variable called `HOST` under the class `METADATA_STORE` in the file

```
csle/simulation-system/libs/csle-common/src/csle/_common
/constants/constants.py
```

Next, define the IP addresses of the cluster nodes and the metastore leader by editing the file: `csle/config.json`.

Lastly, make the `POSTGRES` log files readable by your user by running the commands:

```
1 sudo chmod -R u+rw /var/log/postgresql
2 sudo chown -R <my_user> /var/log/postgresql
```

Listing 23: Commands to make the `POSTGRES` log files readable for a given user.

PostgreSQL Debugging If `POSTGRES` is not starting for some reason, check the reason by running the command:

```
1 sudo service postgresql@17-main status
```

Listing 24: Command to check the status of `POSTGRES`.

You may need to set the permissions of the log and `PID` files, which can be done using the following commands.

```
1 sudo chmod 640 /var/log/postgresql/postgresql-17-main.log
2 sudo chown -R postgres:postgres /var/log/postgresql
3 sudo chmod 750 /var/log/postgresql
4 sudo chown -R postgres:postgres /run/postgresql
5 sudo chmod 755 /run/postgresql
6 sudo systemctl restart postgresql@17-main
```

Listing 25: Commands to set permissions of the `POSTGRES` log files.

You can test the Postgres connection to the database using the following command:

```
1 psql -h localhost -p 5432 -U csle -d csle
```

Listing 26: Command to test the postgres connection.

3.2.4 Installing the Simulation System

The simulation system consists of a set of PYTHON libraries and configuration files. To install the simulation system, the PYTHON libraries need to be installed, and the configuration files need to be inserted into the metastore.

If you do not have PYTHON >3.9 in your base environment, start installing PYTHON 3.11 by running the commands:

```
1 conda create -n py311 python=3.11
2 conda activate py311
```

Listing 27: Command to install PYTHON 3.11 using Anaconda [37].

The simulation system includes 12 PYTHON libraries: csle-collector, csle-ryu, csle-common, csle-attacker, csle-defender, csle-system-identification, gym-csle-stopping-game, csle-agents, csle-rest-api, csle-cli, csle-cluster, gym-csle-intrusion-response-game, csle-base, csle-tolerance, gym-csle-cyborg, gym-csle-apt-game, and csle-attack-profiler. These libraries can either be installed from PYPY⁷ or directly from the source.

To install all libraries at once from PYPY, run the following commands:

```
1 cd simulation-system/libs; ./local_install.sh
2 cd simulation-system/libs; ./local_install_dev.sh
```

Listing 28: Commands to install all CSLE python libraries.

Finally, **on the leader node only**, insert the simulation configurations into the metastore by running the commands:

⁷<https://pypi.org/>

```
1 cd simulation-system/envs
2 make install
3 cd ../../
```

Listing 29: Commands to insert simulation configurations into the metastore.

3.2.5 Installing the Emulation System

The emulation system consists of a set of configuration files and a set of DOCKER images, which are divided into a set of “base images” and a set of “derived images”. The base images contain common functionality required by all images in CSLE, whereas the derived images add specific configurations to the base images, e.g., specific vulnerabilities. To install the emulation system, the configuration files must be inserted into the metastore, and the DOCKER images must be built or downloaded.

Start with adding DOCKER’s official GPG key to UBUNTU’s package manager by running the commands (you can also follow the instructions for installing DOCKER at: <https://docs.docker.com/engine/install/ubuntu/>):

```
1 sudo mkdir -p /etc/apt/keyrings
2 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo
  ↪ gpg --dearmor -o /etc/apt/keyrings/docker.gpg
3 echo "deb [arch=$(dpkg --print-architecture)
  ↪ signed-by=/etc/apt/keyrings/docker.gpg]
  ↪ https://download.docker.com/linux/ubuntu $(lsb_release -cs)
  ↪ stable" | sudo tee /etc/apt/sources.list.d/docker.list >
  ↪ /dev/null
```

Listing 30: Commands to add DOCKER’s official GPG key to UBUNTU’s package manager.

Next, install DOCKER and OVS-SWITCH by running the commands:

```
1 sudo apt-get update
2 sudo apt-get install docker-ce docker-ce-cli containerd.io
  ↪ openvswitch-switch openvswitch-common
3 sudo groupadd docker
4 sudo usermod -aG docker $USER
```

Listing 31: Commands to install DOCKER.

After running the commands above, start a new shell for the changes to take effect.

Next, set up a docker swarm by running the following command on the leader:

```
1 docker swarm init --advertise-addr <ip address of the leader>
```

Listing 32: Command to initialize a DOCKER swarm.

After running the above command, a secret token will be returned. Use this token to run the following command on each worker to add it to the swarm:

```
1 docker swarm join --token <my_token> leader_ip:2377
```

Listing 33: Commands to add a worker node to the DOCKER swarm.

👉 If you forget the swarm token, you can display it by running the following command on the leader: `docker swarm join-token worker`.

On the leader, you can verify that the swarm is set up correctly by running:

```
1 docker node ls
```

Listing 34: Commands to check the nodes in the DOCKER swarm.

After completing the DOCKER installation, pull the base images of CSLE from DOCKERHUB⁸ by running the commands:

```
1 cd emulation-system/base_images
2 make pull
3 cd ../../
```

Listing 35: Commands to pull CSLE base images from DOCKERHUB.

Alternatively, you can build the base images locally (this takes several hours) by running the commands:

```
1 cd emulation-system/base_images
2 make build
3 cd ../../
```

Listing 36: Commands to build CSLE base images.

Next, pull the derived images of CSLE from DOCKERHUB by running the commands:

```
1 cd emulation-system/derived_images
2 make pull
3 cd ../../
```

Listing 37: Commands to pull CSLE derived images from DOCKERHUB.

Alternatively, you can build the derived images locally by running the commands:

⁸<https://hub.docker.com/u/kimham>

```
1 cd emulation-system/derived_images
2 make build
3 cd ../../
```

Listing 38: Commands to build CSLE derived images.

Next, insert the emulation configurations into the metastore by running the commands **on the leader node only**:

```
1 cd emulation-system/envs
2 make install
3 cd ../../
```

Listing 39: Commands to insert emulation configurations into the metastore.

Alternatively, you can install the base images, the derived images, and the emulation configurations all at once by running the commands:

```
1 cd emulation-system
2 make build
3 cd ../
```

Listing 40: Commands to install base images, derived images, and emulation environments.

A few configuration parameters of the kernel need to be updated to execute emulations. In particular, the configuration variables `max_map_count`, `max_user_watches`, and the `ulimit` need to be updated.

Update `max_map_count` by editing the file `/etc/sysctl.conf` and add the following line:

```
1 vm.max_map_count=262144
```

Listing 41: Line to add to `/etc/sysctl.conf`.

Alternatively, for a non-persistent configuration, run the command:

```
1 sudo sysctl -w vm.max_map_count=262144
```

Listing 42: Command to update the configuration variable `max_map_count`.

You can check the configuration by running the command:

```
1 sysctl vm.max_map_count
```

Listing 43: Command to check the configuration of “`max_map_count`”.

Next, update the `ulimit` on the number of open file descriptors by editing the file `/etc/security/limits.conf` and add the following lines:

```
1 <myuser>          soft    nofile    102400
2 <myuser>          hard    nofile    1024000
```

Listing 44: Lines to add to `/etc/security/limits.conf`.

You can check the configuration by running the command:

```
1 ulimit -aH
```

Listing 45: Command to check the `ulimit` configuration.

Finally, update `max_user_watches` by running the command:

```
1 echo fs.inotify.max_user_watches=524288 | sudo tee -a
  ↪ /etc/sysctl.conf && sudo sysctl -p
```

Listing 46: Command to set the configuration variable `max_user_watches`.

3.2.6 Installing the Management System

The management system consists of monitoring systems (i.e., GRAFANA, PROMETHEUS, NODE EXPORTER, CADVISOR, and pgadmin) and the web application that implements the web interface, which is based on `node.js`⁹. Hence, installing the

⁹<https://nodejs.org/en/>

management system corresponds to installing these services and applications.

Start by installing `node.js`, its version manager `nvm`, and its package manager `npm` on all servers (the leader and the workers) by running the commands:

```
1 curl
  ↪ https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh
  ↪ --output nvm.sh
2 chmod u+rx nvm.sh
3 ./nvm.sh
```

Listing 47: Commands to install `nvm`.

Then setup the `nvm` environment variables by adding the following lines to `.bashrc`:

```
1 export NVM_DIR="$HOME/.nvm"
2 [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
```

Listing 48: Commands to setup `NVM` environment variables.

If you are using the fish shell, you can set up `nvm` using the following commands:

```
1 curl
  ↪ https://raw.githubusercontent.com/oh-my-fish/oh-my-fish/master/bin/install
  ↪ | fish # install OMF
2 omf install https://github.com/fabioantunes/fish-nvm # Install
  ↪ fish-nvm
3 omf install bass # Install bass
```

Listing 49: Commands to setup `nvm` with the fish shell.

Then you can install `node.js` and `npm` using the commands:

```
1 nvm -v # Verify nvm installation
2 nvm install node # Install node
3 npm install -g npm # Update npm
4 node -v # Verify the version of node
5 npm -v # Verify the version of npm
```

Listing 50: Commands to install `node.js` and `npm`.

Next, install and build the web application of the management system by running the following commands:

```
1 cd csle/management-system/csle-mgmt-webapp
2 npm install --legacy-peer-deps
3 npm run build
```

Listing 51: Commands to install the web application of the CSLE management system.

 **Note:** if you have an old operating system, you may need to run `export NODE_OPTIONS=--openssl-legacy-provider` before running `npm run build`.

Next, install and start `pgadmin` **on the leader only** by running the following commands:

```
1 docker pull dpape/pgadmin4
2 docker run -p 7778:80 -e "PGADMIN_DEFAULT_EMAIL=user@domain.com"
   ↪ -e "PGADMIN_DEFAULT_PASSWORD=SuperSecret" -d dpape/pgadmin4
```

Listing 52: Commands to start `pgadmin`.

Next, configure `NGINX` on the leader by editing the file:

```
1 /etc/nginx/sites-available/default
```

Listing 53: `NGINX` configuration file.

Replace the current configuration with the following:

```
1 server {
2     listen 80 default_server;
3     listen [::]:80 default_server;
4
5     root /var/www/html;
6
7     index index.html index.htm index.nginx-debian.html;
8
9     server_name _;
10
11     location /pgadmin {
12         proxy_set_header X-Script-Name /pgadmin;
13         proxy_set_header Host $host;
14         proxy_pass http://localhost:7778/;
15         proxy_redirect off;
16     }
17
18     location / {
19         proxy_pass http://localhost:7777/;
20         proxy_buffering off;
21         proxy_set_header X-Real-IP $remote_addr;
22         proxy_set_header X-Forwarded-Host $host;
23         proxy_set_header X-Forwarded-Port $server_port;
24     }
25 }
```

Listing 54: Content of “/etc/nginx/sites-available/default” on the leader.

Restart NGINX on the leader by running the command:

```
1 sudo service nginx restart
```

Listing 55: Command to restart NGINX.

If you have HTTPS enabled on the REST API and have certificates, you can configure them in NGINX on the leader by editing the file

```
1 /etc/nginx/sites-available/default
```

Listing 56: NGINX configuration file.

as follows:

```
1 server {
2     listen 80 default_server;
3     listen [::]:80 default_server;
4     server_name _;
5     return 301 https://$host$request_uri;
6
7     location /pgadmin {
8         proxy_set_header X-Script-Name /pgadmin;
9         proxy_set_header Host $host;
10        proxy_pass http://localhost:7778/;
11        proxy_redirect off;
12    }
13 }
14
15
16 server {
17     listen 443 ssl default_server;
18     listen [::]:443 ssl default_server;
19     ssl_certificate /var/log/csle/certs/csle.dev.crt;
20     ssl_certificate_key /var/log/csle/certs/csle_private.key;
21     root /var/www/html;
22     index index.html index.htm index.nginx-debian.html;
23     server_name csle.dev;
24     location / {
25         proxy_pass http://localhost:7777/;
26         proxy_buffering off;
27         proxy_set_header X-Real-IP $remote_addr;
28         proxy_set_header X-Forwarded-Host $host;
29         proxy_set_header X-Forwarded-Port $server_port;
30     }
31 }
```

Listing 57: Contents of the file "/etc/nginx/sites-available/default" on the leader to allow HTTPS traffic to the web interface.

Next, configure NGINX on the workers by editing the following file

```
1 /etc/nginx/sites-available/default
```

Listing 58: NGINX configuration file.

Open the file on each worker and replace the current configuration with the following (replace “leader-ip” with the actual ip):

```
1 server {
2     listen 80 default_server;
3     listen [::]:80 default_server;
4     root /var/www/html;
5     index index.html index.htm index.nginx-debian.html;
6     server_name _;
7     location /pgadmin {
8         proxy_set_header X-Script-Name /pgadmin;
9         proxy_set_header Host $host;
10        proxy_pass http://leader-ip:7778/;
11        proxy_redirect off;
12    }
13
14    location / {
15        proxy_pass http://localhost:7777/;
16        proxy_buffering off;
17        proxy_set_header X-Real-IP $remote_addr;
18        proxy_set_header X-Forwarded-Host $host;
19        proxy_set_header X-Forwarded-Port $server_port;
20    }
21 }
```

Listing 59: Content of “/etc/nginx/sites-available/default” on a worker.

Next, make the NGINX log files readable by your user by running the commands:

```
1 sudo chmod -R u+rw /var/log/nginx
2 sudo chown -R <my_user> /var/log/nginx
```

Listing 60: Commands to make the NGINX log files readable for a given user.

Lastly, restart NGINX on each worker and on the leader by running the command:

```
1 sudo service nginx restart
```

Listing 61: Command to restart NGINX.

After completing the steps above, install PROMETHEUS and node exporter by running the following commands (these commands should be run on all nodes, both the leader and the workers):

```
1 cd management-system
2 chmod u+x install.sh
3 ./install.sh
```

Listing 62: Commands to install the management system and associated tools.

Next, configure the IP by editing the following files on each server (leader and workers):

```
1 csle/management-system/csle-mgmt-webapp/src
2 /components/Common/serverIp.js
3 csle/management-system/csle-mgmt-webapp/server/server.py
```

Listing 63: File to configure the IPs of servers in the management system.

Next, configure the port of the web interface by editing the following file on each server:

```
1 csle/management-system/csle-mgmt-webapp/src
2 /components/Common/serverPort.js
```

Listing 64: File to configure the port of the web interface.

To start and stop the monitoring systems using the CSLE CLI, their binaries need to be added to the system path.

Add PROMETHEUS binary to the system path by adding the following line to `.bashrc` on all nodes:

```
1 export PATH=/path/to/csle/management-system/prometheus/:$PATH
```

Listing 65: Line to add to `.bashrc` to add PROMETHEUS to the path.

If you have fish shell instead of bash, add the following line to the configuration file of fish:

```
1 fish_add_path /path/to/csle/management-system/prometheus/
```

Listing 66: Line to add to the configuration file of fish to add PROMETHEUS to the path.

Similarly, to add the NODE EXPORTER binary to the path, add the following line to `.bashrc` on all nodes:

```
1 export PATH=/path/to/csle/management-system/node_exporter/:$PATH
```

Listing 67: Line to add to `.bashrc` to add NODE EXPORTER to the path.

If you have fish shell instead of bash, add the following line to the configuration file of fish:

```
1 fish_add_path /path/to/csle/management-system/node_exporter/
```

Listing 68: Line to add to the configuration file of fish shell to add NODE EXPORTER to the path.

Finally, start the csle daemons and setup the management user account with administrator privileges by running the following command on all nodes:

```
1 csle init
```

Listing 69: Command to start the csle daemon and setup the management user account with administrator privileges.

3.3 Quick Start

To verify that CSLE was installed correctly, run the command:

```
1 csle ls --all
```

Listing 70: Command to list all information about CSLE.

The above command lists information about the CSLE installation, including the emulation configurations installed in the metastore. Select one emulation configuration to start an execution, e.g., `csle-level9-0.10.0`, and then run the command:

```
1 csle start csle-level9-0.10.0
```

Listing 71: Command to start an execution of the emulation configuration `csle-level9-0.10.0`.

The above command will start all containers of the emulation and apply the configuration. After this process has been completed, start the monitoring systems and the management system by executing the commands:

```
1 csle start grafana
2 csle start cadvisor
3 csle start prometheus
4 csle start nodeexporter
5 csle start flask
6 csle start nginx
```

Listing 72: Command to start management services of CSLE.

After the above commands have been completed, the web interface of CSLE can be accessed at the URL: <http://localhost:7777/>, from which you can find links to GRAFANA's interface, PROMETHEUS interface, CADVISOR's interface, and interfaces to storage systems (i.e., KAFKA, PRESTO, and ELASTIC-SEARCH).

To get started with reinforcement learning in CSLE, run the examples in the folder: `csle/examples`, e.g.,

```
1 python csle/examples/training/ppo/  
2     stopping_pomdp_defender/run_vs_random_attacker_v_001.py
```

Listing 73: Command to run the Proximal Policy Optimization (PPO) algorithm for learning defender strategies in CSLE.

3.4 Uninstalling CSLE

To uninstall CSLE, run the commands:

```
1 csle rm all  
2 csle clean emulations  
3 csle rm emulations  
4 pip uninstall gym-csle-stopping-game  
5 pip uninstall gym-csle-apt-game  
6 pip uninstall gym-csle-cyborg  
7 pip uninstall gym-csle-intrusion-response-game  
8 pip uninstall csle-tolerance  
9 pip uninstall csle-attack-profiler  
10 pip uninstall csle-common  
11 pip uninstall csle-collector  
12 pip uninstall csle-agents  
13 pip uninstall csle-ryu  
14 pip uninstall csle-rest  
15 pip uninstall csle-system-identification  
16 pip uninstall csle-attacker  
17 pip uninstall csle-base  
18 pip uninstall csle-cluster  
19 pip uninstall csle-cli  
20 cd emulation-system && make rm  
21 cd metastore; make clean
```

Listing 74: Commands to uninstall CSLE.

Also remove the CSLE_HOME environment variable from .bashrc.

3.5 Operating CSLE

This section describes commands and procedures that are useful when operating CSLE. The framework can be operated in two ways, either through the web interface or through the CLI (see Fig. 29). This section focuses on the

CLI, but similar commands can be invoked through the web interface, which should be self-explanatory.

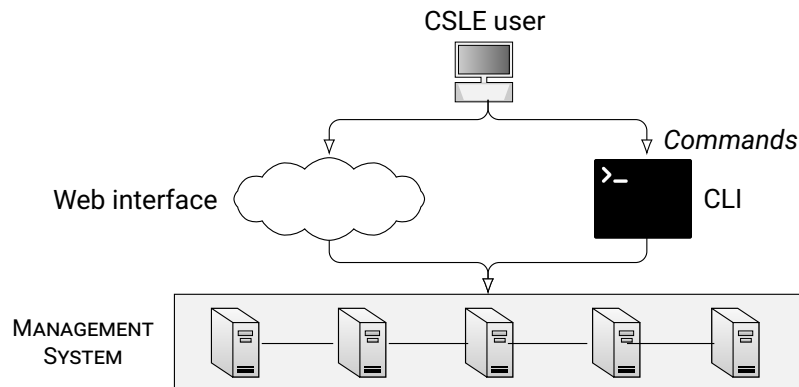


Figure 29: User interfaces of CSLE; a user can execute commands to the management system through two interfaces: a web interface and a Command-Line Interface (CLI).

3.5.1 Listing the Available Commands in the CLI

To list the available commands in the CSLE CLI, run the command:

```
1 csle help
```

Listing 75: Command to list the available commands in the CSLE CLI.

3.5.2 Listing the State of CSLE

Information about the CSLE installation and its current state can be listed by executing the following command:

```
1 csle ls --all
```

Listing 76: Command to list the state of a CSLE installation.

3.5.3 Starting, Stopping, and Resetting the Metastore

The metastore can be started by executing the following command:

```
1 sudo service postgresql start
```

Listing 77: Command to start the metastore.

The metastore can be stopped by executing the command:

```
1 sudo service postgresql stop
```

Listing 78: Command to stop the metastore.

To reset the metastore, execute the commands:

```
1 cd metastore; make clean
2 cd metastore; make build
```

Listing 79: Commands to reset the metastore.

3.5.4 Starting and Stopping the REST API Server

The REST API server can be started by executing the command:

```
1 csle start flask
```

Listing 80: Command to start the REST API server.

The REST API server can be stopped by executing the command:

```
1 csle stop flask
```

Listing 81: Command to stop the REST API server.

3.5.5 Starting and Stopping Monitoring Systems

The monitoring systems GRAFANA, CADVISOR, NODE EXPORTER, and PROMETHEUS can be started by executing the commands:

```
1 csle start grafana
2 csle start cadvisor
3 csle start nodeexporter
4 csle start prometheus
```

Listing 82: Commands to start monitoring systems.

Similarly, GRAFANA, CADVISOR, NODE EXPORTER, and PROMETHEUS can be stopped by executing the commands:

```
1 csle stop grafana
2 csle stop cadvisor
3 csle stop nodeexporter
4 csle stop prometheus
```

Listing 83: Commands to stop monitoring systems.

3.5.6 Starting and Stopping Emulation Executions

To start an execution of an emulation configuration with the name `csle-level9-0.10.0`, execute the following command:

```
1 csle start csle-level9-0.10.0
```

Listing 84: Command to start an execution of the emulation with configuration `csle-level9-0.10.0`.

Similarly, to stop an execution of an emulation configuration with the name `csle-level9-0.10.0`, execute the following command:

```
1 csle stop csle-level9-0.10.0
```

Listing 85: Command to stop an execution of the emulation with configuration `csle-level9-0.10.0`.

The above command will stop all containers but will not remove them, which means that the emulation can be started again with the same configuration by running the command in Listing 84.

To stop an emulation execution and remove all of its containers and virtual networks, run the command:

```
1 csle clean csle-level9-0.10.0
```

Listing 86: Command to stop and clean an execution of the emulation with configuration `csle-level9-0.10.0`.

3.5.7 Access a Terminal in an Emulated Container

To see which containers are running, execute the command:

```
1 csle ls --all
```

Listing 87: Command to list running CSLE containers.

To open a terminal in a running container, e.g., a container with the name `mycontainer`, run the command:

```
1 csle shell mycontainer
```

Listing 88: Command to open a terminal in an emulated container.

3.6 Examples

Examples of CSLE usages can be found at: <https://github.com/Kim-Hammar/csle/examples>. This folder includes examples of the four main usages of CSLE: (1) data collection; (2) system identification; (3) strategy training; and (4), strategy evaluation.

3.6.1 Data collection

A common usage of CSLE is to run emulated cyber attacks against an emulated IT infrastructure and record the resulting traces of system metrics and logs. To do this with CSLE, you have to a) choose a running emulation execution; b) define the sequence of attacker actions; c) define the sequence of defender actions; and d), start the attacker and defender sequences. A code example of performing these steps using the CSLE APIs is given below.

```

1 import csle_common.constants.constants as constants
2 from ..emulation_attacker_action import EmulationAttackerAction
3 from ..emulation_defender_action import EmulationDefenderAction
4 from ..emulation_attacker_stopping_actions import
  ↪ EmulationAttackerStoppingActions
5 from ..emulation_defender_stopping_actions import
  ↪ EmulationDefenderStoppingActions

6 from ..emulation_env_config import EmulationEnvConfig
7 from ..metastore_facade import MetastoreFacade
8 from ..container_controller import ContainerController
9 from csle_system_identification.emulator import Emulator
10 # Select emulation execution
11 execution = get_emulation_execution(ip_first_octet=..,
  ↪ emulation_name=..)

12 emulation_env_config = executions.emulation_env_config
13 attacker_sequence = [...] # Define attacker sequence
14 defender_sequence = [...] # Define defender sequence
15 # Starting the attacker and defender
16 Emulator.run_action_sequences(
17     emulation_env_config=emulation_env_config,
18     attacker_sequence=attacker_sequence,

19     defender_sequence=defender_sequence)
20 # Extract recorded traces and statistics
21 statistics = MetastoreFacade.list_emulation_statistics()
22 traces = MetastoreFacade.list_emulation_traces()

```

Listing 89: Example of collecting attack and system traces with CSLE.

3.6.2 System Identification

A key step in CSLE's reinforcement learning method is *system identification* (see Fig. 1). In this step, data collected from emulated IT infrastructures are used to identify system parameters and models. A code example of system identification with CSLE is given below.

```
1 import csle_common.constants.constants as constants
2 from csle_common.dao.system_identification.
3     system_identification_config
4     import SystemIdentificationConfig
5 from csle_common.metastore.metastore_facade import
6     ↪ MetastoreFacade
7 from csle_common.dao.system_identification.system_model_type
8     ↪ import SystemModelType
9 from csle_common.dao.training.hparam import HParam
10 from csle_system_identification.expectation_maximization.
11     expectation_maximization_algorithm \
12     import ExpectationMaximizationAlgorithm
13 import csle_system_identification.constants.constants as
14     ↪ sid_consts
15 # Select emulation config
16 emulation_env_config =
17     ↪ MetastoreFacade.get_emulation_by_name("csle-level9-002")
18 # Extract statistics from the metastore
19 emulation_statistic =
20     ↪ MetastoreFacade.get_emulation_statistic(id=1)
21 system_identification_config = SystemIdentificationConfig(
22     output_dir=f"{constants.LOGGING.DEFAULT_LOG_DIR}em_level9_test",
23     title="Expectation-Maximization level 9 test",
24     model_type=SystemModelType.GAUSSIAN_MIXTURE,
25     log_every=1,
26     hparams={..})
27 algorithm = ExpectationMaximizationAlgorithm(
28     emulation_env_config=emulation_env_config,
29     emulation_statistics=emulation_statistic,
30     system_identification_config=system_identification_config)
31 system_model = algorithm.fit() # Run the algorithm
32 MetastoreFacade.save_gaussian_mixture_system_model(
33     gaussian_mixture_system_model=system_model) # Save system model
34     ↪ in metastore
```

Listing 90: Example of system identification through expectation maximization with CSLE.

3.6.3 Strategy Training

CSLE includes several numerical algorithms for optimizing defender strategies, e.g., reinforcement learning algorithms, dynamic programming algorithms, game-theoretic algorithms, and general optimization algorithms. A code example of learning security strategies through the Proximal Policy Optimization (PPO) reinforcement learning algorithm is given below.

```
1 import csle_common.constants.constants as constants
2 from csle_common.dao.training.experiment_config import
  ↳ ExperimentConfig
3 from csle_common.metastore.metastore_facade import
  ↳ MetastoreFacade
4 from csle_common.dao.training.agent_type import AgentType
5 from csle_common.dao.training.hparam import HParam
6 from csle_common.dao.training.player_type import PlayerType
7 from csle_agents.agents.ppo.ppo_agent import PPOAgent
8 import csle_agents.constants.constants as agents_constants
9
10 emulation_env_config =
  ↳ MetastoreFacade.get_emulation_by_name("..")
11 # Select simulation environment
12 simulation_env_config =
  ↳ MetastoreFacade.get_simulation_by_name("..")
13 # Setup experiment with hyperparameters
14 experiment_config = ExperimentConfig(output_dir="..", title="PPO
  ↳ test", random_seeds=[..], agent_type=AgentType.PPO,
  ↳ log_every=1, hparams={..}, player_type=PlayerType.DEFENDER,
  ↳ player_idx=0)
15 agent = PPOAgent(emulation_env_config=emulation_env_config,
  ↳ simulation_env_config=simulation_env_config,
  ↳ experiment_config=experiment_config)
16 # Run the algorithm
17 experiment_execution = agent.train()
18 # Save the results and the learned policies
19 MetastoreFacade.save_experiment_execution(experiment_execution)
20 for policy in experiment_execution.result.policies.values():
21     MetastoreFacade.save_ppo_policy(ppo_policy=policy)
```

Listing 91: Example of strategy optimization through the Proximal Policy Optimization (PPO) reinforcement learning algorithm in CSLE.

3.6.4 Strategy Evaluation

CSLE can be used to evaluate learned security strategies in emulated infrastructures running on the emulation system (see Fig. 1). Below is a code example of strategy evaluation with CSLE.

```
1 import gym
2 import csle_common.constants.constants as constants
3 from csle_common.metastore.metastore_facade import
  ↳ MetastoreFacade
4 from csle_common.dao.training.multi_threshold_stopping_policy
  ↳ import MultiThresholdStoppingPolicy
5 from
  ↳ gym_csle_stopping_game.envs.stopping_game_pomdp_defender_env
  ↳ import StoppingGamePomdpDefenderEnv
6 from
  ↳ csle_system_identification.environment_evaluations.stopping_game_emulation_eval
  ↳ import StoppingGameEmulationEval
7 from csle_common.dao.training.player_type import PlayerType
8 from csle_common.dao.training.agent_type import AgentType
9 # Select emulation to be used as evaluation environment
10 emulation_env_config =
  ↳ MetastoreFacade.get_emulation_by_name("..")
11 # Select simulation environment
12 simulation_env_config =
  ↳ MetastoreFacade.get_simulation_by_name("..")
13 config = simulation_env_config.simulation_env_input_config
14 env = gym.make(simulation_env_config.gym_env_name,
  ↳ config=config)
15 # Define security policy to evaluate
16 tspsa_policy = MultiThresholdStoppingPolicy(..)
17 # Perform the evaluation
18 StoppingGameEmulationEval.emulation_evaluation(env=env,
  ↳ n_episodes=10, intrusion_seq=[..],
  ↳ defender_policy=tspsa_policy,
  ↳ emulation_env_config=emulation_env_config,
  ↳ simulation_env_config=simulation_env_config)
```

Listing 92: Example of evaluating learned security strategies in the emulation system of CSLE.

4 Developer Guide

This section contains guidelines for developers. It includes development conventions (§4.1), instructions on how to install development tools (§4.2), guidelines for writing documentation (§4.3), guidelines for debugging (§4.4), instructions on how to run tests (§4.5), instructions on how to perform static code analysis (§4.6), guidelines for dependency management (§4.7), and guidelines for release management (§4.8).

4.1 Development Conventions

Being a project of a certain size and complexity, it is important that the general development conventions of CSLE are followed. By adopting good practices, it will be easier to deal with the growth of this project.

Please note that if you decide to contribute to CSLE, you automatically accept the licensing conditions of this project (CC BY-SA 4.0 license).

4.1.1 General Principles

CSLE tries to follow a few high-level principles in making technical and community decisions, which are listed below. They are goals to shoot for and may not be followed perfectly all the time.

- **Code over configuration.** We aim to make CSLE fully programmable; everything from starting/stopping emulations to configuring a service running on a container should be possible through a program function. To achieve this level of programmability, as much as possible should be defined in code rather than configuration files. If a configuration file is necessary, it should be defined in a serialization format that can easily be converted back to code, e.g., a JSON file that maps to a PYTHON class. Another benefit of defining configuration in code is that we can run the configuration through the code quality toolchain to identify bugs and style errors.
- **Separation of concerns.** CSLE is divided into components (systems), e.g., the management system, the emulation system, and the simulation system. These components interact via APIs and via a shared database (the metastore). As much as possible, each component should be independent of the other components. This principle allows users to install individual components without installing the other components.

- **Release early and often.** We should emphasize smaller, more iterative releases over large and complex ones. This keeps our documentation in line with the latest releases and minimizes the disruption (and subsequent maintenance burden) associated with significant changes. Creating a release is relatively simple and quick, so don't hesitate to release patch versions (or minor versions) as appropriate.

4.1.2 Structure of CSLE

CSLE comprises numerous libraries and artifacts in different programming languages (mainly PYTHON, JAVASCRIPT, BASH, and DOCKERFILES). When you initially consider contributing to CSLE, you might be unsure about which of those libraries implement the functionality you want to change or report a bug for. The list below, which explains the main components of CSLE and their location in the code repository, should help you with that.

- `examples/`. Examples of using CSLE.
- `emulation-system/base_images` and `emulation-system/derived_images`. DOCKERFILES that define the container images of the emulation system.
- `emulation-system/envs`. Emulation configurations.
- `metastore`. SQL files that define the data model of the metastore.
- `management-system/csle-mgmt-webapp`. A web application that implements the web interface of CSLE (see §2.6).
- `simulation-system/envs`. Simulation configurations.
- `simulation-system/libs/csle-base`. A PYTHON library with base classes and definitions used by the other PYTHON libraries in CSLE.
- `simulation-system/libs/csle-agents`. A PYTHON library with implementations of control, learning, and game-theoretic algorithms for finding defender strategies.
- `simulation-system/libs/csle-attacker`. A PYTHON library that contains code for emulating attacker actions (see §2.4.6).
- `simulation-system/libs/csle-cli`. A PYTHON library with the CSLE CLI (see §2.7).
- `simulation-system/libs/csle-collector`. A PYTHON library that implements the monitoring and management agents (see §2.4.9 and §2.4.10).

- `simulation-system/libs/csle-common`. A PYTHON library that contains common functionality to all CSLE PYTHON libraries.
- `simulation-system/libs/csle-defender`. A PYTHON library that contains code for emulating defender actions (see §2.4.7).
- `simulation-system/libs/csle-rest-api`. A PYTHON library with the CSLE REST API (see §2.6.3).
- `simulation-system/libs/csle-ryu`. A PYTHON library with RYU SDN controllers.
- `simulation-system/libs/csle-cluster`. A PYTHON library with a GRPC server for cluster management in CSLE.
- `simulation-system/libs/csle-system-identification`. A PYTHON library with implementations of system identification algorithms to learn system models based on measured data and traces.
- `simulation-system/libs/gym-csle-stopping-game`. A gym environment for the optimal stopping game in [27].
- `simulation-system/libs/gym-csle-apt-game`. A gym environment for an APT stopping game.
- `simulation-system/libs/gym-csle-cyborg`. A gym environment wrapper for CYBORG.
- `simulation-system/libs/gym-csle-intrusion-response-game`. A gym environment for the intrusion response game in [29].
- `simulation-system/libs/csle-tolerance`. An intrusion-tolerant system: Tolerance: (T)w(o)-(l)ev(e)l (r)ecovery (a)nd respo(n)se (c)ontrol with f(e)edback. (to be published).
- `simulation-system/libs/csle-attack-profiler`. An attack profiler based on MITRE ATT&CK.

4.1.3 Code Readability

The code style in CSLE is based on the PEP 8 standard [57] and is defined using the `flake8` PYTHON linter and the `eslint` JAVASCRIPT linter. You can configure your IDE or editor to automatically enforce these guidelines.

The configuration file of the PYTHON linter is located at:

```
1 csle/.flake8
```

Listing 93: Configuration file for the `flake8` PYTHON linter.

The configuration file of the JAVASCRIPT linter is located at:

```
1 csle/management-system/csle-mgmt-webapp/.eslintrc.json
```

Listing 94: Configuration file for the `eslint` JAVASCRIPT linter.

Names of variables, functions, methods, etc., should be clear and descriptive, not cryptic. All PYTHON functions and variables should be written in snake_case, e.g., `stop_all_executions()`. All JAVASCRIPT functions and variables should be in CamelCase, e.g., `getAgentType()`.

It is common practice to name simple loop variables `i`, `j`, and `k`, so there's no need to give them silly names like `the_index` unless necessary for some reason or other.

Avoid long lines (>120 characters) if possible. This principle makes pull requests smaller, makes the code more readable, and benefits co-developers editing code in Emacs/Vim over SSH and/or in narrower windows.

It is highly recommended that you configure your editor or Integrated Development Environment (IDE) to automatically enforce the style guidelines.

4.1.4 Comments in Code

All functions and classes should have comments documenting the input/output arguments, the purpose of the function/class, and its return value. These comments are used to automatically generate API documentation (this process is described in §4.3).

Example of a comment to a PYTHON function:

```

1  @staticmethod
2  def stop_all_executions() -> None:
3      """
4      Stops all emulation executions
5
6      :return: None
7      """
8      executions = MetastoreFacade.list_emulation_executions()
9      for exec in executions:
10         EmulationEnvController.stop_containers(execution=exec)
11         ContainerController.stop_docker_stats_thread(
12             execution=exec)

```

Listing 95: Example of a PYTHON function with a comment.

Example of a comment to a JAVASCRIPT function:

```

1  const convertListToCommaSeparatedString = (listToConvert) => {
2      /**
3       * Converts a list of strings into a single comma-separated
4       * string
5       *
6       * @param listToConvert the list to convert
7       * @returns {string} the comma separated string
8       */
9      var str = ""
10     for (let i = 0; i < listToConvert.length; i++) {
11         if (i !== listToConvert.length-1) {
12             str = str + listToConvert[i] + ", "
13         } else {
14             str = str + listToConvert[i]
15         }
16     }
17     return str
18 }

```

Listing 96: Example of a JAVASCRIPT function with a comment.

4.1.5 Unit and Integration Testing

Every new change to the code must pass the tests (tests are performed automatically on each pull request). Whenever a new feature is added to CSLE,

a corresponding test should be added (see §4.5 for instructions on how to add tests and see §4.1.7 for the pipeline that automates the tests).

4.1.6 How We Use Git

We use the GIT-flow branching model, in which branches are categorized into:

- A master branch.
- A main development branch.
- One or several feature branches.
- One or several hotfix branches.

The code on the main branch (often called `main` or `release`) is stable, adequately tested, and is the version of the code that a typical user should pick. No changes are made directly on the master branch.

The code on the development branch (often called `develop`) should be working but without guarantees. For small features, development might well happen directly on the development branch, and the code here may, therefore, sometimes be broken (this should ideally never happen, though). When the development branch is deemed "done" and has undergone testing and review, it is merged into the master branch. The release is then tagged with an appropriate release version.

A feature branch (often called `feature/some_name` where `some_name` is a very short descriptive name of the feature) is branched off from the main development branch when a new "feature" is being implemented. A new feature is any logically connected set of changes to the code base, regardless of how many files are being changed.

A hotfix branch (often called `hotfix/some_name`) is a branch that implements a bugfix to a release. In terms of branching, it is thus very similar to a feature branch but for the master branch rather than for the development branch. A hotfix should fix critical errors not caught in testing before the release was made. Hotfixes should not implement new behavior unless necessary to fix a critical bug. Hotfixes need to undergo review before they are merged back into the master and development branches.

The master and development branches are never deleted, while the others are transient (temporary, for the duration of the development and code review).

The benefits of this type of branching model in development are:

- Co-developers work on separate branches and do not "step on each other's toes" during the development process, even if they push their work back to GITHUB.

- Co-developers and users have a stable master branch to use.
- Features in "feature" branches are independent of each other. Any conflicts are resolved when merging.

Commit often, possibly several times a day. It's easier to roll back a small commit than to roll back large commits. This practice also makes the code easier to review. Remember to push the commits to GITHUB every once in a while, too. Write a helpful commit message with each commit that describes what the changes are and possibly even why they were necessary.

4.1.7 Continuous Integration

We use continuous integration (CI) with GITHUB Actions¹⁰ to build the project and run tests on every pull request submitted to CSLE. The CI pipeline used in CSLE is illustrated in Fig. 30. Developers make commits on a branch separated from the master/main branch. Once a developer has completed a bugfix or a new feature, he/she submits a pull request to GITHUB. The pull request then triggers a set of automated tests and automated builds using GITHUB actions. If all automated tests pass, the pull request undergoes code review, otherwise the developer has to fix the failing tests or builds. Finally, when the code review and associated fixes are completed, the pull request is merged into the main/master branch and may optionally trigger a release pipeline where build artifacts are pushed to code servers (i.e., DOCKERHUB and PYPI).

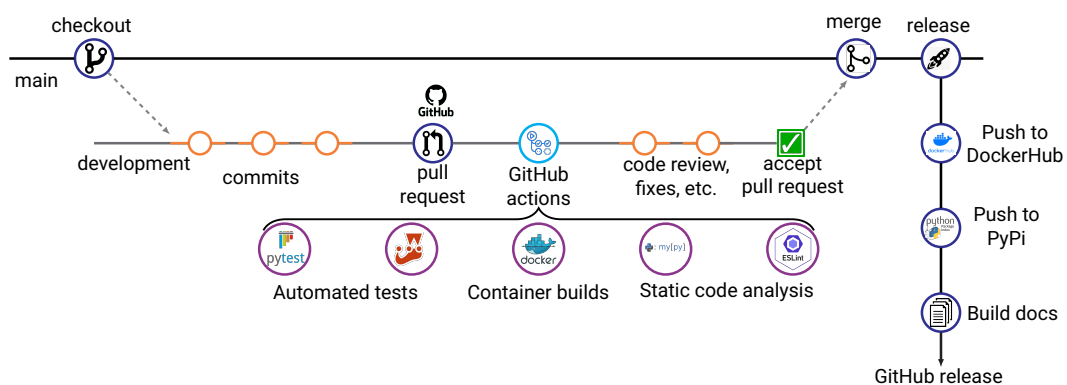


Figure 30: The continuous integration (CI) pipeline of CSLE.

4.1.8 Bug Reports

Please be aware of the following things when filing bug reports:

¹⁰<https://github.com/features/actions>

1. Avoid raising duplicate issues. Use the GITHUB issue search feature to check whether your bug report or feature request has been mentioned in the past. Duplicate bug reports and feature requests are a huge maintenance burden on CSLE's limited resources. If it is clear from your report that you would have struggled to find the original issue, that's ok, but if searching for a selection of words in your issue title would have found the original issue, the duplicate issue will likely be closed abruptly.
2. When filing bug reports about exceptions or tracebacks, please include the complete traceback. Partial tracebacks, or just the exception text, are not helpful. Issues that do not contain complete tracebacks may be closed without warning.
3. Make sure you provide a suitable amount of information to work with. This means that you should provide:
 - Guidance on how to reproduce the issue. Ideally, this should be a small code sample that can be run immediately by the maintainers. Failing that, let us know what you're doing, how often it happens, what environment you're using, etc. Be thorough: it prevents us from needing to ask further questions.
 - Tell us what you expected to happen. When we run your example code, what are we expecting to happen? What does "success" look like for your code?
 - Tell us what happens. It's not helpful for you to say "it doesn't work" or "it fails". Tell us how it fails: do you get an exception? A hang? A non-200 status code? How was the actual result different from your expected result?

If you do not provide all of these things, it will take us much longer to fix your problem. If we ask you to clarify these things and you never respond, we will close your issue without fixing it.

4.1.9 Contribution flow

Our contribution flow is very straightforward and follows an issue-pull request workflow. Contributors must fork the repository to have their contributions landed on the project. If you want to contribute, we have laid down a series of steps that we would like you to follow.

1. **Find an issue to work on.** The first step is to go through the issues to find one that you would like to contribute to (or possibly open a new

issue). Exploring the issues and pull requests will give you an idea of how the contribution flow works. Upon finding something to work on, you can either request that the issue be assigned to you (if someone else has created the Issue) or make your own. To ensure that the issue is received positively by the maintainers, make sure of the following:

- If you are filing a bug report, make sure that the report follows the guidelines stated in §4.1.8.
 - If you are filing a feature request, make sure that you pitch your idea well and constructively.
2. **Getting your pull request merged.** To make sure you make a great contribution, we request you follow the standard Git & GITHUB workflow for code collaboration. We would recommend:
 - Every pull request should have the corresponding issue linked to it.
 - Every pull request should pass the automated CI checks.
 - Every pull request should be as atomic as possible.
 - Every pull request should include a test verifying the new/fixed behavior.
 3. **Reviewing pull requests.** After landing your pull request, the maintainers will review it and provide actionable feedback. Maintainers expect the comments to be resolved once a review has been completed. You can provide updates if you are still working on it to help us understand the areas where we can help.

4.2 Installing development tools

This section contains instructions on installing development tools that CSLE uses, such as test libraries and static code analyzers.

To install all PYTHON build tools at once, run the following commands:

```

1 cd csle/simulation-system/libs/csle-base; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
2 cd csle/simulation-system/libs/csle-collector; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
3 cd csle/simulation-system/libs/csle-ryu; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
4 cd csle/simulation-system/libs/csle-common; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
5 cd csle/simulation-system/libs/csle-attacker; pip install -r
  ↪ requirements_dev.txt; cd ../../../../

6 cd csle/simulation-system/libs/csle-defender; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
7 cd csle/simulation-system/libs/csle-system-identification; pip
  ↪ install -r requirements_dev.txt; cd ../../../../
8 cd csle/simulation-system/libs/gym-csle-stopping-game; pip
  ↪ install -r requirements_dev.txt; cd ../../../../
9 cd csle/simulation-system/libs/gym-csle-apt-game; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
10 cd csle/simulation-system/libs/gym-csle-cyborg; pip install -r
  ↪ requirements_dev.txt; cd ../../../../

11 cd csle/simulation-system/libs/csle-agents; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
12 cd csle/simulation-system/libs/csle-rest-api; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
13 cd csle/simulation-system/libs/csle-cli; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
14 cd csle/simulation-system/libs/csle-cluster; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
15 cd csle/simulation-system/libs/gym-csle-intrusion-response-game;
  ↪ pip install -r requirements_dev.txt; cd ../../../../

16 cd csle/simulation-system/libs/csle-tolerance; pip install -r
  ↪ requirements_dev.txt; cd ../../../../
17 cd csle/simulation-system/libs/csle-attack-profiler; pip install
  ↪ -r requirements_dev.txt; cd ../../../../

```

Listing 97: Commands to install the development dependencies.

It is also possible to install all development tools at once by running the script `simulation-system/libs/local_install_dev.sh`.

Alternatively, install each development library separately by following the commands below.

The `PYTHON` build tool, which is used to package `PYTHON` libraries, can be installed by running the command:

```
1 pip install -q build
```

Listing 98: Command to install the `PYTHON` build tool.

The `twine` tool (a tool for publishing `PYTHON` packages to `PYPI`) can be installed by running the command:

```
1 python3 -m pip install --upgrade twine
```

Listing 99: Command to install `twine`.

The `flake8` `PYTHON` linter can be installed by running the command:

```
1 python -m pip install flake8
```

Listing 100: Command to install `flake8`.

The `flake8-rst-docstrings` extension to the `flake8` `PYTHON` linter can be installed by running the command:

```
1 python -m pip install flake8-rst-docstrings
```

Listing 101: Command to install `flake8-rst-docstrings`.

The `mypy` static type checker for `PYTHON` can be installed by running the command:

```
1 python3 -m pip install -U mypy mypy-extensions mypy-protobuf  
↪ types-PyYAML types-protobuf types-paramiko types-requests  
↪ types-urllib3
```

Listing 102: Command to install `mypy` and the associated type libraries.

The `pytest` and associated test libraries for PYTHON can be installed by running the command:

```
1 pip install -U pytest mock pytest-mock pytest-cov pytest-grpc
```

Listing 103: Command to install `pytest` and `mock`.

Ruby and its bundler, which are used to generate the CSLE documentation page (<https://kim-hammar.github.io/csle/>), can be installed by running the commands:

```
1 sudo apt-get install ruby ruby-dev
2 sudo gem install bundler
```

Listing 104: Commands to install Ruby and its bundler.

The `sphinx` PYTHON library for automatic generation of API documentation can be installed by running the commands:

```
1 pip install sphinx sphinxcontrib-napoleon sphinx-rtd-theme
```

Listing 105: Commands to install `sphinx`.

Lastly, the `tox` PYTHON library for automatic testing can be installed by running the command:

```
1 pip install tox
```

Listing 106: Command to install `tox`.

4.3 Writing Documentation

There are three kinds of documentation in CSLE:

- Release documentation (the document you are reading now). This document is generated for each major release only. It is a PDF generated through $\text{\LaTeX}$.
- Live web documentation at <https://kim-hammar.github.io/csle/>. This documentation is generated with Ruby. The source files are located at `csle/docs`. This documentation is updated automatically on every commit to the project. To generate the web documentation manually, run the command:

```
1 bundle exec Jekyll serve
```

Listing 107: Command to generate the web documentation at <https://kim-hammar.github.io/csle/>.

- PYTHON API documentation. This documentation is generated automatically from code comments using `sphinx`. To generate the API documentation, run the command:

```
1 simulation-system/libs/generate_docs.sh
```

Listing 108: Command to generate the PYTHON API documentation.

4.4 Log files and Debugging

Log files of physical servers in the management system are located at `/tmp/csle`, and PID files are located at `/var/log/csle`. Log files of emulated devices are located at `/`.

To debug management services, check the log files and locate the error. Then, try to fix the error and restart the services using the CLI. To debug emulated components, use SSH or `docker exec`¹¹ to open a terminal inside the component and check the log files.

If you cannot resolve the issue, we recommend restarting all services and seeing if you can reproduce the issue. If you can reproduce the issue and you believe it is a bug, please file a bug report following the instructions listed in §4.1.8.

¹¹<https://docs.docker.com/engine/reference/commandline/exec/>

4.5 Tests

CSLE includes a comprehensive suite of unit tests and integration tests. In this section, we describe how to run these tests.

4.5.1 Unit Tests

CSLE uses `pytest`¹² for PYTHON unit tests and `jest` for JAVASCRIPT unit tests¹³.

The PYTHON unit tests are available at:

- `csle/simulation-system/libs/csle-base/tests`
- `csle/simulation-system/libs/csle-agents/tests`
- `csle/simulation-system/libs/csle-attacker/tests`
- `csle/simulation-system/libs/csle-collector/tests`
- `csle/simulation-system/libs/csle-common/tests`
- `csle/simulation-system/libs/csle-defender/tests`
- `csle/simulation-system/libs/csle-rest-api/tests`
- `csle/simulation-system/libs/csle-ryu/tests`
- `csle/simulation-system/libs/csle-system-identification/tests`
- `csle/simulation-system/libs/gym-csle-stopping-game/tests`
- `csle/simulation-system/libs/gym-csle-apt-game/tests`
- `csle/simulation-system/libs/gym-csle-cyborg/tests`
- `csle/simulation-system/libs/csle-cluster/tests`
- `csle/simulation-system/libs/gym-csle-intrusion-response-game/tests`
- `csle/simulation-system/libs/csle-tolerance/tests`
- `csle/simulation-system/libs/csle-attack-profiler/tests`
- `csle/simulation-system/libs/csle-cli/tests`

¹²<https://docs.pytest.org/en/7.2.x/>

¹³<https://jestjs.io/>

To run the PYTHON unit tests, execute the command:

```
1 simulation-system/libs/unit_tests.sh
```

Listing 109: Command to run the PYTHON unit tests.

When adding new PYTHON unit tests, note that:

- All unit tests must be written in a `tests/` directory inside the PYTHON project.
- File names should strictly start with `"tests_"`.
- Function names should strictly start with `"test"`.

The JAVASCRIPT unit tests are available at:

```
1 csle/management-system/csle-mgmt-webapp/src/
```

Listing 110: Directory with the JAVASCRIPT unit tests.

To run the JAVASCRIPT unit tests, execute the command:

```
1 cd management-system/csle-mgmt-webapp; npm test
```

Listing 111: Command to run the JAVASCRIPT unit tests.

4.5.2 Integration Tests

CSLE includes Vagrant-based integration tests that validate the complete installation via Ansible. These tests use VirtualBox to create VMs and run the full installation process.

Prerequisites Install Vagrant and VirtualBox:

```

1 wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg
  ↪ --dearmor \
2   -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
3 echo "deb [arch=$(dpkg --print-architecture) \
4     signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
  ↪ \
5     https://apt.releases.hashicorp.com $(grep -oP \
6     '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs)
  ↪ main" \
7   | sudo tee /etc/apt/sources.list.d/hashicorp.list
8 sudo apt update && sudo apt install vagrant virtualbox
9 vagrant plugin install vagrant-disksize

```

Listing 112: Commands to install Vagrant and VirtualBox on Linux.

Running the Tests To run the integration tests with a single-node configuration:

```

1 cd vagrant && vagrant up

```

Listing 113: Command to start the Vagrant integration test.

For a two-node cluster configuration:

```

1 cd vagrant && VAGRANT_CONFIG=config/two_node.yaml vagrant up

```

Listing 114: Command to start a two-node Vagrant integration test.

After provisioning completes, run the verification tests:

```

1 vagrant ssh leader -c "cd /vagrant/tests && python3 -m pytest
  ↪ -v"

```

Listing 115: Command to run pytest verification inside the VM.

Cleanup To destroy the VMs and free disk space:


```
1 cd vagrant && vagrant destroy -f
2 vagrant box remove ubuntu/jammy64 --all # Optional: remove base
  ↪ image
```

Listing 116: Commands to clean up Vagrant VMs.

4.6 Static Code Analysis

Static code analyzers are used in CSLE to enforce style guidelines and discover bugs. The `flake8` linter (with the `flake8-rst-docstrings` extension) and the `mypy` type checker are used to analyze the PYTHON code and the `eslint` linter is used to analyze the JAVASCRIPT code.

To run the PYTHON linter on the CSLE code base, execute the command:

```
1 ./python_linter.sh
```

Listing 117: Command to run the PYTHON linter.

Alternatively, the following commands can be executed for the same effect as the command above:

```
1 flake8 simulation-system/
2 flake8 emulation-system/envs
3 flake8 examples/
```

Listing 118: Commands to run the PYTHON linter.

To run the PYTHON static type checker, execute the command:

```
1 simulation-system/libs/type_checker.sh
```

Listing 119: Command to run the PYTHON static type checker.

To run the JAVASCRIPT linter, run the commands:

```
1 cd management-system/csle-mgmt-webapp/; npm run lint
2 cd management-system/csle-mgmt-webapp/; npm run lint:fix
```

Listing 120: Commands to run the JAVASCRIPT linter.

4.7 Dependency Management

PYTHON dependencies in CSLE are managed with PYPI ¹⁴ and JAVASCRIPT dependencies are managed with npm ¹⁵.

The PYTHON dependencies are defined in the following files:

- `csle/simulation-system/libs/csle-base/requirements.txt`
- `csle/simulation-system/libs/csle-base/requirements_dev.txt`
- `csle/simulation-system/libs/csle-base/setup.cfg`
- `csle/simulation-system/libs/csle-agents/requirements.txt`
- `csle/simulation-system/libs/csle-agents/requirements_dev.txt`
- `csle/simulation-system/libs/csle-agents/setup.cfg`
- `csle/simulation-system/libs/csle-attacker/requirements.txt`
- `csle/simulation-system/libs/csle-attacker/requirements_dev.txt`
- `csle/simulation-system/libs/csle-attacker/setup.cfg`
- `csle/simulation-system/libs/csle-cli/requirements.txt`
- `csle/simulation-system/libs/csle-cli/requirements_dev.txt`
- `csle/simulation-system/libs/csle-cli/setup.cfg`
- `csle/simulation-system/libs/csle-cluster/requirements.txt`
- `csle/simulation-system/libs/csle-cluster/requirements_dev.txt`
- `csle/simulation-system/libs/csle-cluster/setup.cfg`
- `csle/simulation-system/libs/gym-csle-intrusion-response-game/requirements.txt`
- `csle/simulation-system/libs/gym-csle-intrusion-response-game/requirements_dev.txt`
- `csle/simulation-system/libs/gym-csle-intrusion-response-game/setup.cfg`
- `csle/simulation-system/libs/csle-collector/requirements.txt`
- `csle/simulation-system/libs/csle-collector/requirements_dev.txt`

¹⁴<https://pypi.org/>

¹⁵<https://www.npmjs.com/>

- csle/simulation-system/libs/csle-collector/setup.cfg
- csle/simulation-system/libs/csle-common/requirements.txt
- csle/simulation-system/libs/csle-common/requirements_dev.txt
- csle/simulation-system/libs/csle-common/setup.cfg
- csle/simulation-system/libs/csle-defender/requirements.txt
- csle/simulation-system/libs/csle-defender/requirements_dev.txt
- csle/simulation-system/libs/csle-defender/setup.cfg
- csle/simulation-system/libs/csle-rest-api/requirements.txt
- csle/simulation-system/libs/csle-rest-api/requirements_dev.txt
- csle/simulation-system/libs/csle-rest-api/setup.cfg
- csle/simulation-system/libs/csle-ryu/requirements.txt
- csle/simulation-system/libs/csle-ryu/requirements_dev.txt
- csle/simulation-system/libs/csle-ryu/setup.cfg
- csle/simulation-system/libs/csle-system-identification/requirements.txt
- csle/simulation-system/libs/csle-system-identification/requirements_dev.txt
- csle/simulation-system/libs/csle-system-identification/setup.cfg
- csle/simulation-system/libs/gym-csle-stopping-game/requirements.txt
- csle/simulation-system/libs/gym-csle-stopping-game/requirements_dev.txt
- csle/simulation-system/libs/gym-csle-stopping-game/setup.cfg
- csle/simulation-system/libs/csle-tolerance/requirements.txt
- csle/simulation-system/libs/csle-tolerance/requirements_dev.txt
- csle/simulation-system/libs/csle-tolerance/setup.cfg
- csle/simulation-system/libs/gym-csle-apt-game/requirements.txt
- csle/simulation-system/libs/gym-csle-apt-game/requirements_dev.txt
- csle/simulation-system/libs/gym-csle-apt-game/setup.cfg

- `csle/simulation-system/libs/gym-csle-cyborg/requirements.txt`
- `csle/simulation-system/libs/gym-csle-cyborg/requirements_dev.txt`
- `csle/simulation-system/libs/gym-csle-cyborg/setup.cfg`
- `csle/simulation-system/libs/csle-attack-profiler/requirements.txt`
- `csle/simulation-system/libs/csle-attack-profiler/requirements_dev.txt`
- `csle/simulation-system/libs/csle-attack-profiler/setup.cfg`

These files need to be updated whenever a PYTHON dependency is added or removed. The dependency structure among the CSLE PYTHON libraries is shown in [Fig. 31](#).

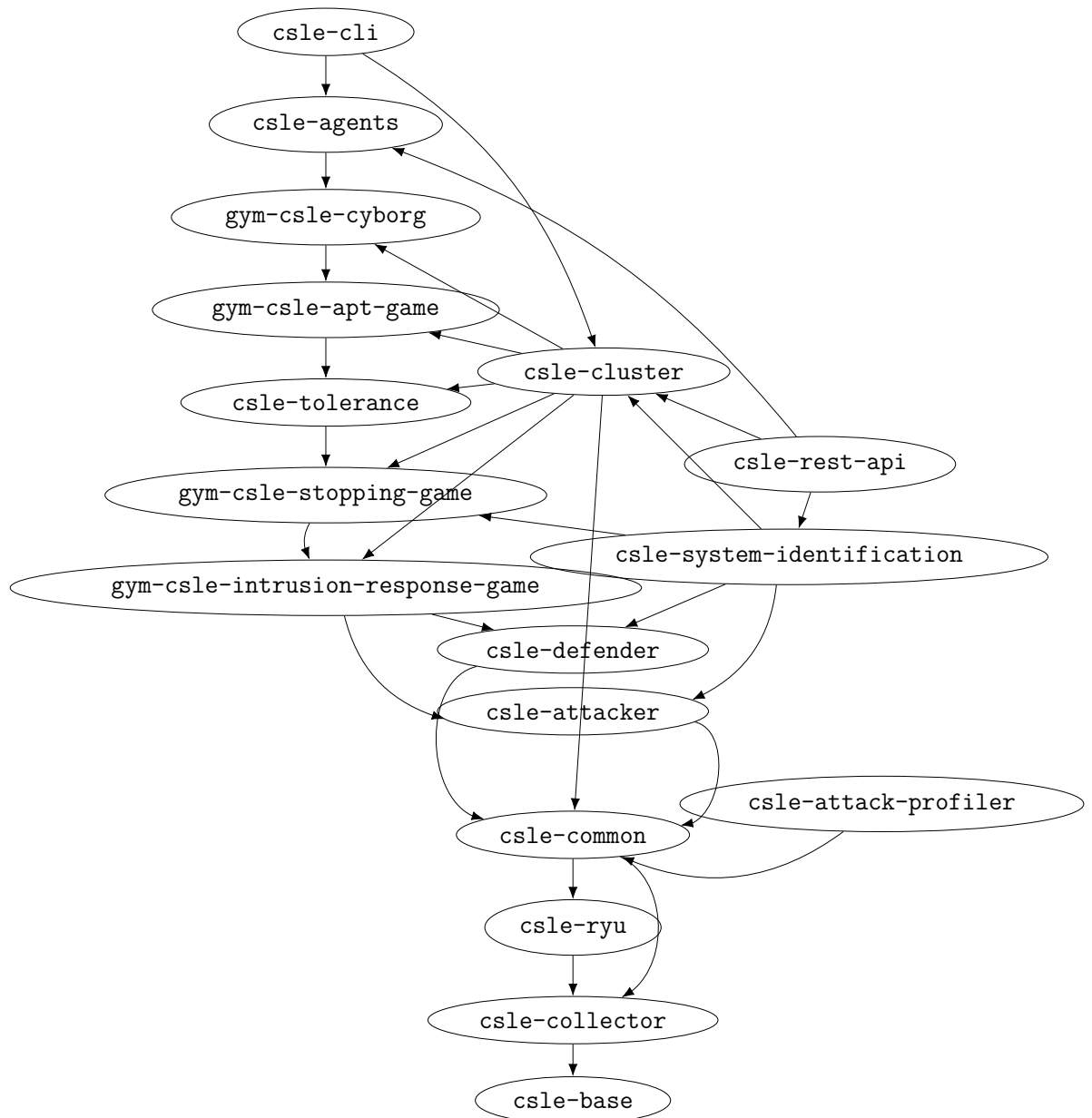


Figure 31: Dependency graph showing the dependencies among the CSLE PYTHON libraries; an arrow from X to Y indicates that X depends on Y; dependency arrows are transitive.

JAVASCRIPT dependencies are defined in the file:

`csle/management-system/csle-mgmt-webapp/package.json`

This file should be updated whenever a JAVASCRIPT dependency is added or removed.

4.8 Release Management

CSLE has semantic versioning, i.e., versions of CSLE are of the form MAJOR.MINOR.PATCH.

- The MAJOR version is incremented when incompatible API changes are made.
- The MINOR version is incremented, and new functionality is added in a backward-compatible manner.
- The PATCH version is incremented when backward-compatible bug fixes are made.

To generate a new release of CSLE, the following steps must be performed:

1. Publish new versions of the PYTHON libraries to: <https://pypi.org/>.
2. Publish new versions of the DOCKER containers to: <https://hub.docker.com/>.
3. Publish a new version of the documentation to: <http://kim-hammar.github.io/csle>.
4. Make a GITHUB release with all artifacts (<https://github.com/Kim-Hammar/csle>).

Python releases. To make a new PYTHON release, do the following:

1. Copy the `csle/simulation_system/libs/.pypirc_template` file to your home directory and rename it to `.pypirc`, e.g., `/home/kim/.pypirc`.
2. Create a PYPI token following the instructions here: <https://pypi.org/manage/account/token/>.
3. Add the PYPI token to the `.pypirc` file.
4. Edit the `RELEASE_CONFIG` variable to match the versions of the release in the file:

```
1 ./simulation-system/libs/make_release.py
```

Listing 121: File to generate PYTHON releases.

5. Run the command below to build all the PYTHON libraries and push the built artifacts to PYPI:

```
1 python ./simulation-system/libs/make_release.py
```

Listing 122: Command to generate a new PYTHON release.

Documentation releases. After making a new PYTHON release, the new API documentation can be generated by running the command:

```
1 simulation-system/libs/generate_docs.sh
```

Listing 123: Command to generate release documentation.

Before running the above command, update the release version in the following files:

```
1 csle/simulation-system/libs/csle-collector/docs/source/conf.py
2 csle/simulation-system/libs/csle-ryu/docs/source/conf.py
3 csle/simulation-system/libs/csle-common/docs/source/conf.py
4 csle/simulation-system/libs/csle-attacker/docs/source/conf.py
5 csle/simulation-system/libs/csle-defender/docs/source/conf.py
6 csle/simulation-system/libs/csle-system-identification/docs/source/conf.py
7 csle/simulation-system/libs/gym-csle-stopping-game/docs/source/conf.py
8 csle/simulation-system/libs/gym-csle-apt-game/docs/source/conf.py
9 csle/simulation-system/libs/gym-csle-cyborg/docs/source/conf.py
10 csle/simulation-system/libs/csle-agents/docs/source/conf.py
11 csle/simulation-system/libs/csle-rest-api/docs/source/conf.py
12 csle/simulation-system/libs/csle-cluster/docs/source/conf.py
13 csle/simulation-system/libs/gym-csle-intrusion-response-game/docs/source/conf.py
14 csle/simulation-system/libs/csle-tolerance/docs/source/conf.py
15 csle/simulation-system/libs/csle-attack-profiler/docs/source/conf.py
```

Listing 124: Configuration files for automatic generation of API documentation.

JavaScript releases. Update the version parameter in the following file:

```
1 csle/management-system/csle-mgmt-webapp/package.json
```

Listing 125: Package file for the management system webapp.

Docker releases. To make a new DOCKER release, do the following:

1. Edit the VERSION variable in the file:

```
1 ./emulation-system/base_images/Makefile
```

Listing 126: MAKEFILE for base DOCKER images in CSLE.

2. Edit the VERSION variable in the file:

```
1 ./emulation-system/derived_images/Makefile
```

Listing 127: MAKEFILE for derived DOCKER images in CSLE.

3. Edit the versions of the derived images in the DOCKERFILES inside the following directory:

```
1 ./emulation-system/derived_images/
```

Listing 128: Directory with derived DOCKER images in CSLE.

4. Edit the versions of the base images in the DOCKERFILES inside the following directory:

```
1 ./emulation-system/base_images/
```

Listing 129: Directory with base DOCKER images in CSLE.

5. Edit the version parameters in all emulation configurations in the following directory:

```
1 ./emulation-system/envs
```

Listing 130: Directory with emulation configurations.

6. Make a new release of <https://github.com/Kim-Hammar/exploit-cve-2017-7494>:

- First clone the repository:

```
1 git clone https://github.com/Kim-Hammar/exploit-CVE-2017-7494
2 cd exploit-CVE-2017-7494
```

Listing 131: Command to clone <https://github.com/Kim-Hammar/exploit-cve-2017-7494>.

- Then update the version in the file `exploit-cve-2017-7494/MAKEFILE`.
- Then build and push the image by running the commands:

```
1 make build
2 make push
```

Listing 132: Commands to build and push the `exploit-cve-2017-7494` image.

7. Build the images by running the command:

```
1 cd emulation-system/; make build
```

Listing 133: Command to build the DOCKER images in CSLE.

8. Push the images to DOCKERHUB by running the command:

```
1 cd emulation-system/; make push
```

Listing 134: Command to push DOCKER images to DOCKERHUB.

9. Insert the new emulation configuration into the metastore by running the command:

```
1 cd emulation-system/; make emulations
```

Listing 135: Command to insert emulation configurations into the metastore.

GitHub releases. To make a release in GITHUB, follow the official guide¹⁶.

5 How-to Guides and Tutorials

This section contains guides and tutorials on how to use CSLE. Users of CSLE can request additional tutorials to be added to this section by creating GITHUB Issues.

5.1 How to: Add Base Containers

To add a base container with the name “my_container”, do the following steps:

1. Create a sub-directory with the name my_container in the following directory:

```
1 csle/emulation-system/base_images/docker_files
```

Listing 136: Directory with base DOCKER images.

2. Add a DOCKERFILE to the following directory:

```
1 csle/emulation-system/base_images/docker_files/my_container
```

Listing 137: Directory with the DOCKERFILE of a container with the name my_container.

3. Add build and push instructions to the following MAKEFILE:

```
1 csle/emulation-system/base_images/Makefile
```

Listing 138: MAKEFILE for base DOCKER images.

4. Update the following README file:

¹⁶<https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository>

```
1 csle/emulation-system/base_images/README.md
```

Listing 139: README file for base DOCKER images.

5. Build and push the image to DOCKERHUB by running the commands:

```
1 cd csle/emulation-system/base_images; make my_container
2 cd csle/emulation-system/base_images; make push_my_container
```

Listing 140: Commands to build and push the base DOCKER image `my_container` to DOCKERHUB.

5.2 How to: Add Derived Containers

To add a derived container with the name “`my_container`”, perform the following steps:

1. Create a sub-directory with the name `my_container` in the directory:

```
1 csle/emulation-system/derived_images/
```

Listing 141: Directory with derived DOCKER images.

2. Add a DOCKERFILE inside the directory:

```
1 csle/emulation-system/derived_images/my_container/
```

Listing 142: Directory with the DOCKERFILE for a derived DOCKER image with the name `my_container`.

3. Add build and push instructions to the following MAKEFILE:

```
1 csle/emulation-system/derived_images/Makefile
```

Listing 143: MAKEFILE for derived DOCKER images.

4. Update the following README file:

```
1 csle/emulation-system/derived_images/README.md
```

Listing 144: README file for derived DOCKER images.

5. Build and push the image to DOCKERHUB by running the commands:

```
1 cd csle/emulation-system/derived_images; make my_container
2 cd csle/emulation-system/derived_images; make push_my_container
```

Listing 145: Commands to build and push the derived DOCKER image my_container to DOCKERHUB.

5.3 How to: Add Emulation Configurations

To add a new emulation configuration with the name “level_13” and version 0.10.0. perform the following steps:

1. Create a sub-directory called level_13 in the folder:

```
1 csle/emulation-system/envs/0.10.0/
```

Listing 146: Directory with emulation configurations with version 0.10.0.

2. Add the emulation configuration file config.py to the directory:

```
1 csle/emulation-system/envs/0.10.0/level_13/
```

Listing 147: Directory with emulation configuration file for the emulation with the name level_13 and version 0.10.0.

3. Update the following README file:

```
1 csle/emulation-system/envs/0.10.0/README.md
```

Listing 148: README file for emulation configurations with version 0.10.0.

4. Update the following MAKEFILE with installation instructions:

```
1 csle/emulation-system/envs/0.10.0/README.md
```

Listing 149: MAKEFILE for emulation configurations with version 0.10.0

5. Insert the emulation configuration into the metastore by running the command:

```
1 csle/emulation-system/envs; make install
```

Listing 150: Command to insert emulation configurations into the metastore.

5.4 How to: Add Simulation Configurations

To add a new simulation environment with the name “test_env” and version 0.0.1, perform the following steps:

1. Create a sub-directory called test_env in the folder:

```
1 csle/simulation-system/envs/
```

Listing 151: Directory with simulation configurations.

2. Add the simulation configuration file config_v_001.py to the directory:

```
1 csle/simulation-system/envs/test_env
```

Listing 152: Directory with configuration file for the simulation environment with the name test_env.

3. Update the following README file:

```
1 csle/simulation-system/envs/README.md
```

Listing 153: README file for simulation configurations.

4. Update the following MAKEFILE with installation instructions:

```
1 csle/simulation-system/envs/Makefile
```

Listing 154: MAKEFILE for simulation configurations.

5. Insert the simulation configuration into the metastore by running the command:

```
1 csle/simulation-system/envs; make install
```

Listing 155: Command to insert simulation configurations into the metastore.

5.5 How to: Update Monitoring Agents

To update the monitoring agents that run on the emulated devices, perform the following steps:

1. Do the necessary code changes to the monitoring agent by editing the files in the directory:

```
1 csle/simulation-system/libs/csle-collector/src/
```

Listing 156: Directory with source code of the monitoring agents in CSLE.

2. Update the GRPC API by editing the protocol-buffer¹⁷ files in the directory:

```
1 csle/simulation-system/libs/csle-collector/protos/
```

Listing 157: Directory with protocol buffers for the monitoring agents in CSLE.

3. Regenerate the GRPC PYTHON files by running the commands:

¹⁷<https://developers.google.com/protocol-buffers>

```

1 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/client_manager/.

2 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/kafka_manager/.

3 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/elk_manager/.

4 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/docker_stats_manager/.

5 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/snort_ids_manager/.

6 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/host_manager/.

7 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/ossec_ids_manager/.

8 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/traffic_manager/.

9 python -m grpc_tools.protoc -I./protos/
  ↳ --python_out=./src/csle_collector/.
  ↳ --grpc_python_out=./src/csle_collector/ryu_manager/.
  ↳ ./protos/ryu_manager.proto

```

Listing 158: Commands to generate GRPC PYTHON files from protobuf files.

5.6 How to: Add Numerical Algorithms

To add a new numerical algorithm called “my_algorithm” to the simulation system, perform the following steps:

1. Create a new sub-directory called my_algorithm inside the directory:

```
1 csle/simulation-system/libs/csle-agents/src/  
2     csle_agents/agents/
```

Listing 159: Directory with numerical algorithms in CSLE.

2. Implement the algorithm in a file called `my_algorithm.py` in the directory:

```
1 csle/simulation-system/libs/csle-agents/src/  
2     csle_agents/agents/my_agent
```

Listing 160: Directory with algorithm implementation for the algorithm with the name `my_agent` in CSLE.

The implementation should be a PYTHON class called “MyAlgorithm” that inherits from the class “BaseAgent”, which is defined in the file:

```
1 csle/simulation-system/libs/csle-agents/src/csle_agents/  
2     agents/base/base_agent.py
```

Listing 161: Base agent file in CSLE.

3. Add an example of how to use the algorithm to the following folder:

```
1 csle/examples
```

Listing 162: Directory with example usages of CSLE.

5.7 How to: Add Configuration Parameters

To add a new configuration parameter, perform the following steps:

1. Add the configuration parameter to the file: `csle/config.json`.
2. Add the new configuration parameter to the file:

```
1 csle/simulation-system/libs/csle-common/src/  
2   csle_common/dao/emulation_config/config.py
```

Listing 163: PYTHON file that manages the CSLE configuration.

3. Add the new parameter to an appropriate class in the following file (add a new class if needed):

```
1 csle/simulation-system/libs/csle-common/src  
2   /csle_common/constants/constants.py
```

Listing 164: PYTHON file with constants in `csle-common`.

6 Frequently Asked Questions

1. Q – How to add a new management user?

A – New management users can be added by administrators through the web interface.

2. Q – Why does my machine crash when I try to run the emulation environment `csle-level9-0.10.0`?

A – Most likely because the machine is running out of memory. The emulation environment `csle-level9-080` includes 34 containers, each of which requires at least 1GB of RAM, meaning that the machine needs at least 34GB of RAM to run this emulation environment.

3. Q – Why does my reinforcement learning algorithm not converge?

A – There are many possible reasons for this; it could be a hyperparameter problem, a bug in your training environment, or simply slow convergence. These issues are not related to CSLE's implementation.

4. Q – How can I use CSLE with my custom code?

A – You have two options. One option is integrating your code with CSLE as a new simulation or emulation environment (see §5 for instructions). Another option is that you write custom code to integrate CSLE with your existing code base. The latter option is not something that CSLE maintainers will provide support for.

5. Q – How do I generate plots?

A – CSLE does not have a plotting library. Plots can be created through standard libraries, e.g., matplotlib [36].

6. Q – How do I install CSLE?

A – See the instructions in §3.

7. Q – Can I install CSLE on Windows?

A – No.

8. Q – Can I install CSLE on Mac?

A – Yes, you should be able to take the installation instructions for Linux and adapt them to a Mac environment. Note that Mac is not a platform actively supported by CSLE. Hence, you can not expect help from maintainers if you run into Mac-specific issues.

9. Q – How do I use the Docker images in CSLE without the rest of the framework?

A – The CSLE DOCKER images can be pulled as standalone images from DOCKERHUB ¹⁸.

10. Q – How do I cite CSLE?

A – A list of publications is available in §1.4.3.

11. Q – Can I use CSLE for commercial purposes?

A – Yes if it is compatible with the license (Creative Commons Attribution-ShareAlike 4.0 International License).

12. Q – Can I use CSLE for research purposes?

A – Yes, CSLE was built for research purposes. If you use CSLE in your research, please cite us.

13. Q – Can I use CSLE in an air-gapped environment?

A – An air-gapped installation is possible but is not something that is officially supported in CSLE.

14. Q – Can I run CSLE in the cloud?

A – Yes, you can deploy CSLE on virtual machines provided by any cloud vendor.

15. Q – Can I take the CSLE code as a base to develop a new framework?

¹⁸<https://hub.docker.com/r/kimham/>

A — Yes, as long as you respect the license (Creative Commons Attribution-ShareAlike 4.0 International License).

16. Q — Can I deploy CSLE on a Kubernetes cluster?

A — It is possible, but it is not something that we have documentation for.

17. Q — How can I contribute to CSLE?

A — See the developer guide in §4.

18. Q — How can I contact the maintainers of CSLE?

A — See §1.5.1.

19. Q — When I try to start an emulation involving OVS switches, the start command gets stuck. How to fix it?

A — This is likely because you have updated your operating system's kernel, which caused the openvswitch module to be removed. Run the command `lsmod | grep openvswitch` to verify that the module exists in the kernel. If it does not exist, run the command `sudo apt install openvswitch-switch openvswitch-common`.

A Setup PyCharm for Remote Development

The best way to debug and implement new features to the emulation system involves three steps: (1) deploy the latest version of the emulation system on your target cluster; (2) clone the latest source code both on the server and on your local machine (e.g., your laptop); (3) connect your Integrated Development Environment (IDE) to the cluster so that you can edit code directly on the cluster. This appendix walks through the steps of setting up the PYCHARM IDE for remote development.

Step 1: Deploy CSLE on the servers. Follow the instructions in §3.

Step 2: Clone the source code. Run the following command on your local machine and on one of the servers where CSLE is installed:

```
1 git clone https://github.com/Kim-Hammar/csle
```

Listing 165: Command for cloning the source code of CSLE.

Step 3: Setup PyCharm for remote development. Start PYCHARM on your local machine and open the CSLE project. Next, go to "Preferences" in PYCHARM (see Fig. 32).

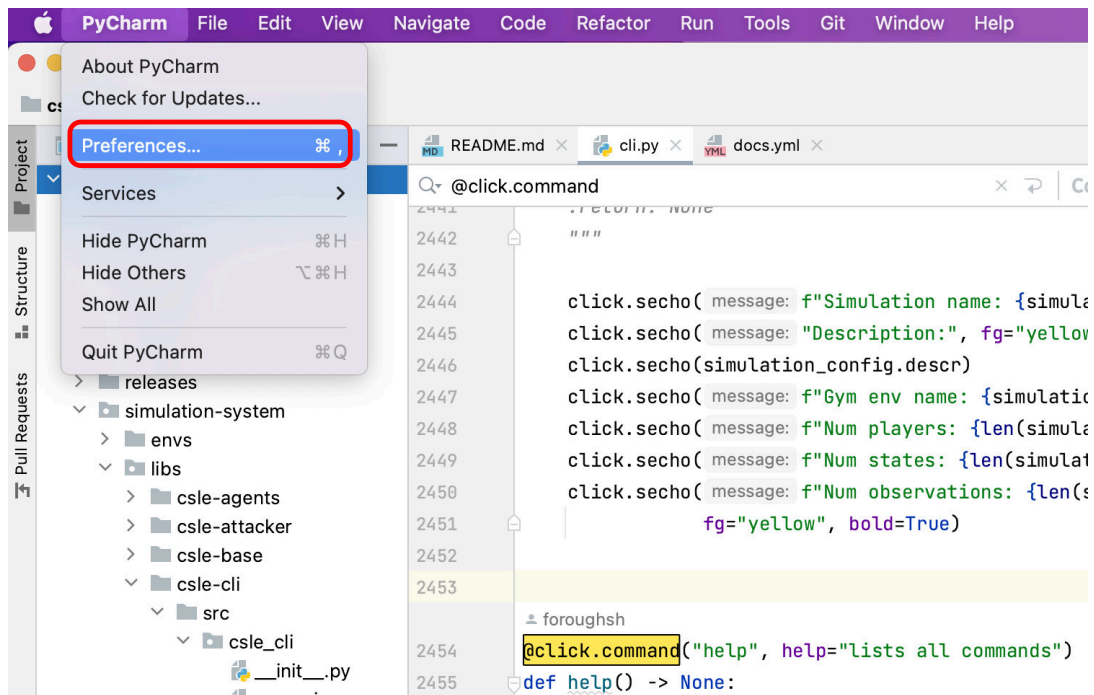


Figure 32: The preferences tab in PYCHARM.

Then, configure an SSH interpreter for the project by selecting the PYTHON interpreter of the server where CSLE is installed (see Figs. 33-36).

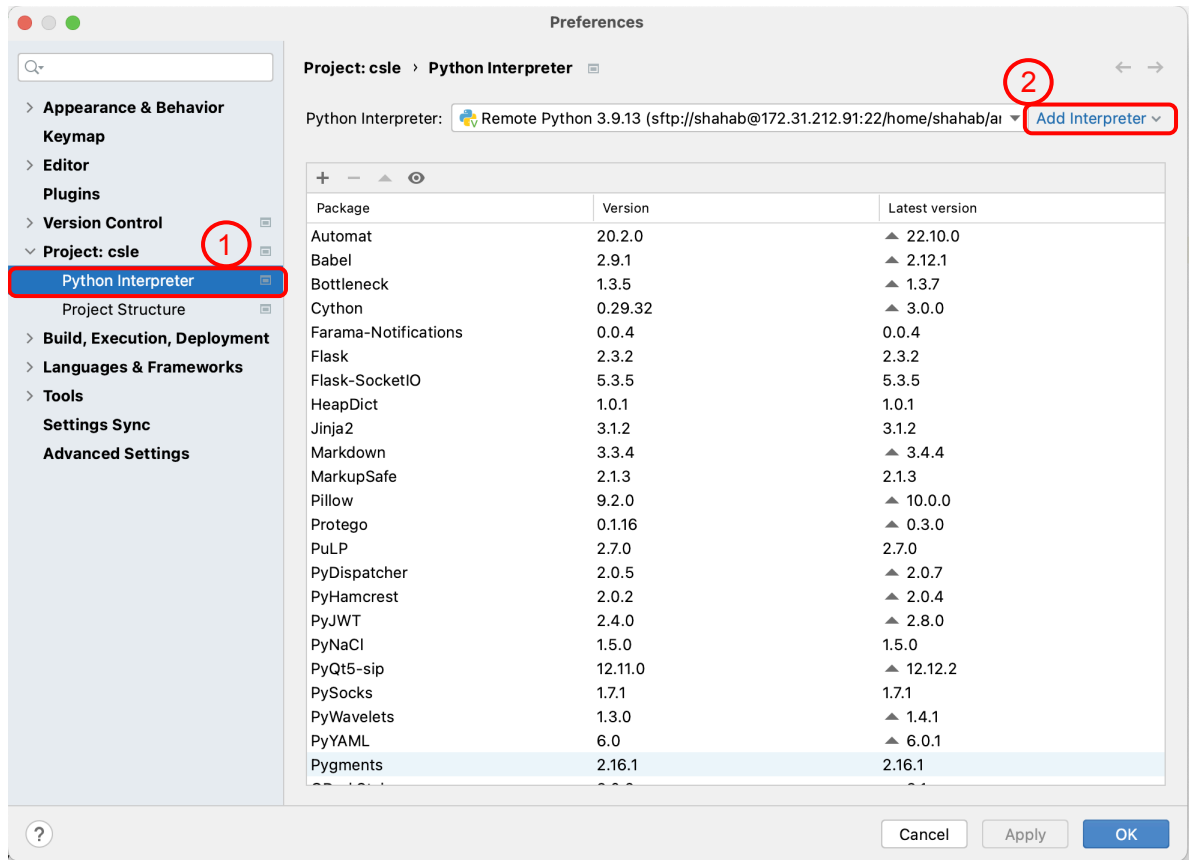


Figure 33: Configuration of a remote SSH PYTHON interpreter in PYCHARM (1/4).

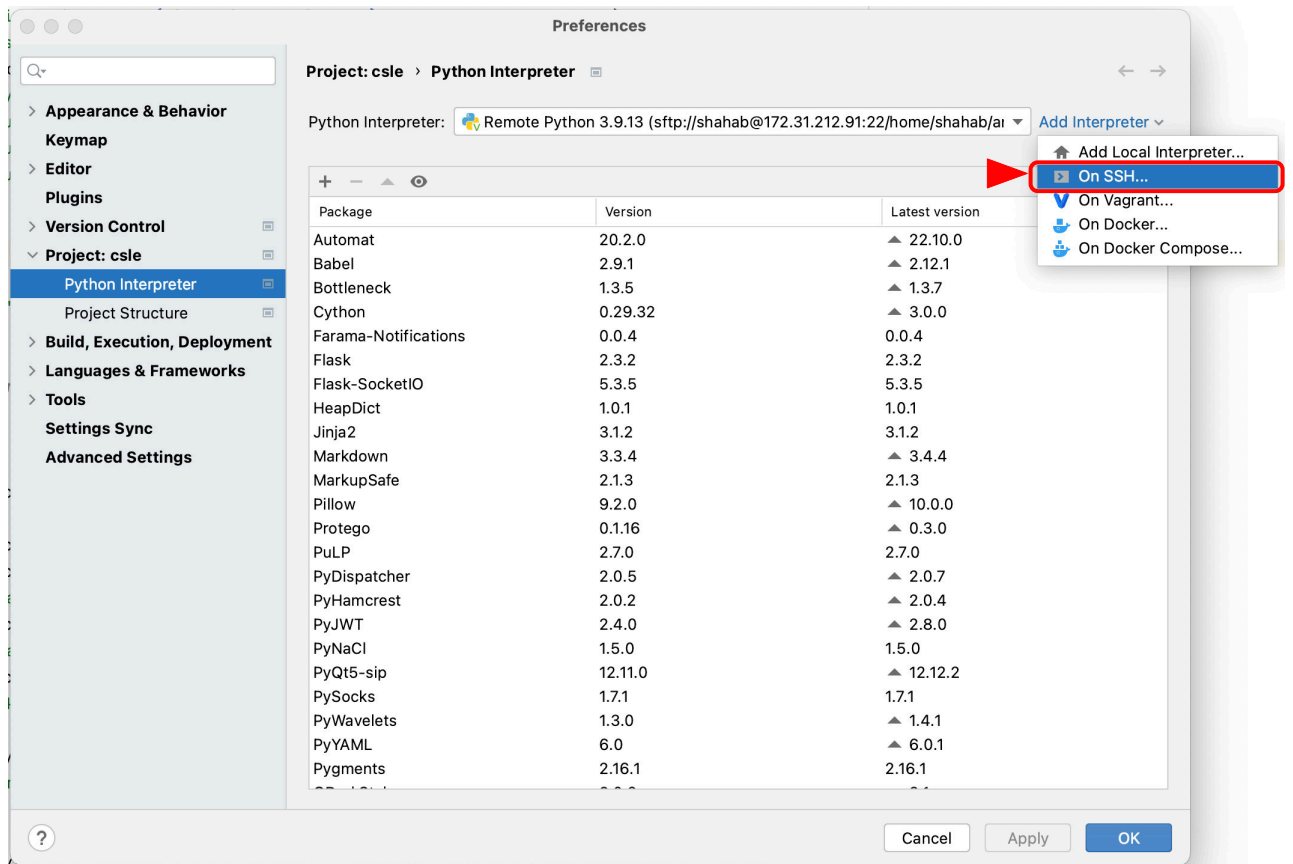


Figure 34: Configuration of a remote SSH PYTHON interpreter in PYCHARM (2/4).

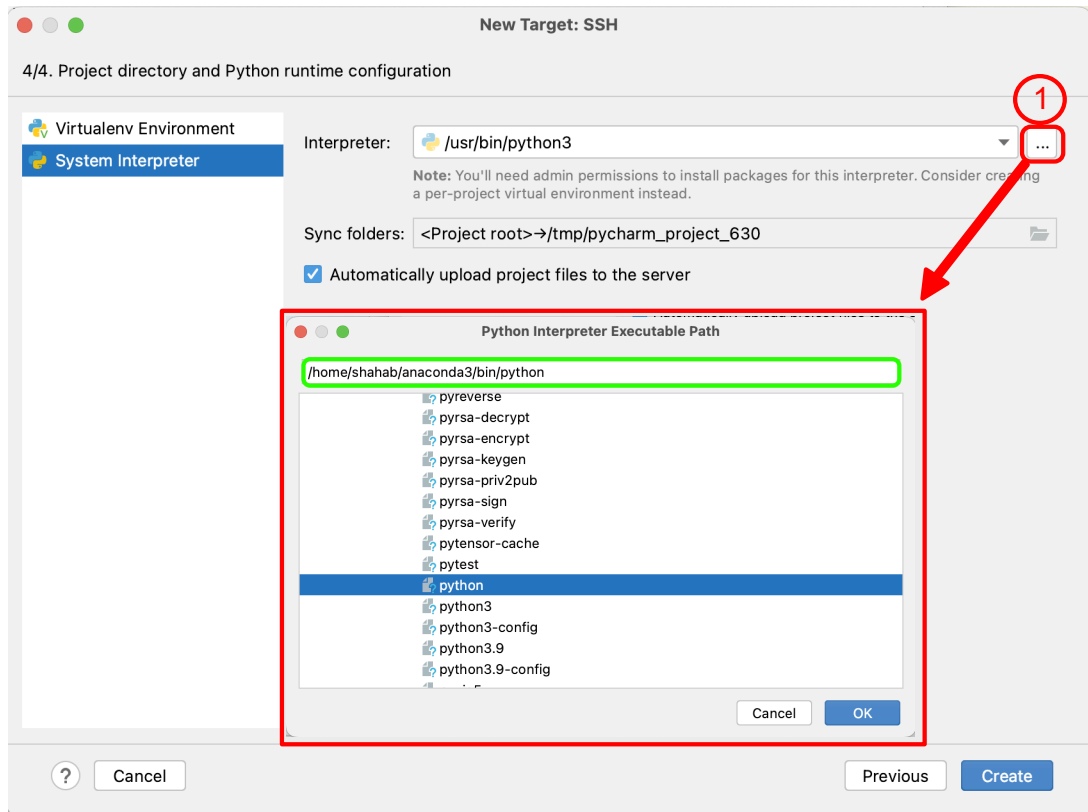


Figure 35: Configuration of a remote SSH PYTHON interpreter in PYCHARM (3/4).

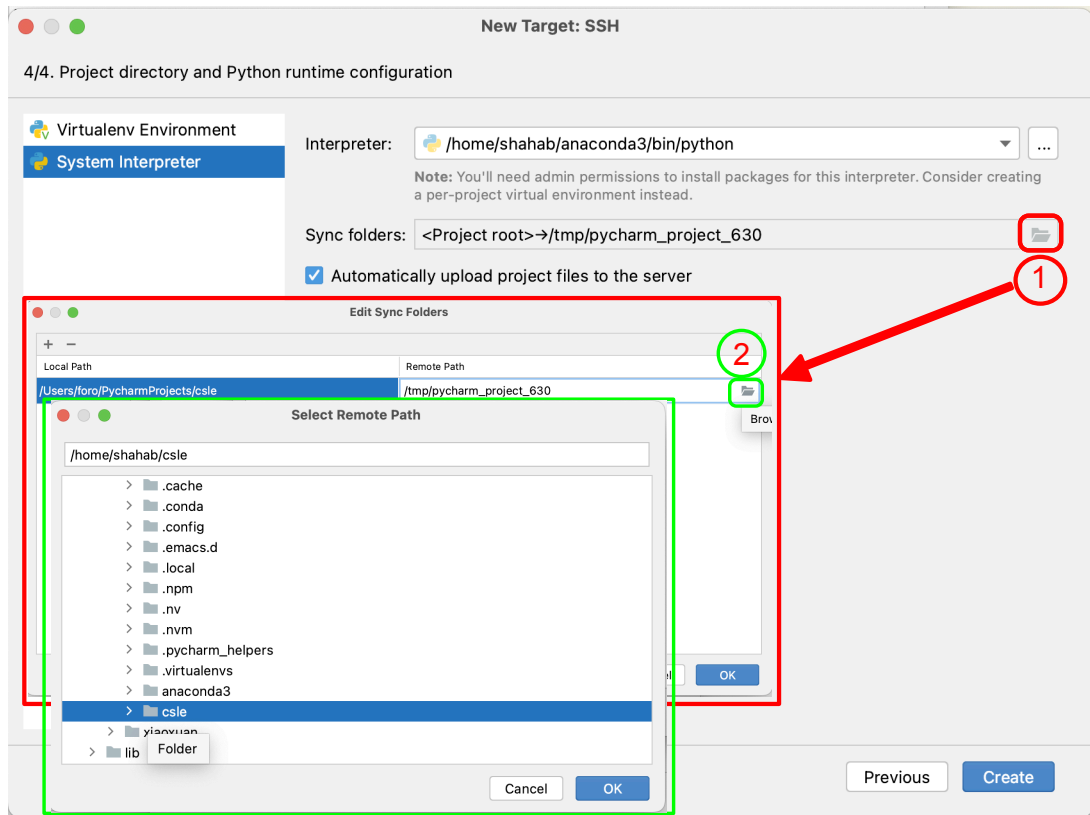


Figure 36: Configuration of a remote SSH PYTHON interpreter in PYCHARM (4/4).

To test that the remote ssh interpreter is working correctly and that the files are synchronized between the remote server and your local machine you can try to add a line of code in the IDE and verify that it also shows up on the server. You can also test the setup by running an example script in the `examples/` folder.

B Useful Git Commands

This appendix describes the minimal git commands necessary to follow the CSLE contribution workflow (see §4).

When you start working on a new feature, first make sure that you have pulled the latest code changes by running the following commands in the root of the CSLE repository:

```
1 git checkout master
2 git pull origin master
```

Listing 166: Commands for pulling the latest code changes from the master branch.

If the above command did not succeed due to code conflicts, you can either resolve the conflicts or choose to reset the code to the latest commit (this will override any local changes) by running the commands:

```
1 git reset --hard origin/master
2 git clean -fd
```

Listing 167: Commands for resetting the local version of the code to the upstream version. (Warning: this will override any local changes to the code.)

Next, create a new branch for your code changes by running the command:

```
1 git checkout -b mybranchname
```

Listing 168: Command for creating a new branch with name "mybranchname".

Then, you can make your code changes. When you have made a set of changes, you can commit them by running the commands:

```
1 git add --all
2 git commit -m "comment for the commit"
```

Listing 169: Commands committing new code changes.

You can make several commits. When you feel that your new feature is ready to be merged into the master branch you can push the code to GITHUB by running the command:

```
1 git push origin mybranchname
```

Listing 170: Command for pushing the branch "mybranchname" to GITHUB.

After pushing the changes, you can go with your web browser to:

`https://github.com/Kim-Hammar/csle`

and open a new pull request.

When the pull request is merged into the master branch, you can delete the local branch and pull the new changes from the master branch by running the commands:

```
1 git checkout master
2 git pull origin master
3 git branch -d mybranchname
```

Listing 171: Commands for (1) switching to the master branch; (2) pulling the latest changes; and (3) deleting the old feature branch with the name “mybranchname”.

After running the above commands, you can verify that your commits are available in the master branch by running the command:

```
1 git log
```

Listing 172: Command for viewing git commits.

References

- [1] Alex Andrew et al. “Developing Optimal Causal Cyber-Defence Agents via Cyber Security Simulation”. In: *Proceedings of the ML4Cyber workshop, ICML 2022, Baltimore, USA, July 17-23, 2022*. PMLR, 2022.
- [2] A.Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. URL: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [3] William Blum. *Gamifying machine learning for stronger security and AI models*. Microsoft Research. 2021.
- [4] A. Clemm and Inc Cisco Systems. *Network Management Fundamentals*. Cisco Press fundamentals series. Cisco Press, 2007. ISBN: 9781587201370. URL: <https://books.google.se/books?id=NvEVQmjScjYC>.

- [5] Umur Cubukcu et al. "Citrus: Distributed PostgreSQL for Data-Intensive Applications". In: *Proceedings of the 2021 International Conference on Management of Data*. SIGMOD '21. Virtual Event, China: Association for Computing Machinery, 2021, 2490–2502. ISBN: 9781450383431. DOI: [10.1145/3448016.3457551](https://doi.org/10.1145/3448016.3457551). URL: <https://doi.org/10.1145/3448016.3457551>.
- [6] Steven Diamond and Stephen Boyd. "CVXPY: A Python-embedded modeling language for convex optimization". In: *Journal of Machine Learning Research* 17.83 (2016), pp. 1–5.
- [7] Brendan Eich. "JavaScript at ten years". In: *Proceedings of the 10th ACM SIGPLAN International Conference on Functional Programming, ICFP 2005, Tallinn, Estonia, September 26-28, 2005*. Ed. by Olivier Danvy and Benjamin C. Pierce. ACM, 2005, p. 129. DOI: [10.1145/1086365.1086382](https://doi.org/10.1145/1086365.1086382). URL: <https://doi.org/10.1145/1086365.1086382>.
- [8] Facebook. *React*. <https://github.com/facebook/react>. 2021. URL: <https://github.com/facebook/react>.
- [9] Google. *cAdvisor*. <https://github.com/google/cadvisor>. 2022. URL: <https://github.com/google/cadvisor>.
- [10] Google. *Google Remote Procedure Call*. 2022.
- [11] Clinton Gormley and Zachary Tong. *Elasticsearch: The Definitive Guide*. 1st. O'Reilly Media, Inc., 2015. ISBN: 1449358543.
- [12] Grafana Labs. *Grafana Documentation*. 2018. URL: <https://grafana.com/docs/> (visited on 07/25/2019).
- [13] Kim Hammar. *Awesome Reinforcement Learning for Cyber Security*. <https://github.com/Limmen/awesome-rl-for-cybersecurity>. 2021. URL: <https://github.com/Limmen/awesome-rl-for-cybersecurity>.
- [14] Kim Hammar. "Optimal Security Response to Network Intrusions in IT Systems". Academic Dissertation which, with due permission of the KTH Royal Institute of Technology, is submitted for public defence for the Degree of Doctor of Philosophy on Thursday the 5th December 2024, at 14:00 in F3, Lindstedtsvägen 26, Stockholm. The defense will be streamed via Zoom: <https://kth-se.zoom.us/j/64592772191> Candidate: Kim Hammar Supervisor: Professor Rolf Stadler, KTH, Sweden Opponent: Professor Tansu Alpcan, The University of Melbourne, Australia Grading committee: Professor Emil Lupu, Imperial College London, UK; Professor Alina Oprea, Northeastern University, USA; Professor Karl H. Johansson, KTH, Sweden; Reviewer: Professor Henrik Sandberg, KTH, Sweden QC 20241111. PhD thesis. KTH, Network and Systems Engineering, 2024, p. 338. ISBN: 978-91-8106-093-5.

- [15] Kim Hammar, Tansu Alpcan, and Emil C. Lupu. *Incident Response Planning Using a Lightweight Large Language Model with Reduced Hallucination*. 2025. arXiv: 2508.05188 [cs.CR]. URL: <https://arxiv.org/abs/2508.05188>.
- [16] Kim Hammar, Neil Dhir, and Rolf Stadler. *Optimal Defender Strategies for CAGE-2 using Causal Modeling and Tree Search*. 2024. arXiv: 2407.11070 [cs.LG]. URL: <https://arxiv.org/abs/2407.11070>.
- [17] Kim Hammar and Tao Li. "Online Incident Response Planning under Model Misspecification through Bayesian Learning and Belief Quantization". In: *Proceedings of the 18th ACM Workshop on Artificial Intelligence and Security*. AISEC'25. New York, NY, USA: Association for Computing Machinery, 2026, 40–51. ISBN: 9798400718953. DOI: 10.1145/3733799.3762965. URL: <https://doi.org/10.1145/3733799.3762965>.
- [18] Kim Hammar and Rolf Stadler. "A System for Interactive Examination of Learned Security Policies". In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. 2022, pp. 1–3. DOI: 10.1109/NOMS54207.2022.9789707.
- [19] Kim Hammar and Rolf Stadler. "An Online Framework for Adapting Security Policies in Dynamic IT Environments". In: *2022 18th International Conference on Network and Service Management (CNSM)*. 2022, pp. 359–363. DOI: 10.23919/CNSM55787.2022.9964838.
- [20] Kim Hammar and Rolf Stadler. "Digital Twins for Security Automation". In: *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. 2023, pp. 1–6. DOI: 10.1109/NOMS56928.2023.10154288.
- [21] Kim Hammar and Rolf Stadler. "Finding Effective Security Strategies through Reinforcement Learning and Self-Play". In: *International Conference on Network and Service Management (CNSM 2020)*. Izmir, Turkey, 2020.
- [22] Kim Hammar and Rolf Stadler. "Intrusion Prevention Through Optimal Stopping". In: *IEEE Transactions on Network and Service Management* 19.3 (2022), pp. 2333–2348. DOI: 10.1109/TNSM.2022.3176781.
- [23] Kim Hammar and Rolf Stadler. "Intrusion Tolerance as a Two-Level Game". In: *Decision and Game Theory for Security*. Ed. by Arunesh Sinha et al. Cham: Springer Nature Switzerland, 2025, pp. 3–23. ISBN: 978-3-031-74835-6.
- [24] Kim Hammar and Rolf Stadler. "Intrusion Tolerance for Networked Systems through Two-Level Feedback Control". In: *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2024, pp. 338–352. DOI: 10.1109/DSN58291.2024.00042.

- [25] Kim Hammar and Rolf Stadler. “Learning Intrusion Prevention Policies through Optimal Stopping”. In: *International Conference on Network and Service Management (CNSM 2021)*. <http://dl.ifip.org/db/conf/cnsm/cnsm2021/1570732932.pdf>. Izmir, Turkey, 2021.
- [26] Kim Hammar and Rolf Stadler. “Learning Near-Optimal Intrusion Responses Against Dynamic Attackers”. In: *IEEE Transactions on Network and Service Management* (2023), pp. 1–1. DOI: [10.1109/TNSM.2023.3293413](https://doi.org/10.1109/TNSM.2023.3293413).
- [27] Kim Hammar and Rolf Stadler. “Learning Security Strategies through Game Play and Optimal Stopping”. In: *Proceedings of the ML4Cyber workshop, ICML 2022, Baltimore, USA, July 17-23, 2022*. PMLR, 2022.
- [28] Kim Hammar and Rolf Stadler. *Online Identification of IT Systems through Active Causal Learning*. 2025. arXiv: [2509.02130](https://arxiv.org/abs/2509.02130) [cs.LG]. URL: <https://arxiv.org/abs/2509.02130>.
- [29] Kim Hammar and Rolf Stadler. *Scalable Learning of Intrusion Responses through Recursive Decomposition*. 2023. arXiv: [2309.03292](https://arxiv.org/abs/2309.03292) [eess.SY]. URL: <https://arxiv.org/abs/2309.03292>.
- [30] Kim Hammar et al. *Adaptive Network Security Policies via Belief Aggregation and Rollout*. 2025. arXiv: [2507.15163](https://arxiv.org/abs/2507.15163) [eess.SY]. URL: <https://arxiv.org/abs/2507.15163>.
- [31] Kim Hammar et al. “Adaptive Security Response Strategies Through Conjectural Online Learning”. In: *IEEE Transactions on Information Forensics and Security* 20 (2025), pp. 4055–4070. DOI: [10.1109/TIFS.2025.3558600](https://doi.org/10.1109/TIFS.2025.3558600).
- [32] Nikolaus Hansen, Youhei Akimoto, and Petr Baudis. *CMA-ES/pycma on Github*. Zenodo, DOI:10.5281/zenodo.2559634. Feb. 2019. DOI: [10.5281/zenodo.2559634](https://doi.org/10.5281/zenodo.2559634). URL: <https://doi.org/10.5281/zenodo.2559634>.
- [33] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [34] Red Hat. *Awesome Reinforcement Learning for Cyber Security*. <https://www.ansible.com/>. 2024. URL: <https://www.ansible.com/>.
- [35] Stephen Hemminger. “Network emulation with NetEm”. In: *Linux Conf* (2005).
- [36] J. D. Hunter. “Matplotlib: A 2D graphics environment”. In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).

- [37] Anaconda Inc. *Anaconda*. <https://docs.conda.io/>. 2022. URL: <https://docs.conda.io/>.
- [38] Jaromír Janisch, Tomáš Pevný, and Viliam Lisý. *NASimEmu: Network Attack Simulator & Emulator for Training Agents Generalizing to Novel Scenarios*. 2023. arXiv: 2305.17246 [cs.CR].
- [39] Jens Kober, J. Andrew Bagnell, and Jan Peters. "Reinforcement Learning in Robotics: A Survey". In: *International Journal of Robotics Research* 32.11 (2013), pp. 1238–1274. DOI: 10.1177/0278364913495721.
- [40] Jay Kreps. "Kafka : a Distributed Messaging System for Log Processing". In: 2011.
- [41] Marc Lanctot et al. *OpenSpiel: A Framework for Reinforcement Learning in Games*. 2019. DOI: 10.48550/ARXIV.1908.09453. URL: <https://arxiv.org/abs/1908.09453>.
- [42] Nevena Lazic et al. "Data center cooling using model-predictive control". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: <https://proceedings.neurips.cc/paper/2018/file/059fdcd96baeb75112f09fa1dcc740cc-Paper.pdf>.
- [43] Li Li, Raed Fayad, and Adrian Taylor. *CyGIL: A Cyber Gym for Training Autonomous Agents over Emulated Network Systems*. 2021. arXiv: 2109.03331 [cs.CR].
- [44] Tao Li et al. "Conjectural Online Learning with First-order Beliefs in Asymmetric Information Stochastic Games". In: *2024 63rd IEEE Conference on Decision and Control (CDC)*. preprint <https://arxiv.org/pdf/2402.18781.pdf>. 2024.
- [45] Nick McKeown et al. "OpenFlow: Enabling Innovation in Campus Networks". In: *SIGCOMM Comput. Commun. Rev.* 38.2 (2008), 69–74. ISSN: 0146-4833. DOI: 10.1145/1355734.1355746. URL: <https://doi.org/10.1145/1355734.1355746>.
- [46] Dirk Merkel. "Docker: lightweight linux containers for consistent development and deployment". In: *Linux journal* 2014 (2014), p. 2.
- [47] Andres Molina-Markham et al. "Network Environment Design for Autonomous Cyberdefense". In: (2021). <https://arxiv.org/abs/2103.07583>.
- [48] Salman Niazi et al. "Leader Election Using NewSQL Database Systems". In: *Distributed Applications and Interoperable Systems*. Ed. by Alysson Bessani and Sara Bouchenak. Cham: Springer International Publishing, 2015, pp. 158–172. ISBN: 978-3-319-19129-4.

- [49] Sean Oesch et al. "Towards a High Fidelity Training Environment for Autonomous Cyber Defense Agents". In: *Proceedings of the 17th Cyber Security Experimentation and Test Workshop*. CSET '24. Philadelphia, PA, USA: Association for Computing Machinery, 2024, 91–99. ISBN: 9798400709579. DOI: [10.1145/3675741.3675752](https://doi.org/10.1145/3675741.3675752). URL: <https://doi.org/10.1145/3675741.3675752>.
- [50] Andrei Paleyes et al. *Emulation of physical processes with Emukit*. 2021. arXiv: [2110.13293](https://arxiv.org/abs/2110.13293) [cs.LG].
- [51] Pallets. *Flask*. <https://github.com/pallets/flask>. 2021. URL: <https://github.com/pallets/flask>.
- [52] Ben Pfaff et al. "The Design and Implementation of Open vSwitch". In: *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. Oakland, CA: USENIX Association, May 2015, pp. 117–130. ISBN: 978-1-931971-218. URL: <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pfaff>.
- [53] Prometheus. *Node exporter*. https://github.com/prometheus/node_exporter. 2022. URL: https://github.com/prometheus/node_exporter.
- [54] Bjorn Rabenstein and Julius Volz. "Prometheus: A Next-Generation Monitoring System (Talk)". In: Dublin: USENIX Association, May 2015.
- [55] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html>.
- [56] Will Reese. "Nginx: The High-Performance Web Server and Reverse Proxy". In: *Linux J*. 2008.173 (2008). ISSN: 1075-3583.
- [57] Guido van Rossum, Barry Warsaw, and Nick Coghlan. *PEP 8 - Style Guide for Python Code*. <https://peps.python.org/pep-0008/>. 2001. URL: <https://peps.python.org/pep-0008/>.
- [58] Raghav Sethi et al. "Presto: SQL on Everything". In: *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. 2019, pp. 1802–1813. DOI: [10.1109/ICDE.2019.00196](https://doi.org/10.1109/ICDE.2019.00196).
- [59] Maxwell Standen et al. "CybORG: A Gym for the Development of Autonomous Cyber Agents". In: *CoRR* (). <https://arxiv.org/abs/2108.09118>.
- [60] Daniel Stenberg. *Curl*. <https://curl.se/>. 2022. URL: <https://curl.se/>.
- [61] Mitchell Stuart et al. *Optimization with PuLP*. <https://coin-or.github.io/pulp/>. 2022. URL: <https://coin-or.github.io/pulp/>.

- [62] The PostgreSQL Global Development Group. *PostgreSQL*. 2021.
- [63] Guido Van Rossum and Fred L Drake Jr. *Python reference manual*. Centrum voor Wiskunde en Informatica Amsterdam, 1995.
- [64] Xiaoxuan Wang and Rolf Stadler. *IT Intrusion Detection Using Statistical Learning and Testbed Measurements*. 2024. arXiv: [2402.13081](#) [cs.LG].
- [65] Matei Zaharia et al. “Spark: Cluster Computing with Working Sets”. In: *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*. HotCloud’10. Boston, MA: USENIX Association, 2010, p. 10.